System Design Interview

The complete step-by-step course: from your first design to senior-level architecture

Beginner Part 1 and 2: how systems are built, one idea at a time

Intermediate Part 3 and 4: distributed systems and real-world designs

Advanced Part 5: architecture, operations, security, multi-region

Low-level design Part 6: classes, patterns, and code you can write in an interview

Mastery Part 7: the interview itself, mocks, and self-assessment

Prepared for Abhishek Kumar · October 2026

Contents

Part 1 · Beginner
Lesson 1 · What system design is 8
Lesson 2 · How a request travels 10
Lesson 3 · Scaling and load balancers 12
Lesson 4 · Numbers you must know 14
Lesson 5 · Databases and indexes 16
Lesson 6 · Caching 18
Lesson 7 · Your first design: a URL shortener 21

Part 2 · Beginner to intermediate
Lesson 8 · Replication and failover 26
Lesson 9 · Sharding and consistent hashing 28
Lesson 10 · Consistency in plain words 31
Lesson 11 · Transactions and isolation 33
Lesson 12 · Queues and asynchronous work 35
Lesson 13 · Your second design: a rate limiter 37

Part 3 · Intermediate
Lesson 14 · Leaders, consensus, and fencing 42
Lesson 15 · Timeouts, retries, and circuit breakers 45
Lesson 16 · Idempotency, outbox, and sagas 48
Lesson 17 · Kafka and event streams 51
Lesson 18 · Stream processing, time, and late data 53
Lesson 19 · Your third design: a chat application 55

Part 4 · Designs at scale
Lesson 20 · News feed and fan-out 59
Lesson 21 · Search and typeahead 61
Lesson 22 · Ride matching with geographic cells 63
Lesson 23 · Payments, where correctness comes first 65
Lesson 24 · File storage and duplicate detection 67
Lesson 25 · Metrics and analytics storage 69
Lesson 26 · Video streaming and the CDN 71
Lesson 27 · Job schedulers and workflows 73

Part 5 · Architecture and operations
Lesson 28 · Architecture styles and when to split a system 77
Lesson 29 · CQRS and event sourcing 79
Lesson 30 · Observability and SLOs 81
Lesson 31 · Safe deployment 83
Lesson 32 · Security in design 85
Lesson 33 · Multi-region and disaster recovery 87

Part 6 · Low-level design
Lesson 34 · Design principles (SOLID) and a method 91
Lesson 35 · Patterns you will actually use 93
Lesson 36 · Concurrency in LLD 95
Lesson 37 · Parking lot 96
Lesson 38 · LRU cache 98
Lesson 39 · Vending machine and elevator (state) 100
Lesson 40 · Splitwise (debt simplification) 102
Lesson 41 · Movie ticket booking 104

Part 7 · Mastery
Lesson 42 · The 45-minute method, step by step 108
Lesson 43 · Recovering from mistakes and pressure 110
Lesson 44 · Your resume and behavioural stories 111
Lesson 45 · Mock interviews for each level 113
Lesson 46 · Final self-assessment 115

Appendix
Appendix B · Numbers and formulas cheat sheet 118
Appendix C · Glossary 119

Interview questions

A. Framework and approach
B. Estimation and numbers 3
C. Storage, caching and sharding 3
D. Consistency, transactions and replication 3
E. Messaging, streaming and resilience 3
F. Case-study questions 3
G. Low-level design 3
H. Behavioural and resume 3

Read once, before Lesson 1 · About 10 minutes

Start here: How to use this course

What this course is

This course teaches system design in a fixed order. Every lesson builds on the one before it. You do not need any background in distributed systems to start. By the end you will be able to take an open-ended question such as "Design Instagram" or "Design a rate limiter", ask the right questions, estimate the scale, draw the architecture, and explain the trade-offs under pressure.

The three levels

Level	Who it is for	What you will be able to do
Beginner (Parts 1–2)	You have built apps but never designed a large system.	Explain requests, scaling, databases, caching, queues. Design a small system end to end.
Intermediate (Parts 3–4)	You know the basics and want to handle failures and scale.	Reason about replication, consensus, retries, idempotency, streams. Design feeds, payments, storage, and video.
Advanced (Part 5)	You are preparing for senior or staff-level rounds.	Choose architecture styles, plan operations, security, multi-region, cost.
Low-level design (Part 6)	Coding-round candidates.	Model classes, apply patterns, write working code with concurrency in mind.
Mastery (Part 7)	Everyone, before the interview.	Run the 45-minute interview clock, handle mistakes, tell your project stories, pass the final check.

How each lesson is laid out

- **Goal**: what you will be able to do at the end.
- **The idea in plain words**: an explanation with an everyday analogy.
- **Figure or code**: a diagram, or a program you can run and its real output.
- **Step by step**: the mechanism, slowly.
- **Complete interview coverage**: what the interviewer checks, a full model answer, follow-up questions, and mistakes that cost points.

> **Key idea.** Do not move on until you can give the model answer out loud in your own words. If you cannot, reread the lesson once, then come back to it in the final review.

The four boxes you will see

> **Plain words.** A simple explanation or analogy. Read this first if a term is new.

> **Key idea.** The one thing to remember from the lesson.

> **Common mistake.** A trap that interviewers see often.

> **Interview tip.** What to say, and how, when this topic comes up.

Suggested pace

Your situation	Plan
Learning from scratch	One lesson a day for about 6 weeks, then Parts 6 and 7 for 2 weeks.
Some experience	Parts 2–5 in 3 weeks, Part 6 in 1 week, Part 7 in 1 week.
Interview in 1 week	Lesson 1, the framework in Lesson 38, the cheat sheets in the Appendix, and two mock interviews in Part 7.

Practice rule. Draw on paper. Say your answer out loud. Time yourself. Reading alone feels like learning but is not enough.

PART 1 · BEGINNER

The Foundations

How requests, servers, databases, and caches fit together. You will design your first system.

- Lesson 1: What system design is
- Lesson 2: How a request travels
- Lesson 3: Scaling and load balancers
- Lesson 4: Numbers you must know
- Lesson 5: Databases and indexes
- Lesson 6: Caching
- Lesson 7: Your first design: a URL shortener

Level: Beginner · About 20 minutes

Lesson 1: What system design is

Goal

Understand what a system design interview tests, and the structure every answer should follow.

The idea in plain words

> **Plain words.** Coding interviews ask "write a function that does X." System design interviews ask "build a service that serves millions of people." The question is vague on purpose. The interviewer wants to see how you turn a vague goal into a clear plan, and how you think about what can go wrong.

A system design answer is a conversation, not a speech. You ask questions, make assumptions explicit, draw a picture, go deeper where it matters, and talk about trade-offs.

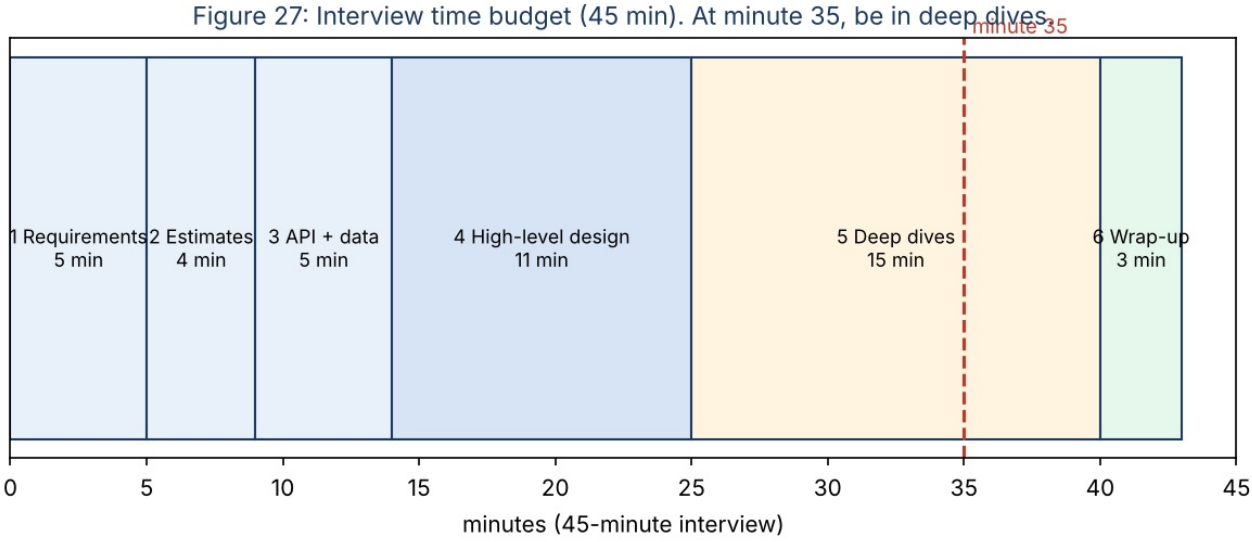
What interviewers score

- **Requirements:** did you clarify what to build before building it?
- **Estimation:** do you know roughly how big the problem is?
- **Design:** is the picture clear, with each part having a job?
- **Depth:** can you explain how one hard part works?
- **Trade-offs:** do you name what you gave up?
- **Communication:** is your reasoning easy to follow?

Step by step: the six steps you will use every time

1. **Requirements** (about 5 minutes): what must the system do, and what must it never do?
2. **Estimates** (about 4 minutes): how many users, requests per second, data size?
3. **API and data** (about 5 minutes): what calls does a user make, and what do we store?
4. **High-level design** (about 11 minutes): boxes and arrows for the main flow.
5. **Deep dives** (about 15 minutes): pick the hardest 2 or 3 parts and go deep.
6. **Wrap-up** (about 3 minutes): what can fail, how we would monitor it, what we would build next.

Figure 27: Interview time budget (45 min). At minute 35, be in deep dives.



The 45-minute clock. By minute 35 you should be in the deep dives.

> **Common mistake.** Starting to draw boxes in the first minute. Without requirements you are designing for a system nobody asked for.

> **Interview tip.** Open with: "Before I design, I'd like to clarify a few requirements. Then I'll estimate scale and sketch the main flow." Interviewers relax when you have a plan.

Going deeper

Interviewers grade the process as much as the design. A strong opening has three parts: the question you clarify, the assumption you state when the answer is missing, and the number you commit to.

Ask about	Why it changes the design
Users and scale (daily active users, peak)	Decides whether one server is enough
Read to write ratio	Decides whether to invest in caching or in write throughput
Latency target	Decides whether a cache or a precomputed result is needed
Consistency need	Decides between a strong and an eventual store
Availability target	Decides on replication and multi-region work
Out of scope	Protects the 45 minutes from features nobody asked for

Example phrasing: "I'll assume 100 million daily users, a 100 to 1 read to write ratio, and reads under 200 ms. Tell me if any of these is wrong." Stating the assumption out loud is what the interviewer is listening for.

Interview follow-up

- **Q: What if the interviewer gives no numbers?** A: State assumptions, label them, and show how the design changes if they are off by 10 times.
- **Q: Should you draw first or talk first?** A: Talk first for two or three minutes, then draw. A diagram without requirements looks like a guess.

Complete interview coverage

What the interviewer checks: that you clarify before designing, state assumptions, and manage time out loud.

A complete answer you should be able to give: Start by asking about users, features in scope, scale, latency, availability and consistency, then write the agreed list down. State the assumptions you make for anything unanswered. Then sketch a plan for the 45 minutes so the interviewer can follow your structure.

Follow-up questions and answers:

- **Q: Should I start drawing immediately?**
  A: No. Two to three minutes of questions first, then draw. A diagram without requirements looks like a guess.
- **Q: What if the interviewer changes the requirement mid-way?**
  A: Acknowledge it, identify which components change, and update the diagram rather than restarting.

Mistakes that cost points: Jumping into boxes and arrows; ignoring non-functional requirements; never saying the assumptions aloud.

Level: Beginner · About 25 minutes

Lesson 2: How a request travels

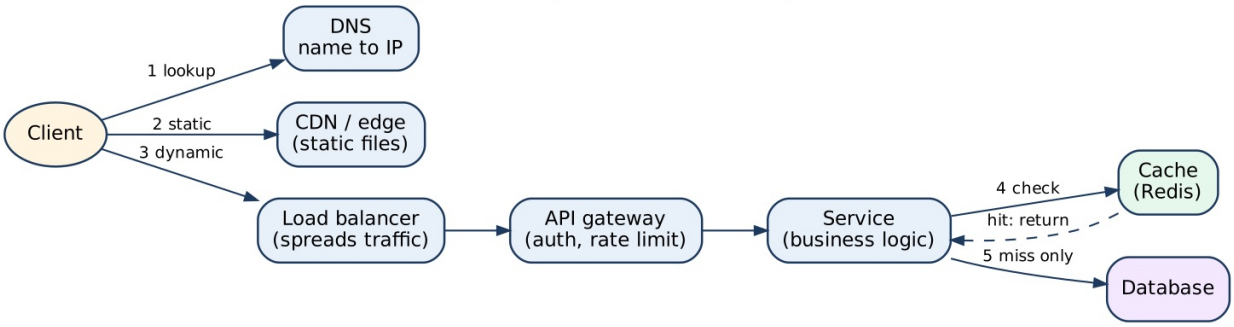
Goal

Trace what happens from the moment a user taps a button to the moment they see a result.

The idea in plain words

> **Plain words.** Think of ordering food. You look up the restaurant's address (DNS), you walk to the counter (load balancer), the cashier takes your order (the server), the kitchen checks if the dish is already prepared (the cache), and only if not, it cooks from the fridge (the database).

Figure 1: The path of one web request



One request, left to right. The cache is checked before the database.

Step by step

1. **DNS:** the phone turns a name like `app.example.com` into an IP address.
2. **Connection:** the phone opens a secure connection (TCP and TLS). This takes a few round trips before any data moves.
3. **Load balancer:** it picks one of many servers for this request.
4. **Application server:** runs your business logic.
5. **Cache:** a fast memory store is checked first.
6. **Database:** only used when the cache does not have the answer.
7. **Response:** the result travels back the same way.

> **Key idea.** Each step adds time. A round trip across a continent is about 150 ms. A database query might be 5–20 ms. A memory lookup is about 0.0001 ms. Good design keeps the slow steps off the hot path.

Going deeper

Knowing the rough cost of each hop lets you explain why caching or co-locating data helps. The orders of magnitude matter more than exact values.

Operation	Rough time
Read from CPU cache	under 1 ns
Read from main memory	about 100 ns
Random read from SSD	about 100 microseconds
Round trip inside one data centre	about 0.5 ms
Round trip across a continent	about 100 ms
Round trip across an ocean	about 150 ms

Worked example. A page makes 10 sequential database calls at 2 ms each. Latency is 20 ms. If the 10 calls are independent and run in parallel, latency drops to about 2 ms plus overhead. Explaining this shows you think about the critical path.

Connection cost. A new HTTPS connection costs one TCP handshake plus a TLS handshake, which is several round trips. That is why connection reuse (keep-alive) and HTTP/2 multiplexing matter, and why a long-lived connection such as WebSocket is cheap per message.

Interview follow-up

- **Q: Why is the DNS lookup slow sometimes?** A: A cold lookup walks through resolvers and can take tens of milliseconds. Caching at the client and resolver removes most of that cost.
- **Q: How does a CDN help latency?** A: It shortens the distance: the round trip becomes the distance to the nearest edge, not to the origin.

Complete interview coverage

What the interviewer checks: that you can follow one request through the system and reason about where latency and failure occur.

A complete answer you should be able to give: A request goes from the client through DNS to the nearest edge or load balancer, reaches a stateless service, which reads from a cache first and then a database, and returns the response. Each hop adds latency and a failure point, so you add timeouts and caching where the cost is highest. Naming the hop where the design is most fragile shows real understanding.

Follow-up questions and answers:

- **Q: Where is the first place a request can fail?**
  A: DNS resolution, the connection setup, or the load balancer. Mention health checks and retries for each.
- **Q: How do you reduce the number of hops?**
  A: Co-locate services, cache responses at the edge, and batch calls where the product allows.

Mistakes that cost points: Describing the path without naming latency or failure at each hop.

Level: Beginner · About 25 minutes

Lesson 3: Scaling and load balancers

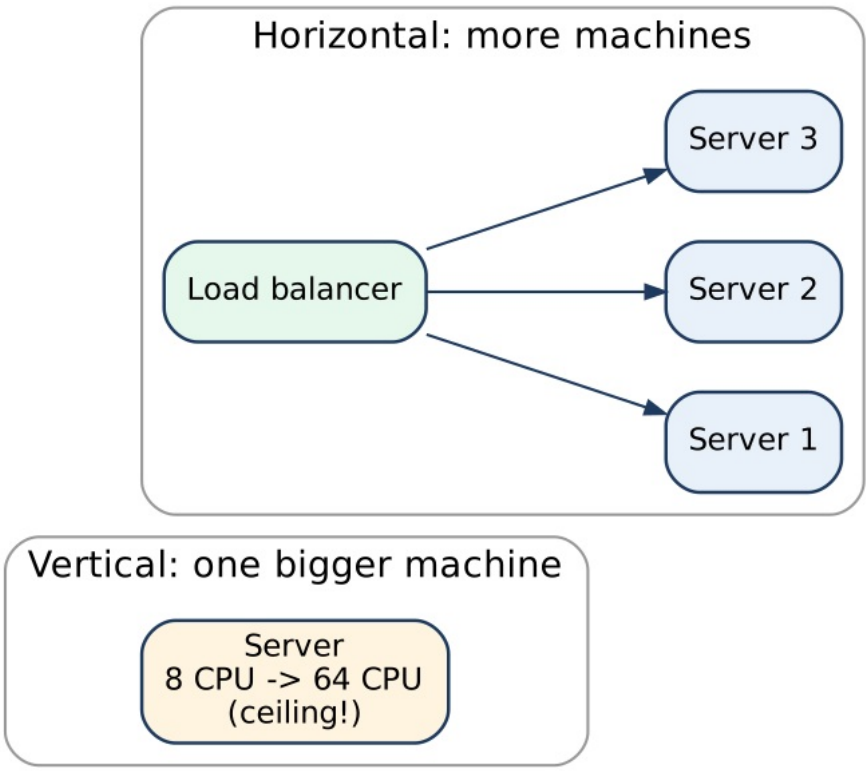
Goal

Know the two ways to scale, and why a load balancer is the first box you add when one server is not enough.

The idea in plain words

> **Plain words.** A small shop with one cashier works until the queue is long. You can hire a faster cashier (vertical scaling), or open more tills and put a person at the door to send customers to the shortest queue (horizontal scaling plus a load balancer).

Figure 2: Vertical vs horizontal scaling



Vertical: one bigger machine. Horizontal: many machines behind a load balancer.

Vertical vs horizontal

	Vertical (scale up)	Horizontal (scale out)
What you do	Buy a bigger machine	Add more machines
Pros	Simple. No code changes.	No hard ceiling. Survives one machine failing.
Cons	A ceiling. One failure takes everything down.	Needs routing, and shared state must be handled.

What a load balancer does

1. Receives every incoming request.
2. Picks a healthy server using an algorithm (for example, round robin, or least connections).
3. Stops sending traffic to a server that fails its health check.

Key idea. Make your application servers stateless: they should not keep user session data in their own memory. Then any server can handle any request, and you can add or remove servers freely.

Common mistake. Saying "I'll scale horizontally" and stopping there. Always say what becomes shared (the database, the cache, sessions) and how you handle it.

Going deeper

Vertical vs horizontal scaling. Vertical scaling buys a bigger machine: simple, but there is a ceiling and a single point of failure. Horizontal scaling adds machines: it needs stateless services, a load balancer and a plan for shared state.

Load balancer algorithm	Good when
Round robin	Requests cost about the same
Least connections	Requests vary in length
Hash on a key (for example user ID)	You need sticky routing to a cache or session
Weighted	Machines differ in size

Layer 4 vs Layer 7. An L4 balancer routes on IP and port: fast and simple. An L7 balancer reads HTTP: it can route by path, header or cookie, and terminate TLS, at more CPU cost.

Health checks. A balancer removes an unhealthy server only after several failed checks, and adds it back only after several passes. Without this, a flapping server causes errors for many users.

Interview follow-up

- **Q: Why must servers be stateless?** A: So any server can take any request, and a crashed server loses no user data. Session state moves to a shared store.
- **Q: What is the load balancer's own failure mode?** A: It becomes a single point of failure. Run a pair with failover, or use DNS and anycast across several.

Complete interview coverage

What the interviewer checks: that you know how to scale stateless services and what load balancing requires.

A complete answer you should be able to give: Scale the stateless application tier horizontally behind a load balancer with health checks, and move session state into a shared store so any server can take any request. Choose the balancing algorithm by request cost, and use Layer 7 routing where you need path or header decisions. Plan for the balancer itself failing, usually with a redundant pair.

Follow-up questions and answers:

- **Q: Why not just buy a bigger server?**
  A: Vertical scaling has a ceiling and a single point of failure; keep it for the database if needed, but scale the services horizontally.
- **Q: What if the load balancer fails?**
  A: Run an active-passive pair or several balancers behind DNS or anycast.

Mistakes that cost points: Saying 'add more servers' without saying where state lives.

Level: Beginner · About 30 minutes

Lesson 4: Numbers you must know

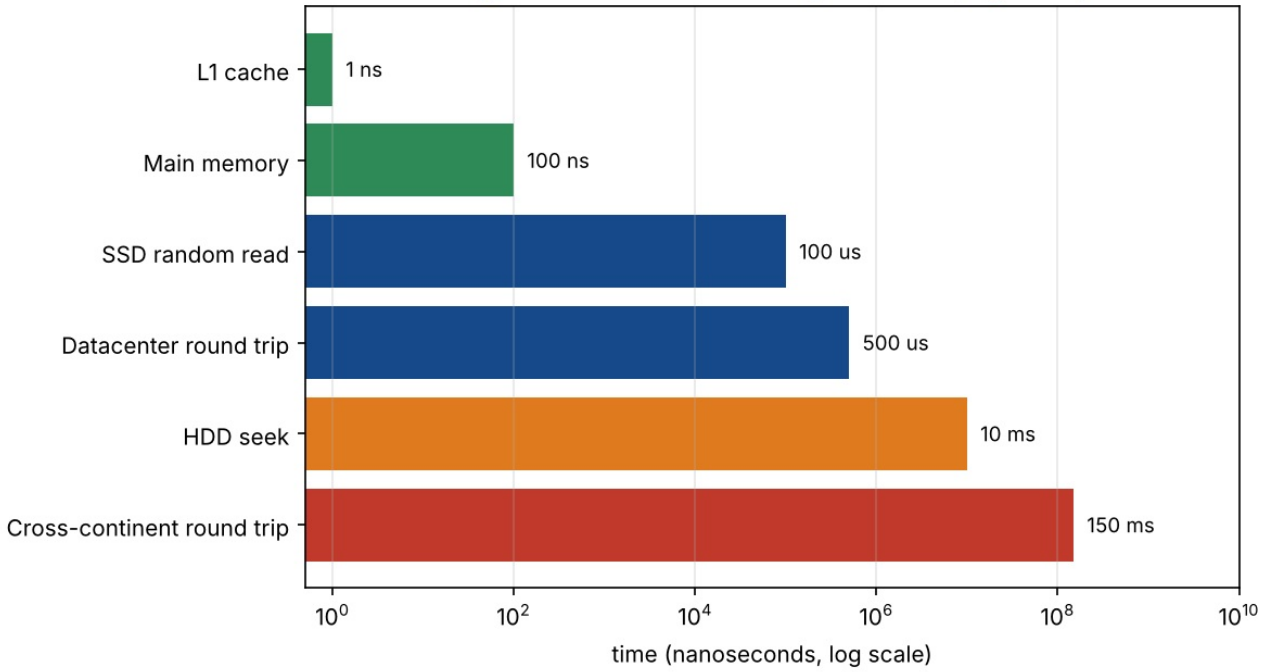
Goal

Estimate traffic, storage, and bandwidth in two minutes, and know how fast each kind of operation is.

The idea in plain words

> **Plain words.** You do not need exact numbers. You need the right order of magnitude. Is it 10 requests per second or 10,000? That difference decides the whole design.

Figure 25: How slow each operation is (log scale)



How slow each operation is. Each step is roughly 10x to 1000x slower than the one before.

The numbers

Thing	Value to remember
Seconds per day	About 86,400. Use 10^5 for quick math.
Memory access	About 100 nanoseconds
SSD random read	About 100 microseconds
Round trip inside a data center	About 0.5 ms
Round trip across continents	About 150 ms

Estimation, step by step

Example: a social app with 10 million daily users, each opening the app 5 times a day, each open making 10 requests.

1. Daily requests: 10M × 5 × 10 = 500 million.
2. Average per second: 500M ÷ 10^5 ≈ 5,000 requests per second.
3. Peak: assume 3× average, so about 15,000 per second.
4. Say it out loud: "About 5,000 average, about 15,000 at peak. That tells me I need several servers and a cache."

Key idea. Always compute an average, then a peak (usually 2 to 3 times the average), and say what each number means for the design.

Common mistake. Giving a precise-looking number such as "4,167 requests per second" with no reason. Round it. Interviewers want the reasoning, not false precision.

Going deeper

Estimation is a sequence of small multiplications. Round aggressively and keep units visible.

Quantity	Value to remember
Seconds per day	86,400 (about 10 to the power 5)
Seconds per year	about 3 times 10 to the power 7
1 million per day	about 12 per second
1 billion per day	about 12,000 per second
Storage of 1 KB times 1 million	1 GB
Storage of 1 KB times 1 billion	1 TB
Bandwidth of 1 MB per second	about 8 Mbit per second

Worked example. 500 million messages a day, each 200 bytes, kept 5 years. Writes: 500M / 86,400 is about 5,800 per second. Storage per day: 500M times 200 bytes is 100 GB. Five years: about 180 TB before replication. With 3 replicas, about 540 TB. Stating the replication factor is a common expected detail.

Interview follow-up

- **Q: Why compute peak as well as average?** A: Systems are sized for peak, not average. A 2 to 3 times multiplier is a common default unless the prompt gives a pattern.
- **Q: How do you estimate memory for a cache?** A: Take the hot working set (often 20 percent of items cause most reads) times the item size.

Complete interview coverage

What the interviewer checks: that you can turn product numbers into QPS, storage and bandwidth quickly and state the assumptions.

A complete answer you should be able to give: Write the assumptions, multiply daily users by actions per user, divide by 86,400 for average QPS, and apply a 2 to 3 times peak factor. Compute storage per record times records per day times retention, then multiply by the replication factor. Bandwidth follows from QPS times size. Say which number drives the design, such as peak QPS deciding the number of servers.

Follow-up questions and answers:

- **Q: Why use 86,400 seconds if you can round to 100,000?**
  A: Rounding is fine for speed, as long as you say you rounded and the error is small.
- **Q: Which estimate matters most?**
  A: Peak QPS for sizing servers and cache, and storage growth for the database plan.

Mistakes that cost points: Doing arithmetic silently; forgetting replication and peak; not saying the units.

Level: Beginner · About 30 minutes

Lesson 5: Databases and indexes

Goal

Choose between SQL and NoSQL at a basic level, and understand why an index makes reads faster but writes slower.

The idea in plain words

Plain words. A database is a filing cabinet. An index is the card catalogue at the front of a library. Without it, finding a book means checking every shelf. With it, you go straight to the right shelf, but each new book needs a catalogue card too.

SQL vs NoSQL

	SQL (PostgreSQL, MySQL)	NoSQL (Cassandra, DynamoDB, MongoDB)
Shape of data	Tables with fixed columns	Flexible: key-value, documents, wide columns
Strength	Joins, transactions, constraints	Very high scale, simple access by key
Choose when	Relationships and correctness matter (orders, money, users)	Massive writes, known access pattern (events, sessions, timelines)

How an index works

1. An index is a sorted copy of one or more columns.
2. Each entry points to the full row.
3. A search on the indexed column becomes a fast lookup instead of a full scan.
4. Every insert or update must also update the index, so writes cost more.

Key idea. A composite index on (city, age) helps "city = Pune" and "city = Pune and age = 30". It does not help "age = 30" alone, because the data is sorted by city first. Like a phone book sorted by surname, then first name.

Common mistake. Adding an index to every column. Each index slows writes and takes storage. Index the columns your queries actually filter on.

Going deeper

Why indexes make reads fast. Without an index a query scans every row. A B-tree keeps keys sorted in pages of several hundred entries, so a lookup takes about 3 or 4 page reads even for hundreds of millions of rows.

Costs of an index. Each index is updated on every write, takes storage, and can slow inserts. Index only what the queries need.

Index type	Example	Use
Single column	index on email	Equality lookup
Composite	(country, city, created_at)	Queries that filter on a left-most prefix
Covering	index includes every selected column	Avoids reading the table
Partial	index only active rows	Smaller and faster for a common subset

Left-most prefix rule. An index on (country, city) helps a query filtering on country, or on country and city. It does not help a query filtering only on city.

SQL vs NoSQL. Choose SQL when you need joins, transactions across rows, and flexible queries. Choose a key-value or wide-column store when access is by a known key at very large scale and the access pattern is fixed in advance.

Interview follow-up

- **Q: Why is a LIKE with a leading wildcard slow?** A: The index cannot be used from the left, so the database scans. A trailing wildcard ("abc%") can use the index.
- **Q: Why are inserts slower with many indexes?** A: Each index must be updated and its pages written, so write cost multiplies by the number of indexes.

Complete interview coverage

What the interviewer checks: that you can choose a database and index for a given access pattern and explain the cost of indexes.

A complete answer you should be able to give: Choose SQL when you need joins, transactions across rows or ad hoc queries, and NoSQL when access is by a known key at large scale. Add an index for each frequent query, put the most selective equality columns first in a composite index, and remember every index slows writes. Mention B-trees for reads and LSM trees for write-heavy stores.

Follow-up questions and answers:

- **Q: Why does a leading wildcard search not use an index?**
  A: The index is sorted from the left, so the database cannot seek a prefix; use a search engine or a trailing wildcard.
- **Q: How do you index for a query that filters on two columns?**
  A: Use a composite index on the equality column first, then the range column.

Mistakes that cost points: Adding indexes for every column; not saying how the index matches the query.

Level: Beginner · About 30 minutes

Lesson 6: Caching

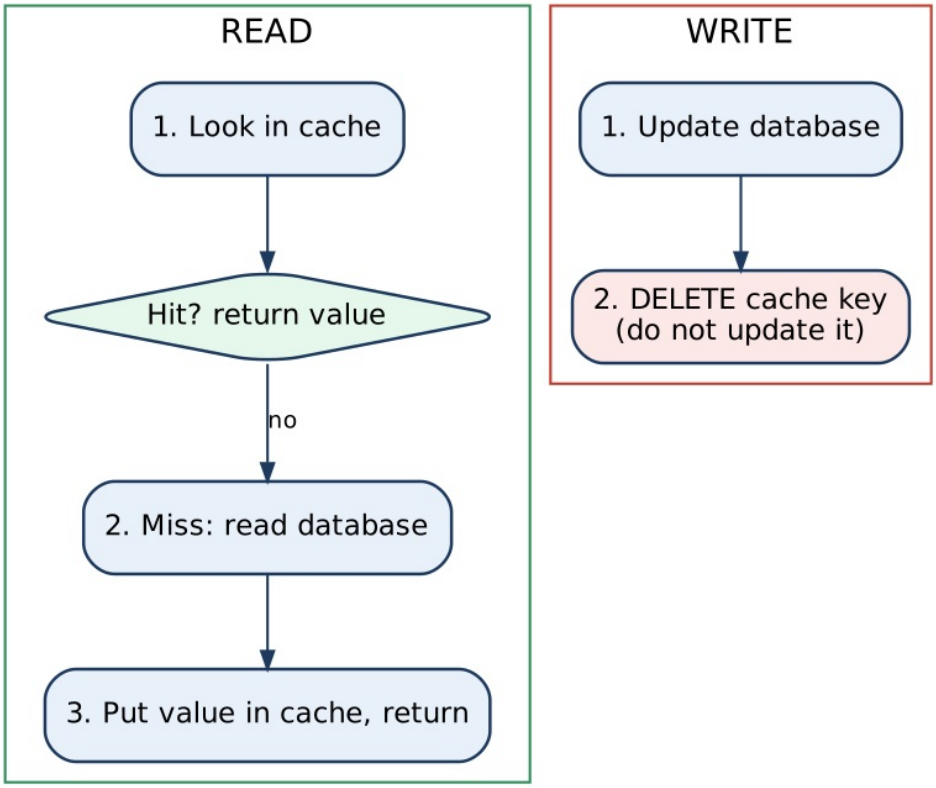
Goal

Use the cache-aside pattern correctly, and calculate how much a cache improves average latency.

The idea in plain words

> **Plain words.** A cache is the notes you keep on your desk. Looking something up in your own notes is faster than walking to the archive. The risk: the notes can go out of date.

Figure 12: Cache-aside, reads and writes



Cache-aside. Reads check the cache first. Writes update the database, then delete the cache entry.

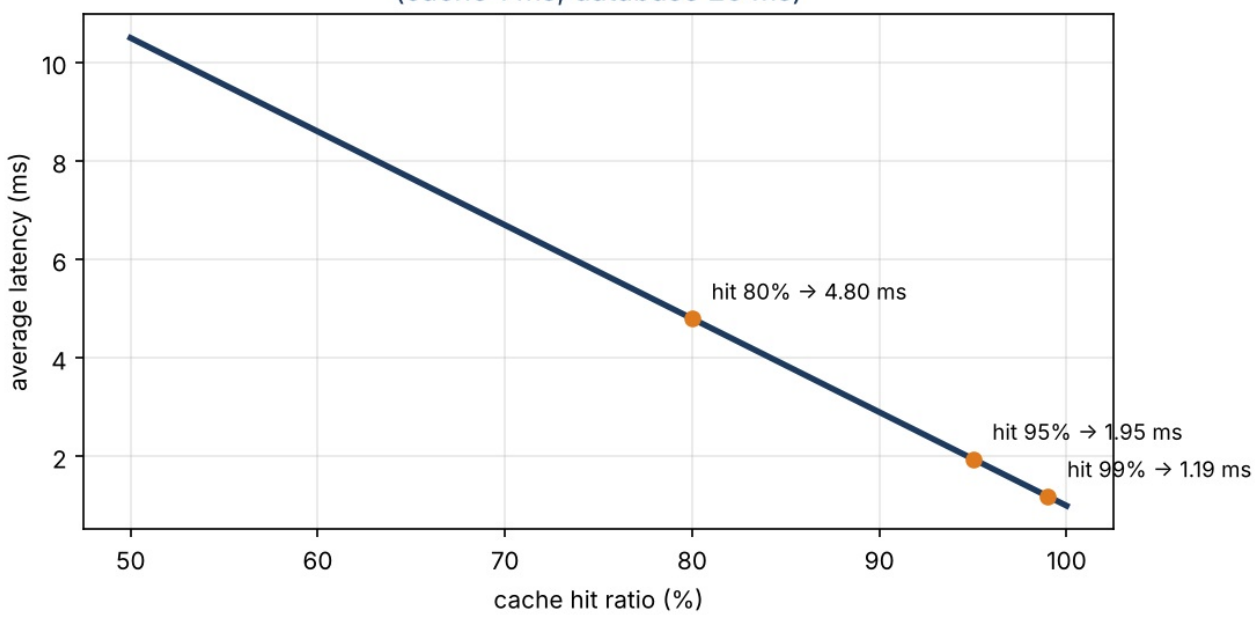
The cache-aside pattern

1. **Read:** check the cache. If found (a hit), return it.
2. **Read miss:** read the database, put the value in the cache, return it.
3. **Write:** update the database first, then delete the cache entry.

Why delete instead of update on write

Two requests can race. If you update the cache, an old value can end up in it and stay there. If you delete it, the next read fetches the fresh value from the database.

Figure 26: Cache hit ratio drives average latency
(cache 1 ms, database 20 ms)



Average latency with a 1 ms cache and a 20 ms database. Small drops in hit ratio cost a lot.

Hit ratio math

Average latency = hit ratio × cache time + (1 − hit ratio) × database time.

With a 95% hit ratio: 0.95 × 1 + 0.05 × 20 = 1.95 ms. With 80%: 0.8 × 1 + 0.2 × 20 = 4.8 ms. The database is only 20× slower, yet the average more than doubles.

Key idea. The hit ratio is the most important number for a cache. Protect it.

Common mistake. Caching everything with no expiry. Stale data will hurt users. Set a time-to-live (TTL) and say how stale the data may be.

Going deeper

Pattern	How it works	Trade-off
Cache-aside	App checks cache, reads database on miss, fills cache	Simple; cache can be stale until expiry or delete
Read-through	Cache loads from the database itself on miss	Cleaner app code; cache library must know the source
Write-through	Every write goes to cache and database together	Fresh reads; slower writes
Write-behind	Write to cache, flush to database later	Fast writes; data loss risk if the cache crashes

Eviction. LRU is the common default. LFU keeps frequently used items even if they were not touched recently, which suits stable popularity. TTL removes items after a time, which bounds staleness.

Cache stampede. A popular key expires, and thousands of requests miss at once and hit the database. Fixes: single-flight (one request recomputes, others wait), a short lock, or refresh before expiry. Adding random jitter to TTLs stops many keys expiring together.

Hit ratio arithmetic. With cache time 1 ms, database time 20 ms, and hit ratio 0.9, average latency is 0.9 times 1 plus 0.1 times 20, which is 2.9 ms. Going from 0.9 to 0.99 drops it to 1.19 ms. The last few percent of hit ratio matter a lot.

Interview follow-up

- **Q: Why delete the cache key on write instead of updating it?** A: Two concurrent writers can leave the cache holding the older value if updates race. Deleting forces the next read to load the current value.
- **Q: How do you cache a user profile that changes often?** A: Short TTL plus delete on write, and accept

a few seconds of staleness if the product allows it.

Complete interview coverage

What the interviewer checks: that you can add a cache, choose an invalidation method, and explain hit ratio and stampede.

A complete answer you should be able to give: Add a cache in front of the database for read-heavy keys, using cache-aside with a TTL. On write, update the database and delete the cache key so the next read reloads the current value. Protect against stampede with single-flight and TTL jitter. Quantify the benefit with the hit ratio formula so the trade-off is visible.

Follow-up questions and answers:

- **Q: Why not update the cache on write?**
  A: Concurrent updates can leave the older value in the cache; deleting is safer.
- **Q: What is the cost of caching?**
  A: Staleness within the TTL, extra memory, and the cache becoming a dependency that must be made highly available.

Mistakes that cost points: Caching without an invalidation plan; forgetting the stale window.

Level: Beginner · About 45 minutes · Practice lesson

Lesson 7: Your first design: a URL shortener

Goal

Run the full six-step framework on a real question, from start to finish.

The question

"Design a URL shortener like bit.ly. A user gives a long link and gets a short one. Visiting the short link redirects to the long one."

Step 1: Requirements

- Create a short link from a long URL.
- Redirect a short link to the original URL.
- Optional: custom alias, expiry date, click count.
- Redirects must be fast (under 50 ms at p99). Redirects are much more common than creates.
- Out of scope for now: user accounts, malware scanning.

Step 2: Estimates

- 100 million new links per month: about 100M ÷ 2.6M seconds ≈ 40 writes per second, peak about 120.
- Read to write ratio 100 to 1: about 4,000 reads per second on average, peak about 12,000.
- Storage: 500 bytes per link × 100M per month ≈ 600 GB per year.

Step 3: API and data

- `POST /urls` with a long URL, returns a short code.
- `GET /{code}` returns a redirect.
- Data: a key-value table keyed by the short code, with the long URL and an expiry.

Step 4: Making the short code

Use a counter, not a hash of the URL. A hash can collide, and the same URL would always produce the same code, which leaks information. A unique number encoded in base 62 (letters, digits) gives $62^7 \approx 3.5$ trillion codes with only seven characters. Servers take blocks of 10,000 numbers from a counter service, so they do not ask for a number on every request.

One way to get unique numbers without asking a central counter for each one is a Snowflake-style ID: a timestamp, a machine number, and a sequence number packed into one 64-bit value. Here is a small version:

Code (runs with Python 3, standard library only)

```
import threading as _th

class Snowflake:
    EPOCH = 1700000000000  # ms
    def __init__(self, machine_id: int):
        self.machine = machine_id & 0x3FF   # 10 bits
        self.seq = 0
        self.last = -1
        self.lock = _th.Lock()

    def next_id(self, now_ms: int) -> int:
        with self.lock:
            if now_ms == self.last:
                self.seq = (self.seq + 1) & 0xFFF   # 12 bits
                if self.seq == 0:
                    raise RuntimeError("sequence exhausted this ms; wait")
            else:
                self.seq = 0
            self.last = now_ms
            return ((now_ms - self.EPOCH) << 22) | (self.machine << 12) | self.seq

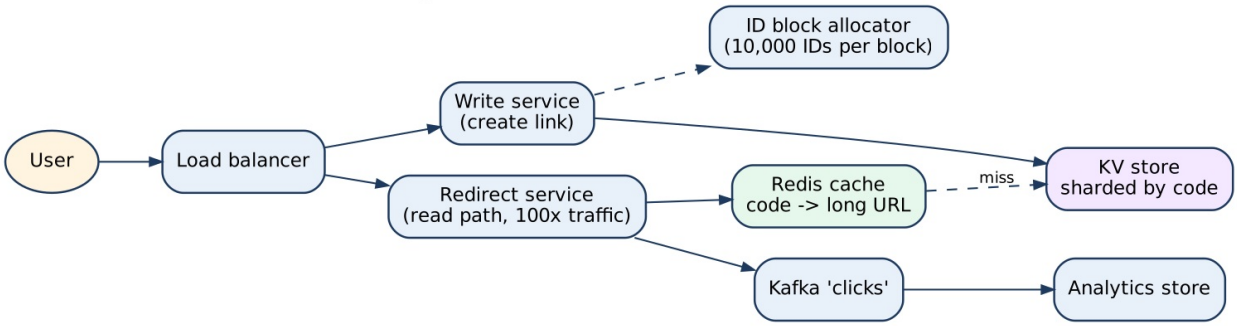
gen = Snowflake(machine_id=7)
ids = [gen.next_id(1700000001000) for _ in range(3)] + [gen.next_id(1700000001001)]
print("ids:", ids)
print("strictly increasing:", all(a < b for a, b in zip(ids, ids[1:])))
```

Output when you run it:

```
ids: [4194332672, 4194332673, 4194332674, 4198526976]
strictly increasing: True
```

Step 5: High-level design

Figure 17: URL shortener architecture



URL shortener: redirects are served from the cache, clicks go to a stream for analytics.

Step 6: Deep dives and wrap-up

- **Hot link:** one link gets 50,000 requests per second. Add a small in-memory cache on each redirect server.
- **Expiry:** check the expiry on read. Clean up expired links in batches later, not by scanning the whole table.
- **Failure:** if the cache is down, reads fall through to the database and become slower but still work.
- **Analytics:** send click events to a queue and count them later. Redirects never wait for counting.

> **Key idea.** Separate the read path from the write path. Reads are 100 times more common, so they get the cache and the most servers.

Going deeper

Code generation options.

Approach	Pros	Cons
Hash the long URL, take first 7 characters	Same URL gives same code	Collisions must be checked and retried
Global counter, encode in base 62	No collisions	Counter is a bottleneck; codes are guessable
Snowflake-style ID, encode in base 62	Unique across machines, no coordination per write	Slightly longer codes

Capacity. Seven base-62 characters give 62 to the power 7, about 3.5 trillion codes. A 100 million links per year service uses about 0.01 percent of that in a decade.

301 vs 302. A 301 (permanent) redirect lets browsers cache it, so later clicks skip your server and your analytics miss them. A 302 (temporary) keeps every click on your server, costing more load but giving accurate counts. State the choice and its reason.

Interview follow-up

- **Q: How do you handle a custom alias that is already taken?** A: A unique constraint on the alias, and the insert returns a conflict the API reports as an error.
- **Q: What about expired links?** A: Store an expiry, check it on redirect, and run a background job to delete old rows.

Complete interview coverage

What the interviewer checks: that you can produce a complete small design end to end, including unique

IDs and redirects.

A complete answer you should be able to give: Accept a long URL, generate a unique short code with a Snowflake-style ID encoded in base 62, store the mapping, and return the short URL. For redirects, look up the code, use a cache for hot codes, and return a redirect. Choose 301 or 302 based on whether you need click analytics. Address expiry and abuse as follow-ups.

Follow-up questions and answers:

- **Q: How do you avoid collisions?**
  A: Unique IDs from a generator rather than random codes, or a unique constraint with retry.
- **Q: How do you scale reads?**
  A: Cache by code, and serve redirects from the edge where possible.

Mistakes that cost points: Using random codes without a uniqueness check; forgetting the redirect code choice.

PART 2 · BEGINNER TO INTERMEDIATE

Data and Reliability

How data is copied, split, and kept consistent, and how to protect a system from too much traffic.

- Lesson 8: Replication and failover
- Lesson 9: Sharding and consistent hashing
- Lesson 10: Consistency in plain words
- Lesson 11: Transactions and isolation
- Lesson 12: Queues and asynchronous work
- Lesson 13: Your second design: a rate limiter

Level: Beginner to intermediate · About 30 minutes

Lesson 8: Replication and failover

Goal

Explain why data is copied to several machines, how a failover works, and what can be lost in the process.

The idea in plain words

> **Plain words.** Keep copies of an important document in several safe places. If one burns down, another copy is still there. The catch: if you copy the document after every edit, you wait for the copies to finish. If you do not wait, a crash can lose the last edit.

Leader and followers

- One node is the **leader**. It accepts all writes.
- The other nodes are **followers**. They copy the leader's change log and serve reads.
- If the leader dies, a follower is promoted to leader. This is a **failover**.

Synchronous vs asynchronous

	Synchronous	Asynchronous
How the leader replies	Waits for a follower to confirm	Replies right away
Speed	Slower	Faster
Risk on failover	No acknowledged write is lost	The last few acknowledged writes can be lost

> **Key idea.** Asynchronous replication is fast but can lose the most recent writes during failover. Say which one you chose and why. For payments you usually choose synchronous or a quorum; for a like counter, asynchronous is fine.

Step by step: a failover

1. Followers notice the leader has stopped sending heartbeats for longer than a timeout.
2. The most up-to-date follower is chosen as the new leader.
3. Clients are pointed at the new leader.
4. The old leader must be fenced off. If it comes back, it must not accept writes (Lesson 14 explains how).

> **Common mistake.** Saying "we replicate, so we never lose data". Ask: synchronous or asynchronous? What is the window of possible loss?

Going deeper

Replication lag. With asynchronous replication, a follower can be seconds behind. A user who writes and then reads from a follower may see old data. Fixes: read from the leader for that user for a short time, or route by session.

Failover risks. If the leader fails and the follower is promoted before the old leader's last writes arrived, those writes are lost. If the old leader comes back and still accepts writes, you have split brain: two leaders, conflicting data. Consensus (Lesson 14) and fencing prevent this.

Choice	Durability	Latency
Synchronous replica	No acknowledged write is lost on failover	Slower; one slow replica slows writes
Asynchronous replica	Recent writes can be lost	Fast writes
Semi-synchronous (one sync, rest async)	Good balance	Moderate

Interview follow-up

- **Q: How do you choose the new leader?** A: The replica with the most up-to-date log. Automation must check this before promoting.
- **Q: What is multi-leader replication for?** A: Writes near users in several regions. It needs a conflict rule such as last-write-wins or merge.

Complete interview coverage

What the interviewer checks: that you know how replicas stay in sync and what happens when the leader fails.

A complete answer you should be able to give: Use a leader with followers for each shard, replicate the log, and decide between synchronous and asynchronous replication based on the durability need. On failure, promote the most up-to-date follower through an automated process that ensures the old leader is fenced. State the replication lag users may see and how reads handle it.

Follow-up questions and answers:

- **Q: What is split brain?**
  A: Two nodes both think they are leader and accept writes; prevent it with consensus and fencing.
- **Q: Sync or async?**
  A: Sync for no data loss at the cost of latency; async for speed with a loss window.

Mistakes that cost points: Ignoring the failover data-loss window; not mentioning replication lag.

Level: Beginner to intermediate · About 40 minutes

Lesson 9: Sharding and consistent hashing

Goal

Split data across machines, choose a shard key, and explain why consistent hashing moves few keys when servers change.

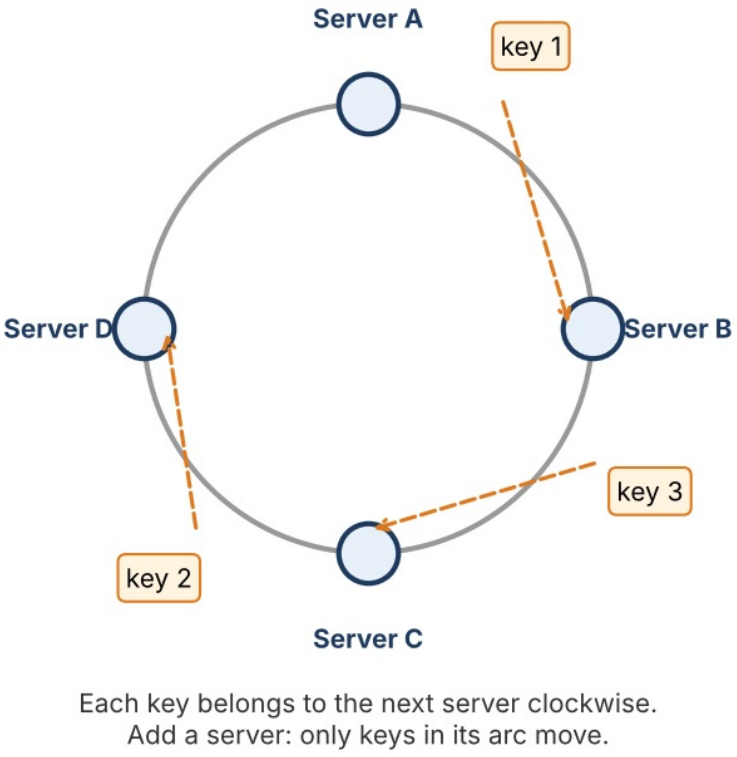
The idea in plain words

> **Plain words.** A library too big for one building is split into branches. You need a rule to know which branch holds which book. Sharding is that split. Consistent hashing is a rule that rarely needs to be rewritten when a branch opens or closes.

Sharding basics

- **Range sharding:** users A–H on shard 1, I–Q on shard 2. Simple, but timestamps all land on the newest shard (a hot spot).
- **Hash sharding:** shard = hash(key) mod N. Even spread, but adding a shard moves almost every key.
- **Consistent hashing:** servers and keys sit on a ring. Adding a server moves only a small share of keys.

Figure 24: Consistent hashing ring



Each key belongs to the next server clockwise. Adding a server only moves keys in its arc.

Why virtual nodes

With only a few servers, one server can end up with a large arc. Each server is placed at many points (for example 100 or 200 virtual nodes) so the load evens out.

Code (runs with Python 3, standard library only)

```
import bisect, hashlib, collections

def h(s: str) -> int:
    return int(hashlib.md5(s.encode()).hexdigest(), 16)

class Ring:
    def __init__(self, vnodes=100):
        self.vnodes = vnodes
        self.keys = []      # sorted hash positions on the ring
        self.owner = {}     # position -> physical node name

    def add(self, node):
        for i in range(self.vnodes):
            p = h(f"{node}#{i}")
            bisect.insort(self.keys, p)
            self.owner[p] = node

    def remove(self, node):
        for i in range(self.vnodes):
            p = h(f"{node}#{i}")
            self.keys.remove(p)
            del self.owner[p]

    def lookup(self, key: str) -> str:
        p = h(key)
        i = bisect.bisect_right(self.keys, p) % len(self.keys)  # next clockwise
        return self.owner[self.keys[i]]

ring = Ring(vnodes=100)
for n in ["A", "B", "C", "D"]:
    ring.add(n)
keys = [f"user:{i}" for i in range(10000)]
before = {k: ring.lookup(k) for k in keys}
print("load per node:", dict(collections.Counter(before.values())))
ring.add("E")
after = {k: ring.lookup(k) for k in keys}
moved = sum(before[k] != after[k] for k in keys)
print(f"after adding E: {moved/len(keys):.1%} of keys moved (ideal ~20%)")
print("all moved keys now live on E:", all(after[k]=="E" for k in keys if before[k]!=after[k]))
```

Output when you run it:

```
load per node: {'B': 2466, 'A': 2577, 'C': 2509, 'D': 2448}
after adding E: 18.9% of keys moved (ideal ~20%)
all moved keys now live on E: True
```

Key idea. In the output, adding server E moved about one fifth of the keys (about 1 in 5 with five servers), and every moved key went to E. Nothing else reshuffled.

Common mistake. Choosing a shard key with uneven traffic, such as a celebrity user ID or a single big tenant. One shard then does most of the work. Pick a key with many distinct values and an even access pattern.

Going deeper

Strategy	Strength	Weakness
Range sharding (A to F, G to M)	Range queries are efficient	Hot ranges (recent timestamps, popular names)
Hash sharding (hash(key) mod N)	Even spread	Changing N moves almost all keys
Consistent hashing	Adding or removing a node moves about 1/N of keys	More complex; uneven without virtual nodes

Why virtual nodes. Each physical server takes many points on the ring (for example 100 to 200). This evens out the load and lets a stronger server take more points.

Hot keys. One celebrity account can overload one shard no matter how keys are hashed. Fixes: split the hot key into sub-keys (counter:1 to counter:10 summed on read), cache it at the edge, or give it a dedicated shard.

Resharding. Move ranges or virtual nodes gradually, dual-write during the move, and verify before cutting over.

Interview follow-up

- **Q: Why not use hash mod N?** A: Adding one server changes N, and nearly every key maps to a new server, causing a cache-wide miss storm.
- **Q: What is a good shard key?** A: High cardinality, even traffic, and present in most queries so you do not scatter-gather.

Complete interview coverage

What the interviewer checks: that you can split data across machines, choose a partitioning scheme, and handle hot keys.

A complete answer you should be able to give: Partition by a shard key with high cardinality and even traffic. Use consistent hashing with virtual nodes so adding a server moves only a fraction of keys. Plan for hot keys by splitting them or giving them dedicated capacity, and describe how you would reshard without downtime by dual-writing and cutting over.

Follow-up questions and answers:

- **Q: Range or hash?**
  A: Range for range queries with careful hot-spot handling; hash for even spread.
- **Q: How do you add a shard?**
  A: Move a portion of keys, dual-write during the move, verify, then switch reads.

Mistakes that cost points: Choosing a shard key that every query must scatter across.

Level: Beginner to intermediate · About 35 minutes

Lesson 10: Consistency in plain words

Goal

Understand quorums, the CAP choice, and the consistency levels you can promise a user.

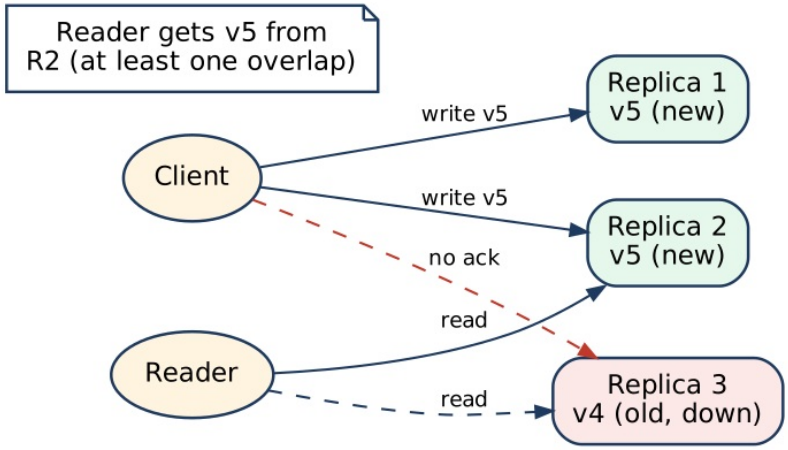
The idea in plain words

Plain words. Two bank branches share a ledger. If the phone line between them breaks, one branch must choose: stop serving customers until the line is fixed (consistent but unavailable), or keep serving with a slightly outdated ledger (available but possibly wrong). That choice is CAP.

Quorums: reading the latest write

Store three copies. A write is confirmed when two copies accept it (W = 2). A read asks two copies (R = 2). Because 2 + 2 is greater than 3, the two copies you read must include at least one copy that has the newest write.

Figure 6: Quorum, N=3, W=2, R=2 (R + W > N, so reads overlap writes)



Three copies, write to two, read from two. Any two reads overlap the newest write.

Key idea. R + W greater than N means reads see the latest confirmed write. Write it on the board: N = 3, W = 2, R = 2 works.

Code (runs with Python 3, standard library only)

```
def quorum_ok(n, w, r):
    return r + w > n

for n, w, r in [(3,2,2), (3,1,1), (5,3,3), (5,2,2)]:
    print(f"N={n} W={w} R={r}: R+W>N is {quorum_ok(n,w,r)}  -> reads see latest write: {quorum_ok(n,w,r)}")
```

Output when you run it:

```
N=3 W=2 R=2: R+W>N is True  -> reads see latest write: True
N=3 W=1 R=1: R+W>N is False  -> reads see latest write: False
N=5 W=3 R=3: R+W>N is True  -> reads see latest write: True
N=5 W=2 R=2: R+W>N is False  -> reads see latest write: False
```

CAP in one sentence

When the network splits, you choose between refusing some requests (consistency) or answering with possibly old data (availability). Without a split, the real trade-off is latency against consistency (PACELC).

Consistency levels to name in interviews

- **Strong:** every read sees the latest write. Needed for balances, inventory, unique usernames.
- **Read-your-writes:** after you post a comment, you see it. Achieve by reading from the leader for a short time after a write.
- **Eventual:** everyone converges later. Fine for like counts and view counts.

> **Common mistake.** "Pick two of CAP forever." CAP only applies during a partition, and most of the time you can have both.

Going deeper

Model	Guarantee	Example
Linearizable	Reads see the latest completed write, in real-time order	Leader lock, account balance check
Read-your-writes	You see your own writes	Profile edit shows immediately
Monotonic reads	Time does not go backwards for you	Comment thread never loses a reply
Eventual	Replicas converge if writes stop	Like counts

CAP in plain words. When a network partition happens, a system must choose: refuse some requests (stay consistent) or answer with possibly stale data (stay available). CAP applies only during partitions. PACELC adds the everyday trade-off: without a partition, you still choose between latency and consistency.

Quorum reminder. With N replicas, W writes and R reads, R + W greater than N guarantees each read overlaps the latest write. Example: N=3, W=2, R=2.

Interview follow-up

- **Q: Which consistency does a bank balance need?** A: Strong for the check-and-debit; eventual is acceptable for the displayed history if it is labelled.
- **Q: Is quorum the same as linearizability?** A: No. Quorum alone can still return stale values under some races; linearizable quorum reads need extra steps such as a read repair or a leader confirmation.

Complete interview coverage

What the interviewer checks: that you can name consistency levels and justify which one each data type needs.

A complete answer you should be able to give: Match consistency to the business need: strong for money and inventory checks, read-your-writes for user profiles, and eventual for counts and feeds. Explain CAP during partitions and PACELC for normal operation. Name how you provide the guarantee, such as leader reads, quorums or version checks.

Follow-up questions and answers:

- **Q: Is quorum linearizable?**
  A: Not automatically; linearizable reads need extra steps such as leader confirmation or read repair.
- **Q: Which consistency for a like counter?**
  A: Eventual, because a slightly stale count is acceptable.

Mistakes that cost points: Calling a whole system 'strongly consistent' without saying which data needs it.

Level: Beginner to intermediate · About 35 minutes

Lesson 11: Transactions and isolation

Goal

Recognize the two classic concurrency bugs (lost update and write skew) and know how to prevent each.

The idea in plain words

> **Plain words.** Two people edit the same shared spreadsheet at the same moment. Without a rule, one person's changes can silently overwrite the other's. A transaction groups changes so they all happen or none do, and isolation decides what each person can see of the other's unfinished work.

Bug 1: lost update

1. Balance is 100. Alice reads 100. Bob reads 100.
2. Alice writes 100 − 30 = 70.
3. Bob writes 100 − 50 = 50. Alice's withdrawal has vanished.

Fix: lock the row while you read and write (SELECT ... FOR UPDATE), or use a version number and retry if it changed.

Bug 2: write skew

Rule: at least one doctor must stay on call. Doctor A and Doctor B are both on call. Both check "is someone else on call?" (yes), and both go off call. Nobody is on call. Each transaction looked fine on its own snapshot.

Fix: serializable isolation, or lock the shared rows that the rule depends on.

Isolation levels in one table

Level	What it prevents	Still allows
Read committed	Reading unfinished data	Lost updates, write skew
Snapshot	Most read anomalies	Write skew
Serializable	Everything	Nothing, but slower

> **Key idea.** Name the anomaly before you name the fix. "This is a lost update, so I'll use a conditional update with a version" is a strong answer.

Going deeper

ACID in short. Atomicity: all or nothing. Consistency: constraints hold. Isolation: concurrent transactions appear to run one at a time, to a degree. Durability: committed data survives a crash.

Anomaly	What goes wrong	Prevented by
Dirty read	See another transaction's uncommitted write	Read committed or higher
Non-repeatable read	Same row read twice, different values	Repeatable read or snapshot
Phantom	Same query returns new rows	Serializable, or predicate locks
Lost update	Two read-modify-write cycles overwrite each other	Atomic update, row lock, version check
Write skew	Both check a rule, then each writes a different row, breaking the rule	Serializable

Example of write skew. Two doctors are on call. Each checks "at least two on call" (both see yes), then each takes themselves off. Now nobody is on call. Snapshot isolation allows this; serializable does not.

Interview follow-up

- **Q: Why not always use serializable?** A: It costs throughput through locking or aborts. Use it for the critical rule and weaker levels elsewhere.
- **Q: How do you avoid lost updates cheaply?** A: A conditional update: UPDATE ... WHERE version = old, and retry if zero rows changed.

Complete interview coverage

What the interviewer checks: that you know which concurrency anomalies matter and how to prevent them.

A complete answer you should be able to give: Name the anomaly a rule could suffer: lost update for counters, write skew for rules spanning rows, phantoms for range checks. Prevent lost updates with atomic conditional updates, and use serializable isolation or explicit locks for multi-row rules. Mention that stronger isolation costs throughput, so use it only where needed.

Follow-up questions and answers:

- **Q: Why not serializable everywhere?**
  A: Locking and aborts reduce throughput; use it for critical rules only.
- **Q: How do you prevent a double debit?**
  A: A conditional update on the balance version, retried on conflict.

Mistakes that cost points: Saying 'use transactions' without naming the anomaly.

Level: Beginner to intermediate · About 30 minutes

Lesson 12: Queues and asynchronous work

Goal

Use a queue to take slow work off the request path, and understand the difference between a queue and a log.

The idea in plain words

> **Plain words.** A restaurant takes your order and gives you a ticket. The kitchen works through tickets at its own pace. You do not wait at the counter while your dish is cooked. A queue is the rail where tickets wait.

Why queues help

- **Speed:** the user gets a fast reply ("order received") while slow work happens later (sending emails, resizing images).
- **Smoothing:** a burst of 10,000 requests becomes a steady stream the workers can handle.
- **Safety:** if a worker crashes, the message is still in the queue and can be retried.

Queue or log?

	Queue (SQS, RabbitMQ)	Log (Kafka)
After a worker reads a message	It is removed	It stays, and other readers can read it too
Replay old messages	Usually no	Yes
Best for	Tasks done once (send email)	Events many teams use (analytics, feeds)

> **Common mistake.** Forgetting that a retried message can arrive twice. Workers must be safe to run twice (Lesson 16 covers this).

Going deeper

Pattern	Behaviour	Use
Work queue	Each message goes to one worker, removed when acknowledged	Send email, resize image
Publish-subscribe	Each subscriber gets a copy	Notify several services of an event
Log (Kafka style)	Messages kept; consumers track their own position	Replay, new consumers, analytics

Acknowledgement and retries. A message is redelivered if the worker crashes before acknowledging. So delivery is at least once, and the handler must be idempotent (Lesson 16).

Dead-letter queue. After a fixed number of failed attempts, move a message aside so one bad message does not block the queue. Someone inspects and replays it later.

Backpressure. If producers outpace consumers, the queue grows. Bound its length, scale consumers, or slow producers. An unbounded queue turns a slowdown into an outage later.

Interview follow-up

- **Q: Does a queue guarantee order?** A: Only within one partition or one consumer. With several consumers on one queue, order across messages is lost.
- **Q: Why use a queue instead of calling the service directly?** A: It decouples timing, absorbs spikes, and lets failed work retry without the caller waiting.

Complete interview coverage

What the interviewer checks: that you can decouple work with a queue and reason about retries, ordering and backlog.

A complete answer you should be able to give: Put slow or unreliable work on a queue so producers do

not wait. Consumers acknowledge after success, so messages are at-least-once and handlers must be idempotent. Use a dead-letter queue for poison messages and bound the queue or scale consumers to control backlog. State which ordering guarantee you need and where it holds.

Follow-up questions and answers:

- **Q: Does a queue keep order?**
  A: Only within one partition or consumer; design the key so related messages share it.
- **Q: What if the queue grows?**
  A: Scale consumers, shed low-priority work, or apply backpressure to producers.

Mistakes that cost points: Forgetting redelivery; assuming global order.

Level: Intermediate · About 50 minutes · Practice lesson

Lesson 13: Your second design: a rate limiter

Goal

Design a rate limiter that protects an API, and implement the two most common algorithms.

The question

"Design a rate limiter that allows each API key a fixed number of requests and returns 429 when the limit is exceeded. It must work across many servers."

Step 1: Clarify

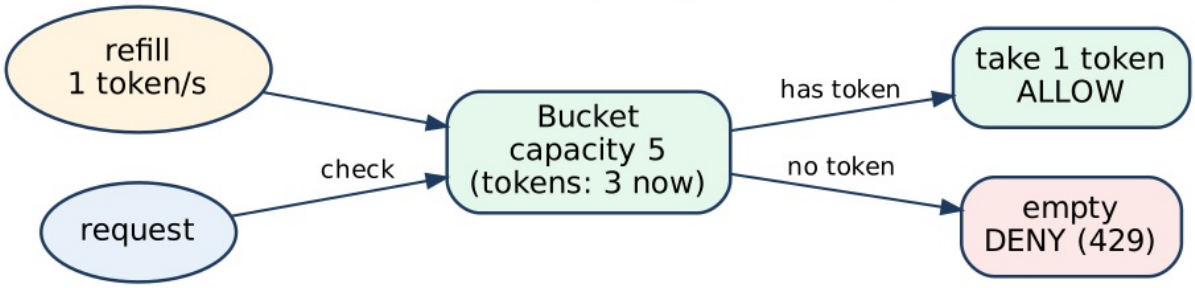
- Limit per API key (and per IP for anonymous traffic).
- Allow short bursts, but keep the long-run average under the limit.
- Latency overhead under 1 ms. Accuracy within a few percent.
- If the limiter store is down: choose fail-open for public read APIs, fail-closed for costly writes. Say which, and why.

Step 2: Algorithm, step by step

Token bucket

> **Plain words.** A bucket holds tokens up to a capacity. Tokens drip in at a steady rate. Each request takes one token. If the bucket is empty, the request is rejected.

Figure 14: Token bucket (capacity 5, refill 1 per second)



Token bucket: refill at a steady rate, take one token per request.

Code (runs with Python 3, standard library only)

```
import time

class TokenBucket:
    def __init__(self, capacity: float, refill_per_sec: float, now=time.monotonic):
        self.capacity, self.rate, self.now = capacity, refill_per_sec, now
        self.tokens, self.last = capacity, now()

    def allow(self, cost: float = 1.0) -> bool:
        t = self.now()
        self.tokens = min(self.capacity, self.tokens + (t - self.last) * self.rate)
        self.last = t
        if self.tokens >= cost:
            self.tokens -= cost
            return True
        return False

# Deterministic demo with a fake clock
clock = [0.0]
tb = TokenBucket(capacity=5, refill_per_sec=1, now=lambda: clock[0])
print("burst of 7 at t=0:", [tb.allow() for _ in range(7)])
clock[0] = 2.0
```

```
print("after 2 seconds, 2 more:", [tb.allow() for _ in range(3)])
```

Output when you run it:

```
burst of 7 at t=0: [True, True, True, True, True, False, False]
after 2 seconds, 2 more: [True, True, False]
```

Sliding window counter

With a limit of 100 per minute, a fixed window lets 200 requests through in two seconds when a burst straddles the minute boundary. A sliding window counter weights the previous window to fix this cheaply.

Code (runs with Python 3, standard library only)

```
class SlidingWindowCounter:
    """Approximates a sliding window using two fixed windows."""
    def __init__(self, limit: int, window: float, now):
        self.limit, self.window, self.now = limit, window, now
        self.curr_idx, self.curr, self.prev = -1, 0, 0

    def allow(self) -> bool:
        t = self.now()
        idx = int(t // self.window)
        if idx != self.curr_idx:
            self.prev = self.curr if idx == self.curr_idx + 1 else 0
            self.curr, self.curr_idx = 0, idx
        elapsed = (t % self.window) / self.window
        estimate = self.prev * (1 - elapsed) + self.curr
        if estimate < self.limit:
            self.curr += 1
            return True
        return False

clock = [0.0]
sw = SlidingWindowCounter(limit=10, window=60, now=lambda: clock[0])
clock[0] = 59.0
print("first 10 in window 0 ->", sum(sw.allow() for _ in range(10)))
clock[0] = 61.0
print("at t=61 (just after boundary), allowed:", sum(sw.allow() for _ in range(10)), "of 10 (previous window still weighs
in)")
```

Output when you run it:

```
first 10 in window 0 -> 10
at t=61 (just after boundary), allowed: 1 of 10 (previous window still weighs in)
```

Step 3: Where it runs and how it scales

- Run the check in the API gateway, so every service is protected in one place.
- Store counters in a shared fast store (Redis). Use an atomic script so two servers cannot both take the last token.
- Read the server's clock inside the script, so skewed clocks on app servers cannot grant extra tokens.

> **Key idea.** The check must be atomic. Read-check-write done in three separate calls lets two requests both pass.

> **Common mistake.** Counting in the application database. It is slow under load, and hot keys cause locking.

Going deeper

Algorithm	Behaviour	Weakness
Fixed window counter	Count per minute, reset at the boundary	Double burst across the boundary
Sliding window log	Store each timestamp	Memory grows with traffic
Sliding window counter	Weight the previous window	Approximate, but cheap
Token bucket	Tokens refill at a steady rate; bursts up to capacity	Needs two numbers tuned

Distributed limits. If ten servers each allow 100 per minute, a user can make 1,000. Keep the counter in a shared store (for example Redis) and make check-and-decrement a single atomic script, so two servers cannot both spend the last token.

Response. Return HTTP 429 with a Retry-After header, so well-behaved clients back off.

Interview follow-up

- **Q: Per-user or per-IP?** A: Per authenticated user where possible. Per IP punishes users behind shared NAT.
- **Q: What if Redis is down?** A: Decide explicitly: fail open (allow traffic, risk overload) or fail closed (block, risk outage). Name the choice for each endpoint.

Complete interview coverage

What the interviewer checks: that you can design a rate limiter that works across many servers.

A complete answer you should be able to give: Choose token bucket for controlled bursts or a sliding window counter for accuracy, and key the limit by authenticated user with an IP fallback. Keep the counters in a shared store and make check-and-update one atomic operation. Return 429 with Retry-After, and state whether failures open or close the limiter.

Follow-up questions and answers:

- **Q: What if the store is down?**
  A: Decide per endpoint: fail open for low-risk paths, fail closed for abuse-sensitive ones.
- **Q: Why is fixed window weak?**
  A: Two bursts can pass at the boundary, double the intended rate.

Mistakes that cost points: A per-server limiter that a user can multiply by the number of servers.

PART 3 · INTERMEDIATE

Distributed Systems

Machines fail, messages repeat, and clocks disagree. These lessons show how good designs cope.

- Lesson 14: Leaders, consensus, and fencing
- Lesson 15: Timeouts, retries, and circuit breakers
- Lesson 16: Idempotency, outbox, and sagas
- Lesson 17: Kafka and event streams
- Lesson 18: Stream processing, time, and late data
- Lesson 19: Your third design: a chat application

Level: Intermediate · About 45 minutes

Lesson 14: Leaders, consensus, and fencing

Goal

Explain how a group of machines agrees on one leader, and why a paused leader must be stopped from writing.

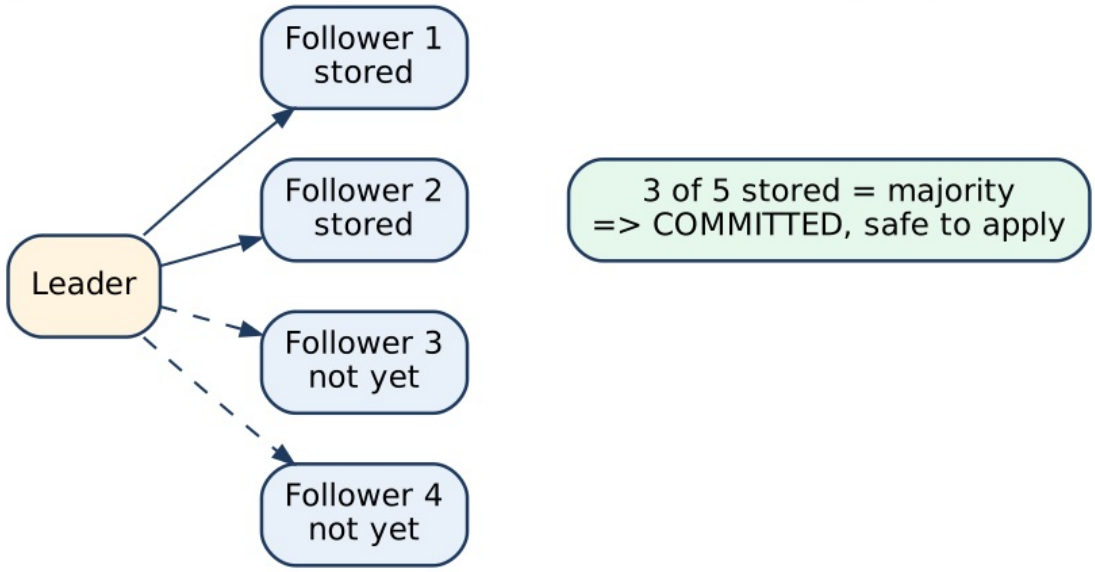
The idea in plain words

> **Plain words.** A committee elects a chair. A candidate needs more than half the votes, so two chairs can never both be legitimate. Each term of office has a number, and anyone holding an old term is ignored.

Raft, step by step

1. Every node is a follower and waits for a heartbeat from the leader.
2. If the heartbeat stops, a node waits a random time, then becomes a candidate for a new term and asks for votes.
3. A node that receives a majority of votes becomes leader.
4. The leader sends log entries to followers. An entry is committed when a majority has stored it.

Figure 7: Raft, a log entry commits when a majority stores it



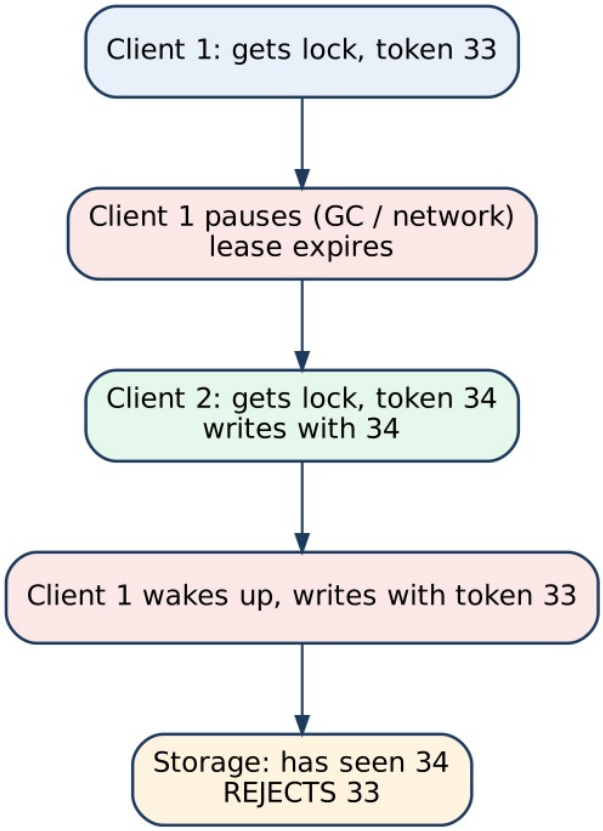
An entry commits when a majority (3 of 5) has stored it. A minority cannot commit alone.

> **Key idea.** A group of 5 tolerates 2 failures, because the remaining 3 are still a majority. A group of 7 tolerates 3.

The paused leader problem

A leader can pause (garbage collection, a slow network) and wake up believing it is still leader. A lock or lease expires in the meantime, so a second client takes over. Now two clients can write.

Figure 8: Fencing token stops a paused client from writing



Fencing: the storage remembers the highest token and rejects the old one.

Code (runs with Python 3, standard library only)

```
class Storage:
    def __init__(self):
        self.highest = 0
        self.data = None
    def write(self, token, value):
        if token < self.highest:
            return f"REJECTED (token {token} < seen {self.highest})"
        self.highest = token
        self.data = value
        return "OK"

class LockService:
    def __init__(self): self.token = 0
    def acquire(self):
        self.token += 1
        return self.token

lock = LockService(); store = Storage()
t1 = lock.acquire()          # client 1 gets token 1, then pauses (GC)
t2 = lock.acquire()          # lease expired, client 2 gets token 2
print("client 2 writes:", store.write(t2, "from client 2"))
print("client 1 wakes and writes:", store.write(t1, "from client 1 (stale!)"))
print("stored value:", store.data)
```

Output when you run it:

```
client 2 writes: OK
client 1 wakes and writes: REJECTED (token 1 < seen 2)
stored value: from client 2
```

Common mistake. Using a lock with an expiry and assuming the lock is safe. Without a fencing token, a paused holder can still write. Lock services help, but the protected resource must check tokens.

Going deeper

How Raft elects a leader. Each node is a follower until it hears from a leader. If it hears nothing for a random timeout (for example 150 to 300 ms), it becomes a candidate, increments the term number, votes for itself and asks the others. A candidate with a majority becomes leader. The random timeout makes split votes rare.

How a write commits. The leader appends the entry to its log and sends it to followers. Once a majority has stored it, the leader marks it committed and applies it. With 5 nodes, the cluster tolerates 2 failures.

Cluster size	Failures tolerated
3	1
5	2
7	3

Why fencing. A leader can pause (for example during garbage collection), lose leadership, and wake up still believing it leads. Its writes must be rejected. Each lease or leadership term carries an increasing token, and storage rejects any token lower than one it has seen.

Interview follow-up

- **Q: Why not use an odd number of nodes?** A: Four nodes tolerate one failure, the same as three. The extra node adds cost without adding tolerance.
- **Q: Who uses consensus in practice?** A: Coordination services such as etcd and ZooKeeper, and databases that replicate logs. Applications usually use them rather than implement Raft themselves.

Complete interview coverage

What the interviewer checks: that you can explain consensus, leader election and why fencing is needed.

A complete answer you should be able to give: A consensus protocol such as Raft elects a leader by majority vote and commits an entry once a majority stores it, so a minority partition cannot make progress. Use it for metadata and leader election, not for every write. Fencing tokens stop a paused old leader from writing after a new one has been elected.

Follow-up questions and answers:

- **Q: Why an odd number of nodes?**
  A: Five nodes tolerate two failures; an even count adds cost without extra tolerance.
- **Q: Who should implement Raft?**
  A: Almost nobody; use a coordination service instead.

Mistakes that cost points: Describing consensus without the majority rule or the stale-leader problem.

Level: Intermediate · About 40 minutes

Lesson 15: Timeouts, retries, and circuit breakers

Goal

Protect your system when a dependency is slow or failing: set timeouts, retry safely, and stop calling a broken service.

The idea in plain words

> **Plain words.** If a shop's card machine is down, you do not keep swiping every second. You wait a bit, try again a few times, and if it still fails, you stop trying for a while and pay another way. Those three ideas are timeouts, backoff, and a circuit breaker.

Timeouts

Every call to another service needs a deadline. Without one, a slow dependency holds your threads until your whole service is stuck. Derive the timeout from the dependency's normal p99 latency plus a margin.

Retries with backoff and jitter

Retrying immediately makes an overloaded service worse. Wait a random time between zero and a cap that doubles each attempt. The randomness (jitter) spreads retries out.

Code (runs with Python 3, standard library only)

```
import random

def retry(fn, attempts=5, base=0.1, cap=2.0, sleep=time.sleep):
    for attempt in range(attempts):
        try:
            return fn()
        except IOError:
            if attempt == attempts - 1:
                raise
            # "full jitter": random wait between 0 and the exponential cap
            sleep(random.uniform(0, min(cap, base * 2 ** attempt)))

random.seed(1)
calls = []
def flaky_then_ok():
    calls.append(1)
    if len(calls) < 3:
        raise IOError("503")
    return "ok"
print("result:", retry(flaky_then_ok, sleep=lambda s: print(f"  waited {s:.3f}s")), "after", len(calls), "calls")
```

Output when you run it:

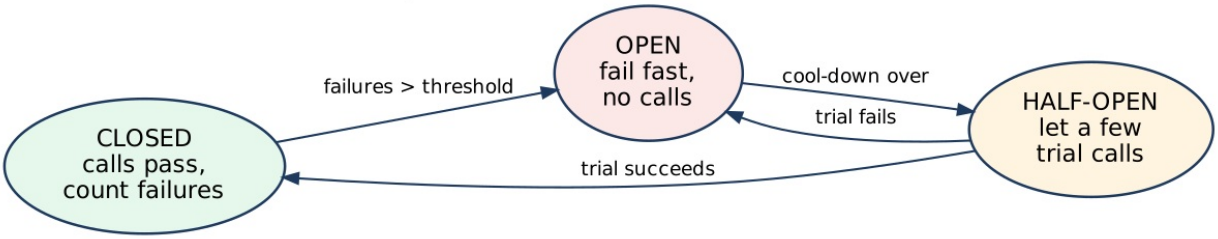
```
  waited 0.013s
  waited 0.169s
result: ok after 3 calls
```

> **Common mistake.** Retrying at every layer. Three layers with three retries each send 27 times the load to the bottom service. Retry in one place, usually close to the caller.

Circuit breaker

After repeated failures, the breaker opens and calls fail instantly. After a cool-down, a trial call decides whether to close it again.

Figure 15: Circuit breaker states



Circuit breaker states: closed, open, half-open.

Code (runs with Python 3, standard library only)

```
class CircuitBreaker:
    def __init__(self, failure_threshold=3, cooldown=5.0, now=time.monotonic):
        self.th, self.cooldown, self.now = failure_threshold, cooldown, now
        self.state, self.failures, self.opened_at = "CLOSED", 0, 0.0

    def call(self, fn):
        if self.state == "OPEN":
            if self.now() - self.opened_at >= self.cooldown:
                self.state = "HALF_OPEN"
            else:
                raise RuntimeError("circuit open: failing fast")
        try:
            result = fn()
        except Exception:
            self.failures += 1
            if self.state == "HALF_OPEN" or self.failures >= self.th:
                self.state, self.opened_at = "OPEN", self.now()
            raise
        self.state, self.failures = "CLOSED", 0
        return result

clock = [0.0]
cb = CircuitBreaker(failure_threshold=3, cooldown=5, now=lambda: clock[0])
def flaky(): raise IOError("timeout")
for i in range(4):
    try: cb.call(flaky)
    except Exception as e: print(f"call {i+1}: {type(e).__name__}: {e}  state={cb.state}")
clock[0] = 6.0
try:
    cb.call(lambda: "ok")
    print("after cool-down, trial succeeded, state =", cb.state)
except Exception as e:
    print("unexpected", e)
```

Output when you run it:

```
call 1: OSError: timeout  state=CLOSED
call 2: OSError: timeout  state=CLOSED
call 3: OSError: timeout  state=OPEN
call 4: RuntimeError: circuit open: failing fast  state=OPEN
after cool-down, trial succeeded, state = CLOSED
```

> **Key idea.** Only retry operations that are safe to repeat (see Lesson 16). A retry of a non-idempotent call can charge a customer twice.

Going deeper

Timeouts first. A call without a timeout can hang forever and hold threads, which turns a slow dependency into a full outage. Set a timeout from the dependency's measured latency, for example its p99 plus a margin.

Retry rules. Retry only on errors that might succeed next time (timeouts, 503), never on errors that will repeat (400, 404). Retry only operations that are idempotent (Lesson 16). Use exponential backoff with full jitter: wait a random time between 0 and min(cap, base times 2 to the attempt).

Retry budget. If 10 percent of traffic is already retries, stop retrying. Without this limit, retries multiply load on the failing service (the 27 times amplification described in Lesson 13).

Circuit state	Behaviour

Closed	Calls go through; failures are counted
Open	Calls fail immediately for a cool-down period
Half-open	A few probe calls test recovery; success closes, failure reopens

Interview follow-up

- **Q: What is a bulkhead?** A: Separate thread pools or connection pools per dependency, so one slow dependency cannot exhaust shared resources.
- **Q: What should the client do when the circuit is open?** A: Serve a fallback (cached or default data) or fail fast with a clear error.

Complete interview coverage

What the interviewer checks: that you can keep a failing dependency from taking down the whole system.

A complete answer you should be able to give: Set timeouts from measured latency, retry only idempotent operations on transient errors with exponential backoff and full jitter, and cap retries with a budget. Wrap the dependency in a circuit breaker that fails fast when it is unhealthy, and isolate dependencies with bulkheads. Define a fallback for each user-facing path.

Follow-up questions and answers:

- **Q: Why jitter?**
  A: Without it, many clients retry together and keep the dependency overloaded.
- **Q: What is a retry budget?**
  A: A limit on the fraction of traffic that can be retries.

Mistakes that cost points: Retrying non-idempotent writes; no timeouts.

Level: Intermediate · About 50 minutes

Lesson 16: Idempotency, outbox, and sagas

Goal

Make an operation safe to repeat, publish events without losing or inventing them, and undo a multi-step business action when a step fails.

Idempotency

> **Plain words.** Pressing the lift button twice does not call two lifts. An idempotent operation gives the same result no matter how many times you ask. Networks repeat requests, so your payment API must behave like the lift button.

Pattern: the client sends an **idempotency key** (a unique ID for this attempt). The server stores the key with its result. A repeat with the same key returns the saved result. The same key with a different body is rejected.

Code (runs with Python 3, standard library only)

```
import threading

class IdempotentPayments:
    def __init__(self):
        self._lock = threading.Lock()
        self._results = {}   # idempotency_key -> (request_hash, response)
        self.charges = 0

    def pay(self, key: str, amount_minor: int) -> dict:
        req_hash = str(amount_minor)
        with self._lock:                      # check and store are atomic
            if key in self._results:
                saved_hash, saved = self._results[key]
                if saved_hash != req_hash:
                    return {"error": "422: key reused with different body"}
                return {**saved, "replayed": True}
            self.charges += 1                 # the one real side effect
            resp = {"status": "CAPTURED", "amount": amount_minor, "charge_no": self.charges}
            self._results[key] = (req_hash, resp)
            return {**resp, "replayed": False}

p = IdempotentPayments()
print(p.pay("order-42", 50000))
print(p.pay("order-42", 50000))   # network retry with same key
print(p.pay("order-42", 99999))   # same key, different body
print("real charges made:", p.charges)
```

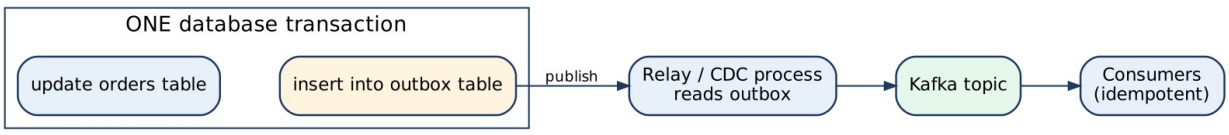
Output when you run it:

```
{'status': 'CAPTURED', 'amount': 50000, 'charge_no': 1, 'replayed': False}
{'status': 'CAPTURED', 'amount': 50000, 'charge_no': 1, 'replayed': True}
{'error': '422: key reused with different body'}
real charges made: 1
```

Transactional outbox

Problem: you save an order to the database, then publish an event to Kafka. If the process crashes between the two, the order exists but no event was sent (or the reverse).

Figure 10: Transactional outbox, no lost events



Write the change and the event in one transaction. A relay sends the event afterward.

Code (runs with Python 3, standard library only)

```
import sqlite3, json

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE orders (id INTEGER PRIMARY KEY, status TEXT)")
db.execute("CREATE TABLE outbox (id INTEGER PRIMARY KEY AUTOINCREMENT, payload TEXT, sent INTEGER DEFAULT 0)")

def place_order(order_id, fail_after_insert=False):
    with db:  # one transaction: both writes or neither
        db.execute("INSERT INTO orders VALUES (?, 'PLACED')", (order_id,))
        db.execute("INSERT INTO outbox (payload) VALUES (?)", (json.dumps({"event": "OrderPlaced", "id": order_id}),))
        if fail_after_insert:
            raise RuntimeError("crash before commit")

def relay(publish):
    rows = db.execute("SELECT id, payload FROM outbox WHERE sent=0 ORDER BY id").fetchall()
    for rid, payload in rows:
        publish(payload)                      # at-least-once: may resend after a crash
        db.execute("UPDATE outbox SET sent=1 WHERE id=?", (rid,))
    db.commit()

try:
    place_order(1, fail_after_insert=True)
except RuntimeError:
    pass
print("after crash: orders =", db.execute("SELECT COUNT(*) FROM orders").fetchone()[0],
      "| outbox =", db.execute("SELECT COUNT(*) FROM outbox").fetchone()[0], "(nothing half-written)")
place_order(2)
sent = []
relay(sent.append)
print("published events:", sent)
```

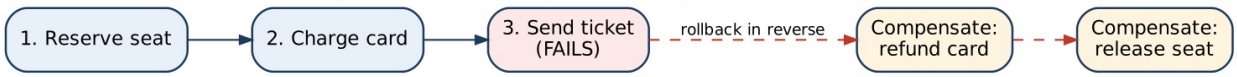
Output when you run it:

```
after crash: orders = 0 | outbox = 0 (nothing half-written)
published events: ['{"event": "OrderPlaced", "id": 2}']
```

Saga

Booking a seat, charging a card, and sending a ticket cannot share one database transaction across services. A saga runs each step and, if a later step fails, runs the undo steps in reverse order.

Figure 11: Saga, steps with compensations on failure



Saga: if sending the ticket fails, refund the card, then release the seat.

Code (runs with Python 3, standard library only)

```
class SagaFailed(Exception): pass

def run_saga(steps):
    """steps: list of (name, action, compensation). Compensate completed steps in reverse."""
    done = []
    try:
        for name, action, comp in steps:
            action()
            done.append((name, comp))
            print(f"  ok: {name}")
    except Exception as e:
        print(f"  FAILED: {e}; compensating")
        for name, comp in reversed(done):
            comp()
            print(f"  undone: {name}")
        raise SagaFailed(str(e))

def fail(): raise RuntimeError("email service down")
try:
    run_saga([
        ("reserve seat", lambda: None, lambda: None),
        ("charge card",  lambda: None, lambda: print("    refund issued")),
        ("send ticket",  fail,          lambda: None),
    ])
except SagaFailed:
    print("saga ended in a consistent state (seat released, card refunded)")
```

Output when you run it:

```
  ok: reserve seat
  ok: charge card
  FAILED: email service down; compensating
    refund issued
  undone: charge card
  undone: reserve seat
saga ended in a consistent state (seat released, card refunded)
```

Key idea. The three tools go together: idempotency makes retries safe, the outbox makes events reliable, and sagas make multi-step work recoverable.

Common mistake. Saying "exactly once" delivery. Networks can always repeat a message. Aim for "exactly-once effect": at-least-once delivery plus idempotent processing.

Going deeper

Exactly once is an illusion made of two parts. True exactly-once delivery over a network is not possible. What systems achieve is at-least-once delivery plus idempotent processing, which looks exactly once to the user.

Idempotency key design. The client generates a unique key per intended operation (for example a UUID per checkout). The server stores the key with the result in the same transaction as the business change. A repeat with the same key returns the stored result. A repeat with the same key but a different body is an error.

Outbox pattern. Writing to the database and publishing an event are two systems, so one can succeed without the other. Instead, write the event into an outbox table in the same transaction as the change. A relay reads the outbox and publishes, marking rows sent. The relay can repeat, so consumers must be idempotent.

Saga styles.

Style	How	Trade-off
Orchestration	A coordinator calls each step and compensations	Clear flow; the coordinator is central
Choreography	Services react to each other's events	No central point; flow is hard to see

Interview follow-up

- **Q: What is a compensation?** A: A business action that undoes an earlier step (refund a charge, release a reserved seat). It is not a database rollback because the earlier step is already committed.
- **Q: What if a compensation itself fails?** A: Retry it with idempotency, and if it still fails, alert and queue it for a person.

Complete interview coverage

What the interviewer checks: that you can make operations safe to repeat and keep data and events consistent.

A complete answer you should be able to give: Every state-changing request carries an idempotency key stored with its result in one transaction, so a repeat returns the same outcome. Publish events with the outbox pattern so the change and event cannot diverge. For multi-service flows, use a saga with compensations, and say which step is hardest to undo.

Follow-up questions and answers:

- **Q: Is exactly-once possible?**
  A: Not over a network; it is at-least-once plus idempotent processing.
- **Q: Orchestration or choreography?**
  A: Orchestration for clear control; choreography for loose coupling.

Mistakes that cost points: Claiming exactly-once delivery; no plan for compensation failure.

Level: Intermediate · About 45 minutes

Lesson 17: Kafka and event streams

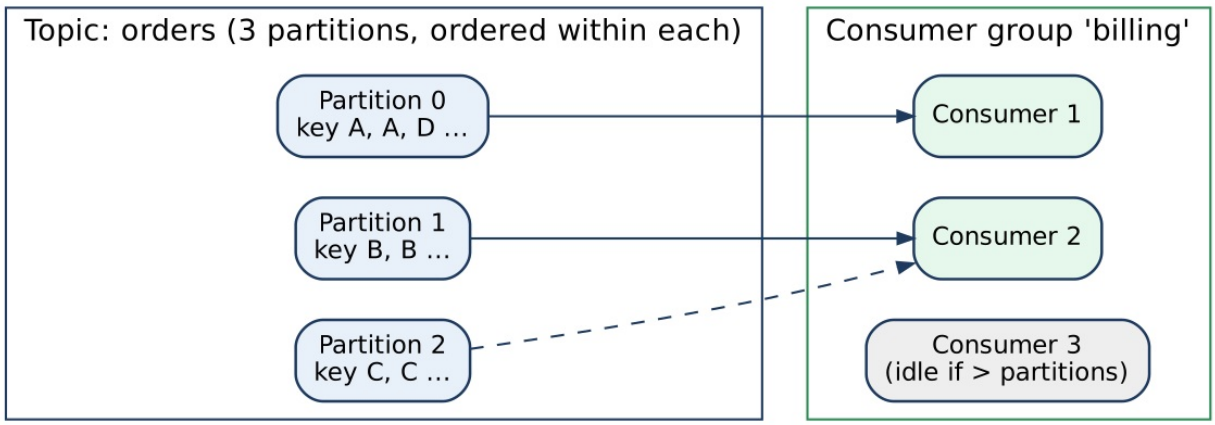
Goal

Explain topics, partitions, consumer groups, offsets, and the delivery guarantees you can configure.

The idea in plain words

Plain words. A Kafka topic is a set of numbered logbooks. Each logbook (partition) is written in order and never edited. Readers keep a bookmark (offset). A team of readers (consumer group) splits the logbooks between them, one reader per logbook.

Figure 9: Kafka topic, partitions, and consumer groups



A topic has partitions. A consumer group shares partitions. Extra consumers sit idle.

The rules to know

- **Order** holds only inside one partition. Use a key (for example the order ID) so all events for one order go to the same partition.
- **Parallelism** is limited by partitions. Five partitions support at most five active consumers in a group.
- **Acknowledgements:** acks=all with min in-sync replicas of 2 and replication factor 3 survives one broker failure without losing acknowledged messages.
- **Duplicates:** a consumer that crashes after processing but before committing its offset will process the message again. Make consumers idempotent.
- **Retention:** messages stay for a set time, so you can replay them.

Key idea. Choose the partition key to match the ordering you need. Changing the number of partitions later changes which partition each key goes to.

Common mistake. Using a random key. Events for the same entity then land on different partitions and can be processed out of order.

Going deeper

Partitions and keys. A topic is split into partitions. Messages with the same key go to the same partition, so they stay in order. Ordering across partitions is not guaranteed.

Consumer groups. Consumers in one group share the partitions; each partition goes to one consumer in the group. Adding consumers beyond the partition count does nothing. Each group tracks its own offset, so several groups read the same log independently.

Setting	Effect
acks=0	Fastest; messages can be lost
acks=1	Leader confirms; lost if leader fails before replication
acks=all with min.insync.replicas=2	Confirmed only after replicas hold it; the usual safe choice

Retention and compaction. Logs are kept for a time or size. Compacted topics keep the latest value per key, which suits "current state" streams such as a user profile.

Rebalancing. When a consumer joins or dies, partitions move. Messages after the last committed offset are processed again, so handlers must be idempotent.

Interview follow-up

- **Q: How many partitions should a topic have?** A: Enough for your peak consumer count and throughput, plus headroom. Partitions are hard to reduce later, so start with a sensible number.
- **Q: Is Kafka a queue?** A: It is a replicated log. Consumers track offsets, so it behaves like a queue per group and like pub-sub across groups.

Complete interview coverage

What the interviewer checks: that you can use a log-based stream: partitions, consumer groups, offsets and durability settings.

A complete answer you should be able to give: Use Kafka-style partitions keyed by the entity whose order matters, so each key stays ordered. Consumer groups share partitions and track offsets, so consumers can replay. Set acks to all with a minimum in-sync replica count for durability, and make consumers idempotent because rebalances replay messages.

Follow-up questions and answers:

- **Q: How many partitions?**
  A: Enough for peak consumer parallelism with headroom; partitions are hard to reduce.
- **Q: Is Kafka a queue?**
  A: It behaves like a queue per group and like pub-sub across groups.

Mistakes that cost points: Assuming order across partitions; using acks equal to zero for important data.

Level: Intermediate · About 45 minutes

Lesson 18: Stream processing, time, and late data

Goal

Count events by the time they happened, close windows correctly, and handle events that arrive late.

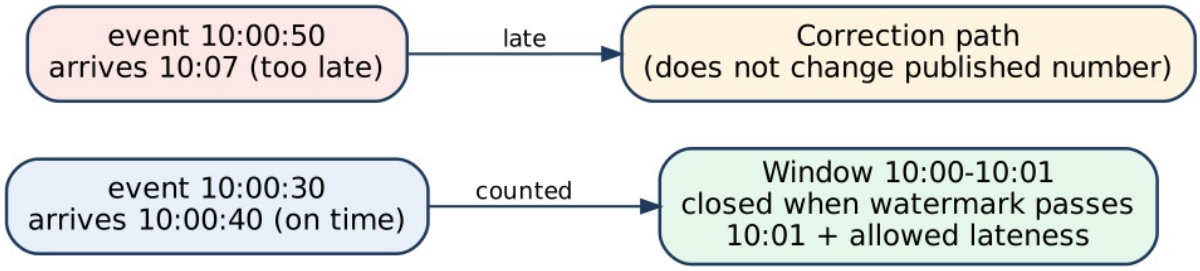
The idea in plain words

> **Plain words.** A friend posts a photo of a sunset at 6:00 pm, but their phone was offline and it reaches you at 6:20 pm. If you count it under "posts at 6:20", your chart is wrong. Count it under the time it happened. The question is how long you wait for stragglers before you publish a number.

Key terms

- **Event time:** when it happened. **Processing time:** when your system saw it. Use event time for correct charts.
- **Window:** a bucket of time, such as one minute.
- **Watermark:** the system's belief about how far behind the events are. When the watermark passes the end of a window (plus an allowed lateness), the window closes.
- **Late event:** an event that arrives after its window closed. Send it to a correction path rather than silently changing a published number.

Figure 23: Event time, windows, and late data



Events count in their event-time window. Very late events go to a correction path.

Code (runs with Python 3, standard library only)

```
from collections import defaultdict

class MinuteCounter:
    """Counts events per minute (by EVENT time). Closes a window when watermark passes it."""
    def __init__(self, allowed_lateness=300):
        self.windows = defaultdict(int)
        self.closed = set()
        self.watermark = 0
        self.lateness = allowed_lateness
        self.corrections = []

    def on_event(self, event_time: int):
        minute = event_time // 60 * 60
        if minute in self.closed:
            self.corrections.append(event_time)     # too late: do not change published number
            return
        self.windows[minute] += 1
        self.watermark = max(self.watermark, event_time)
        for m in list(self.windows):
            if m + 60 + self.lateness <= self.watermark:
                self.closed.add(m)
                print(f"  window {m} closed with count {self.windows[m]}")

mc = MinuteCounter(allowed_lateness=300)
for t in [30, 45, 70, 80, 100]:
    mc.on_event(t)
mc.on_event(900)                 # watermark jumps to 15:00
```

```
mc.on_event(40)                  # very late event for minute 0
print("late events sent to correction path:", mc.corrections)
```

Output when you run it:

```
  window 0 closed with count 2
  window 60 closed with count 3
late events sent to correction path: [40]
```

Key idea. Allowed lateness is a trade-off: longer means more accurate numbers but slower results. Choose it per use case and say so.

Going deeper

Event time vs processing time. A mobile app records a click at 10:00 but uploads it at 10:05 after being offline. Counting by processing time puts the click in the wrong minute. Counting by event time is correct, but then you must decide when a window is complete.

Watermarks. A watermark is the system's estimate of "no events older than this will arrive". When the watermark passes the end of a window, the window emits. Events arriving later than the watermark are late data.

Window	Shape	Example
Tumbling	Fixed, non-overlapping	Clicks per minute
Sliding	Fixed length, moves by a step	Errors in the last 5 minutes, every minute
Session	Ends after a gap of inactivity	A user's visit

Late data policy. Choose one: drop it, add it as a correction (emit an updated result), or send it to a side output for review. Name the choice in the design.

Exactly-once in streams. Use checkpoints of state plus source offsets, and write output transactionally, so a restart produces the same result.

Interview follow-up

- **Q: What if the watermark is too aggressive?** A: Windows close too early and miss data. Too lax, and results are delayed. Tune it to the observed lateness distribution.
- **Q: Batch or stream?** A: Stream for fresh results; batch for complete, corrected results. Lambda runs both; Kappa uses one stream and replays.

Complete interview coverage

What the interviewer checks: that you can handle time, late events and windows correctly.

A complete answer you should be able to give: Compute aggregations by event time, not processing time, and use watermarks to decide when a window is complete. Choose a late-data policy: drop, correct the result, or send to a side output. Name the window type you need and how the job recovers exactly-once state after a restart.

Follow-up questions and answers:

- **Q: What if the watermark is too early?**
  A: Late events are missed; tune it to the observed lateness distribution.
- **Q: Batch or stream?**
  A: Stream for fresh results; batch for complete corrections; Kappa replays one stream.

Mistakes that cost points: Using processing time for business metrics.

Level: Intermediate · About 60 minutes · Practice lesson

Lesson 19: Your third design: a chat application

Goal

Design a messaging system with online delivery, ordering, read receipts, and offline push, using the ideas from Parts 2 and 3.

The question

"Design a chat app like WhatsApp: one-to-one and group messages, delivery and read receipts, online status, message history."

Step 1: Requirements

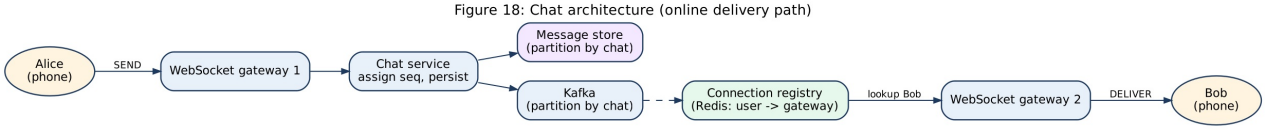
- Messages arrive in order within a conversation.
- Online delivery under about 300 ms within a region.
- Offline users get a push notification and catch up when they reconnect.
- History is kept forever.

Step 2: Estimates

- 50 million daily users × 40 messages ≈ 2 billion messages a day, about 23,000 per second on average, about 70,000 at peak.
- Each device holds one open WebSocket. About 20 million open connections at peak; at 100,000 per gateway server, that is about 200 gateways.

Step 3: Design

Figure 18: Chat architecture (online delivery path)



Chat: the sender's gateway sends to the chat service, which saves and publishes. The registry finds the recipient's gateway.

Step 4: Send flow

1. The sender's app sends a message with a client-generated ID (so retries do not duplicate it).
2. The chat service assigns the next sequence number for the conversation and saves the message.
3. The sender receives an acknowledgement ("sent").
4. The event is published to Kafka with the conversation ID as the key, so order is kept.
5. The registry finds the recipient's gateway and pushes the message. The recipient's app acknowledges, and the sender sees "delivered".
6. If the recipient is offline, a push notification is sent. On reconnect, the app asks for messages after its last sequence number.

> **Key idea.** Ordering comes from one sequence counter per conversation, not from device clocks. Clocks disagree; sequence numbers do not.

Step 5: Deep dives

- **Large groups:** do not copy a message into 100,000 inboxes. Store it once and let readers pull from the conversation's log.
- **Presence:** online status changes often. Broadcasting every change to every contact is too much. Show status only for people on screen, and use heartbeats that expire.
- **Gateway failure:** clients reconnect elsewhere and resume from their last sequence number. Nothing is lost, because messages are saved before the acknowledgement.

Going deeper

Connection layer. Each user holds a WebSocket to a gateway server. The gateway keeps a map from user to connection. Gateways are stateless with respect to messages: state lives in a directory service that maps user to gateway.

Ordering. Global ordering of all messages is expensive. Require order only within a conversation: each conversation gets a sequence number from its partition. Clients detect gaps and request missing messages.

Delivery. The client generates a message ID. The server stores the message with that ID, so a retry does not duplicate it. The recipient acknowledges; undelivered messages stay in storage and are pushed when the user reconnects.

Presence. A user is online while heartbeats arrive. Heartbeats are noisy, so mark a user offline only after several missed beats. Publishing every status change to every friend does not scale; publish to recently active conversations only.

Group fan-out. A 1,000-member group multiplies writes by 1,000. Small groups fan out on write; very large channels fan out on read, as in the news feed (Lesson 20).

Interview follow-up

- **Q: How do you store messages?** A: Wide-column or key-value storage partitioned by conversation ID, ordered by sequence or time. Scans per conversation are then one partition read.
- **Q: What happens when a gateway dies?** A: Clients reconnect to another gateway, and the directory updates. Messages are not lost because they were stored before delivery.

Complete interview coverage

What the interviewer checks: that you can design real-time connections, ordering, delivery and presence.

A complete answer you should be able to give: Keep a WebSocket connection per user on stateless gateways, with a directory mapping users to gateways. Order messages within each conversation using sequence numbers, store each message under a client-generated ID so retries do not duplicate, and push undelivered messages on reconnect. Presence uses heartbeats with a grace period, and large groups fan out differently from small ones.

Follow-up questions and answers:

- **Q: How do you store messages?**
  A: Partition storage by conversation, ordered by sequence, so history is a single partition read.
- **Q: What if a gateway dies?**
  A: Clients reconnect elsewhere; messages were already stored, so nothing is lost.

Mistakes that cost points: Trying to guarantee global ordering; ignoring reconnect and missed messages.

PART 4 · INTERMEDIATE

Designs at Scale

Eight real-world systems. Each lesson applies the earlier parts to one famous kind of product.

- Lesson 20: News feed and fan-out
- Lesson 21: Search and typeahead
- Lesson 22: Ride matching with geographic cells
- Lesson 23: Payments, where correctness comes first
- Lesson 24: File storage and duplicate detection
- Lesson 25: Metrics and analytics storage
- Lesson 26: Video streaming and the CDN
- Lesson 27: Job schedulers and workflows

Level: Intermediate · About 50 minutes

Lesson 20: News feed and fan-out

Goal

Decide when to do the work of building a feed: when a post is written, or when a feed is read.

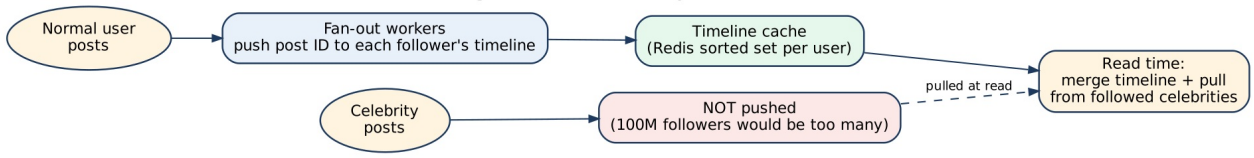
The idea in plain words

> **Plain words.** Imagine a newsletter. You can print a copy for every subscriber the moment an issue is ready (push), or you can keep one copy and have each reader collect the issues they follow when they visit (pull). Printing is fast to read but expensive when one author has a million subscribers. Pull is cheap to write but slow to read if a reader follows thousands of authors.

The two approaches

	Push on write (fan-out)	Pull on read
Cost of a new post	Copy the post ID to every follower's timeline	Store only once
Cost of opening the feed	Read one precomputed list: fast	Merge many authors at read time: slow
Breaks when	A celebrity has 100 million followers	A user follows 5,000 accounts

Figure 19: News feed, hybrid fan-out



Hybrid: push normal posts, pull celebrity posts at read time.

The hybrid design

1. Push the post ID into follower timelines when the author has fewer than a threshold of followers (for example, 10,000).
2. For celebrities, do not push. When a user opens the feed, merge their precomputed timeline with the latest posts of the few celebrities they follow.
3. Store IDs in timelines, not full posts. One post then serves millions of timelines, and an edit or delete is a single write.
4. Paginate with a cursor (the last seen post's time and ID), not an offset, so new posts do not shift pages.

> **Key idea.** The threshold is a tuning knob. Say what you would measure to set it: the number of followers at which push costs more than pull.

> **Common mistake.** Designing one approach for all users. Real feeds have a very skewed follower distribution, which is why the hybrid exists.

Going deeper

Ranking. A real feed is not chronological only. A ranking model scores candidate posts using relationship strength, recency and engagement. The design question is where ranking happens: on a candidate set precomputed for the user, which is common.

Timeline storage. Keep each user's timeline as a capped list of post IDs (for example the latest 800). Trim old entries. Hydrate the post bodies from a separate cache when the feed is read.

Celebrity threshold. Choose the threshold from measured cost: the number of timeline writes a post triggers versus the read-time merge cost. Revisit it as traffic grows.

Edits and deletes. Because timelines hold IDs, a delete removes one post and the feed shows nothing for it. This is why ID-based timelines matter.

Interview follow-up

- **Q: What if a timeline write fails for one follower?** A: Retry asynchronously through the queue; a missing entry for a while is acceptable for a feed.
- **Q: How do you handle a user who follows 50,000 accounts?** A: Cap the number of pulled celebrities, pre-merge the rest in the background, and limit feed depth.

Complete interview coverage

What the interviewer checks: that you can choose push or pull for a feed and handle the high-fan-out case.

A complete answer you should be able to give: Use a hybrid: push new posts into timelines of followers for ordinary accounts, and pull posts from very large accounts at read time, merging the two. Store post IDs in timelines, hydrate bodies from a cache, and paginate with a cursor. Explain the threshold choice with the cost of each write versus each read merge.

Follow-up questions and answers:

- **Q: What if a timeline write fails?**
  A: Retry through the queue; a short delay on a feed is acceptable.
- **Q: What about deletes?**
  A: Storing IDs means a delete removes the post everywhere it is referenced.

Mistakes that cost points: Only push or only pull; storing full posts in every timeline.

Level: Intermediate · About 40 minutes

Lesson 21: Search and typeahead

Goal

Return the top suggestions for a prefix in under 100 ms, and explain how the data is built and updated.

The idea in plain words

> **Plain words.** A dictionary with the most popular words printed first on each page. When you type "ca", you see the popular words that begin with "ca" immediately. Nobody searches the whole dictionary while you type.

How it is built

1. **Offline:** a job reads query logs, counts each phrase, removes unsafe or rare phrases, and computes the top 10 phrases for every prefix (up to about 15 characters).
2. **Serve:** store those lists in memory, sharded by the first characters of the prefix. A lookup is a direct read, not a search.
3. **Update:** build a new version in the background, then swap the pointer. If the new version is bad, swap back.
4. **Fresh trends:** a small real-time counter for the last hour is merged in at serve time.

Code (runs with Python 3, standard library only)

```
import math

class BloomFilter:
    def __init__(self, n_items: int, fp_rate: float = 0.01):
        self.m = int(-n_items * math.log(fp_rate) / (math.log(2) ** 2))  # bits
        self.k = max(1, round(self.m / n_items * math.log(2)))            # hash count
        self.bits = bytearray(self.m // 8 + 1)

    def _positions(self, item: str):
        d = hashlib.sha256(item.encode()).digest()
        a, b = int.from_bytes(d[:8], "big"), int.from_bytes(d[8:16], "big")
        return [(a + i * b) % self.m for i in range(self.k)]

    def add(self, item: str):
        for p in self._positions(item):
            self.bits[p // 8] |= 1 << (p % 8)

    def might_contain(self, item: str) -> bool:
        return all(self.bits[p // 8] & (1 << (p % 8)) for p in self._positions(item))

bf = BloomFilter(n_items=10000, fp_rate=0.01)
for i in range(10000):
    bf.add(f"item{i}")
print("all added items found (no false negatives):", all(bf.might_contain(f"item{i}") for i in range(10000)))
fp = sum(bf.might_contain(f"other{i}") for i in range(10000)) / 10000
print(f"false positive rate on 10,000 new items: {fp:.2%} (target 1%)")
print(f"bits per item: {bf.m/10000:.1f}, hash functions: {bf.k}")
```

Output when you run it:

```
all added items found (no false negatives): True
false positive rate on 10,000 new items: 1.04% (target 1%)
bits per item: 9.6, hash functions: 7
```

> **Key idea.** Precompute the answer for every prefix. Typeahead is a read-heavy problem, so spend work offline to make each request a lookup.

> **Common mistake.** Using "LIKE 'prefix%'" in a database on every keystroke. It sorts and scans per request and cannot rank cheaply at scale. Also, clients should debounce keystrokes (wait about 150 ms) to cut requests.

Going deeper

Inverted index. Full-text search maps each term to the list of documents containing it. A query intersects

these lists. Ranking uses term frequency and document importance.

Typeahead data structure. A trie stores prefixes; each node keeps its top suggestions, so a lookup walks the prefix and reads the stored list.

Freshness vs cost. Rebuilding the whole suggestion set hourly is cheap and stale by up to an hour. Trending terms need a streaming counter merged at read time (described in the lesson).

Safety. Filter offensive or private suggestions before they are published. This is a required step, not an afterthought.

Interview follow-up

- **Q: Why not use a database LIKE query?** A: It scans and cannot rank quickly at millions of queries per second.
- **Q: How do you handle typos?** A: Edit-distance candidates from an n-gram index, then re-rank by popularity.

Complete interview coverage

What the interviewer checks: that you can serve prefix suggestions quickly and keep them fresh and safe.

A complete answer you should be able to give: Precompute the top suggestions for each prefix offline from query logs, store them in memory sharded by prefix, and swap in new versions without downtime. Add a small trending counter at read time for freshness, and filter unsafe suggestions before publishing. Debounce on the client to limit requests.

Follow-up questions and answers:

- **Q: Why not a database LIKE query?**
  A: It scans and cannot rank quickly at high request rates.
- **Q: How do you handle typos?**
  A: Fuzzy candidates from an n-gram index, re-ranked by popularity.

Mistakes that cost points: Querying the database on every keystroke.

Level: Intermediate · About 50 minutes

Lesson 22: Ride matching with geographic cells

Goal

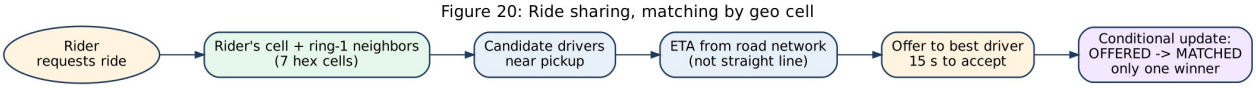
Find nearby drivers fast, match a rider to exactly one driver, and keep location data cheap.

The idea in plain words

> **Plain words.** Cut the city map into a honeycomb of hexagons. To find a taxi, you look in your hexagon and the six around it. You never measure distances to every taxi in the city.

Step by step

Figure 20: Ride sharing, matching by geo cell



Matching: search nearby cells, rank by road ETA, offer to one driver, lock the winner.

1. Each driver app sends its location every few seconds. Store only the latest location in memory, grouped by cell. Keep a sampled copy for analytics.
2. A ride request searches the rider's cell and its neighbors.
3. Rank candidates by road-network travel time (ETA), not straight-line distance. A river can make the straight line misleading.
4. Offer the ride to the best driver with a 15-second timeout. If refused, offer to the next.
5. When accepted, update the trip with a conditional write: "change state to matched only if still offered to this driver". Only one write can win.

> **Key idea.** Two riders can request the same driver at the same time. The conditional write is what prevents double matching.

> **Common mistake.** Writing every location update to a database. At a million updates per second, that is the bottleneck. Keep location in memory and treat it as disposable (drivers resend it every few seconds).

Going deeper

Cell choice. Geohashes and hexagonal grids (such as H3) split the map into cells. Cell size is a trade-off: small cells mean few drivers per search and more neighbour lookups; large cells mean more candidates per search.

Dispatch loop. A matching service ranks candidates by ETA, filters by vehicle type and rating, and offers to one driver at a time. Batch matching (collecting requests for a few seconds) improves global efficiency at a small latency cost.

Surge pricing. A demand-to-supply ratio per cell sets the price multiplier. It is a read model updated every minute.

Location pipeline. Drivers send updates every 4 seconds. At a million drivers that is 250,000 writes per second, so location is kept in memory and treated as disposable.

Interview follow-up

- **Q: What if the driver app loses signal?** A: The last known location expires after a timeout, and the driver is excluded from matching until it reports again.
- **Q: How do you compute ETA at scale?** A: Precomputed road-network travel times per segment, updated with live traffic, and a shortest-path search on that graph.

Complete interview coverage

What the interviewer checks: that you can match two parties in real time and prevent double assignment.

A complete answer you should be able to give: Index drivers by geographic cell, search the rider's cell and neighbours, rank candidates by road ETA, and offer to one driver at a time with a timeout. Keep location in memory since it is refreshed constantly. Assign with a conditional write so only one request wins the driver.

Follow-up questions and answers:

- **Q: Why not straight-line distance?**
  A: Rivers and one-way roads make it misleading; road ETA is the real cost.
- **Q: How do you handle a driver who loses signal?**
  A: Expire the last location so the driver stops being matched until it reports again.

Mistakes that cost points: Writing every location update to the main database; no atomic assignment.

Level: Intermediate to advanced · About 55 minutes

Lesson 23: Payments, where correctness comes first

Goal

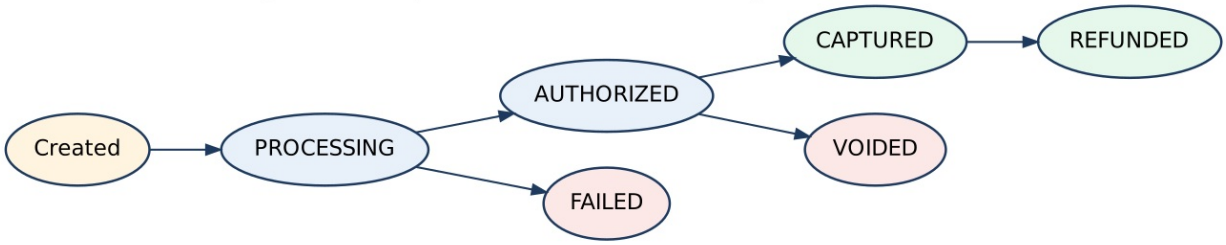
Design a payment flow that never charges twice, never loses money, and always reconciles with the bank.

The idea in plain words

Plain words. A bank does not just store a number for your balance. Every movement of money is written as two entries: money leaves one account and enters another. The totals always balance. If they do not, someone made an error, and you can find it.

The state machine

Figure 21: Payment state machine (only allowed moves)



A payment moves only along allowed arrows. Anything else is rejected.

Step by step

1. The client sends a payment with an idempotency key (Lesson 16). A repeat returns the same result.
2. The payment is saved as CREATED, with its version number. Each state change is a conditional update on the version.
3. The payment processor is called with our payment ID as its idempotency key.
4. If the processor times out, the outcome is unknown. Do not mark it failed. Ask the processor for the status first.
5. On success, ledger entries are written in the same database transaction as the state change.
6. Every night, reconciliation compares our records with the processor's settlement file. Mismatches go to a person.

Key idea. Use integers in minor units (cents or paise), never floating-point numbers, for money. Floats cause rounding errors.

Common mistake. Treating a timeout as a failure. The money may have moved. Unknown is a valid state, and it must be resolved, not guessed.

Going deeper

Card data never touches your system. Use a tokenizing payment provider: the client sends card details directly to them and gets a token. Your system stores the token only. This keeps you out of the highest compliance scope (PCI DSS).

Authorization vs capture. Authorization reserves money; capture actually moves it. Hotels and fuel stations authorize first. Keep these as separate states in the state machine.

Refunds and partial refunds. Each refund is its own payment-side operation with its own idempotency key, linked to the original payment. Never edit the original amount.

Reconciliation. The bank's settlement file is the external truth. Match on payment ID, amount and status. Unmatched items are investigated, not auto-corrected.

Interview follow-up

- **Q: The client retries after a timeout. What happens?** A: The same idempotency key returns the same payment result; no second charge is created.
- **Q: How do you handle currency?** A: Store amount and currency together, in minor units. Convert only at display or settlement, with the rate recorded.

Complete interview coverage

What the interviewer checks: that you understand payment correctness: idempotency, state, timeouts and reconciliation.

A complete answer you should be able to give: Every payment request has an idempotency key and a state machine with version checks, so a repeat returns the same result. Card data stays with a tokenizing provider. A timeout means unknown, so query the provider before deciding. Use integer minor units, ledger entries written in the same transaction, and a nightly reconciliation against the bank's settlement file.

Follow-up questions and answers:

- **Q: What if the client retries after a timeout?**
  A: The same key returns the stored result; no second charge.
- **Q: How do you handle refunds?**
  A: A separate linked operation with its own key; never edit the original amount.

Mistakes that cost points: Treating a timeout as a failure; using floating-point amounts.

Level: Intermediate · About 45 minutes

Lesson 24: File storage and duplicate detection

Goal

Store large files efficiently, sync only what changed, and detect duplicates quickly.

The idea in plain words

> **Plain words.** Imagine a 500-page book. If you fix one typo, you do not need to reprint the whole book; you replace the one page. Splitting files into chunks does the same: only changed chunks are uploaded.

Chunks and content addresses

- Split each file into chunks (about 4 MB). Name each chunk by its hash. Two identical chunks are stored once.
- A file version is a list of chunk hashes (a manifest).
- Use content-defined chunk boundaries, so inserting text at the start only changes nearby chunks.

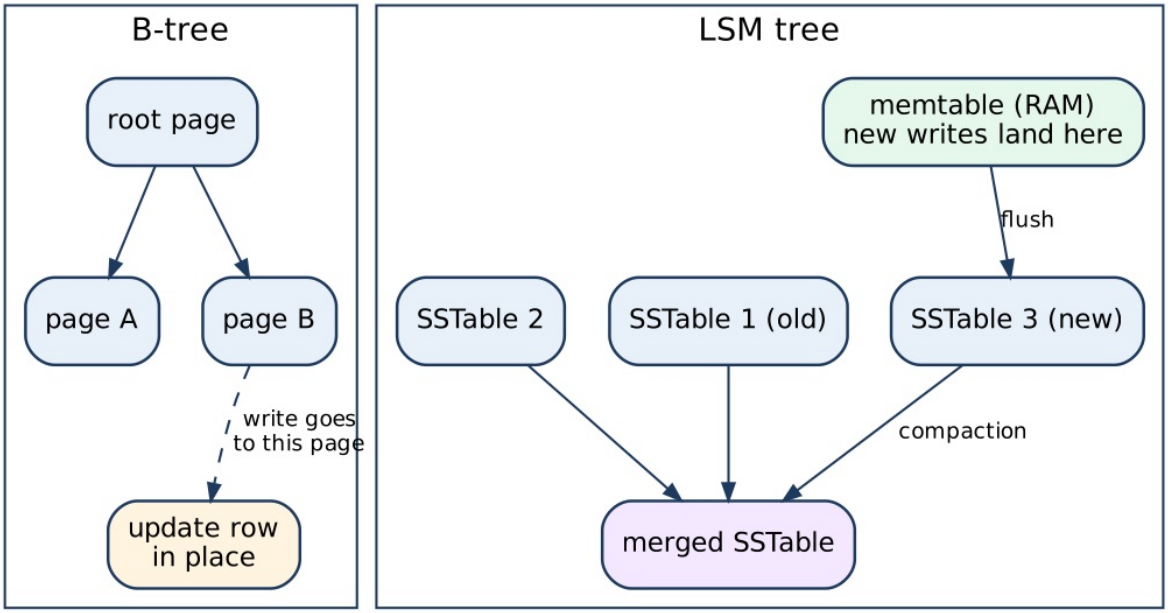
Commit safely

1. The client sends the chunk hashes. The server replies with the ones it does not have.
2. The client uploads only the missing chunks.
3. The client commits the new manifest with the version it started from. If another device changed the file first, the commit fails, and the client saves a conflict copy instead of overwriting.

Duplicate detection with a Bloom filter

A Bloom filter answers "have I seen this?" with very little memory. It never says no when the answer is yes. A "maybe" is checked against the real store. Crawlers and caches use this to skip work.

Figure 4: B-tree (update in place) vs LSM tree (append, then merge)



Storage engines: the layout of data on disk decides whether reads or writes are fast.

> **Key idea.** Commit metadata only after the chunks exist. A manifest that points to missing chunks is a corruption.

Going deeper

Why 4 MB chunks. Smaller chunks dedupe better and sync faster on edits, but create more metadata. Larger chunks reduce metadata but resend more on each change. Name the trade-off and pick a size.

Metadata store. File trees, versions and permissions live in a database separate from chunk bytes. Chunks live in object storage keyed by hash, so they are immutable and cacheable.

Garbage collection. A chunk is deleted only when no version references it. Run a mark phase over live manifests, then sweep unreferenced chunks after a safety delay, so an in-flight upload is not deleted.

Sharing and permissions. Check permissions on every metadata read. Shared links need expiry and revocation.

Interview follow-up

- **Q: How do large uploads resume?** A: The server tracks which chunks it has; the client resends only the rest.
- **Q: Why not store files in the database?** A: Large blobs bloat the database, slow backups, and cost more than object storage per gigabyte.

Complete interview coverage

What the interviewer checks: that you can store large files efficiently, sync changes, and avoid overwrites.

A complete answer you should be able to give: Split files into content-addressed chunks stored in object storage, keep the file version as a manifest of chunk hashes, and upload only missing chunks. Commit the manifest against the version the client started from, and on conflict keep both copies. Garbage collect chunks only after no manifest references them and after a safety delay.

Follow-up questions and answers:

- **Q: Why content-defined chunks?**
  A: Edits in the middle change only nearby chunks, so sync stays small.
- **Q: How do large uploads resume?**
  A: The server reports which chunks it has; the client sends the rest.

Mistakes that cost points: Storing blobs in the metadata database; deleting chunks that an upload still needs.

Level: Intermediate to advanced · About 45 minutes

Lesson 25: Metrics and analytics storage

Goal

Store millions of time-stamped numbers per second, compress them, and keep old data cheaply by downsampling.

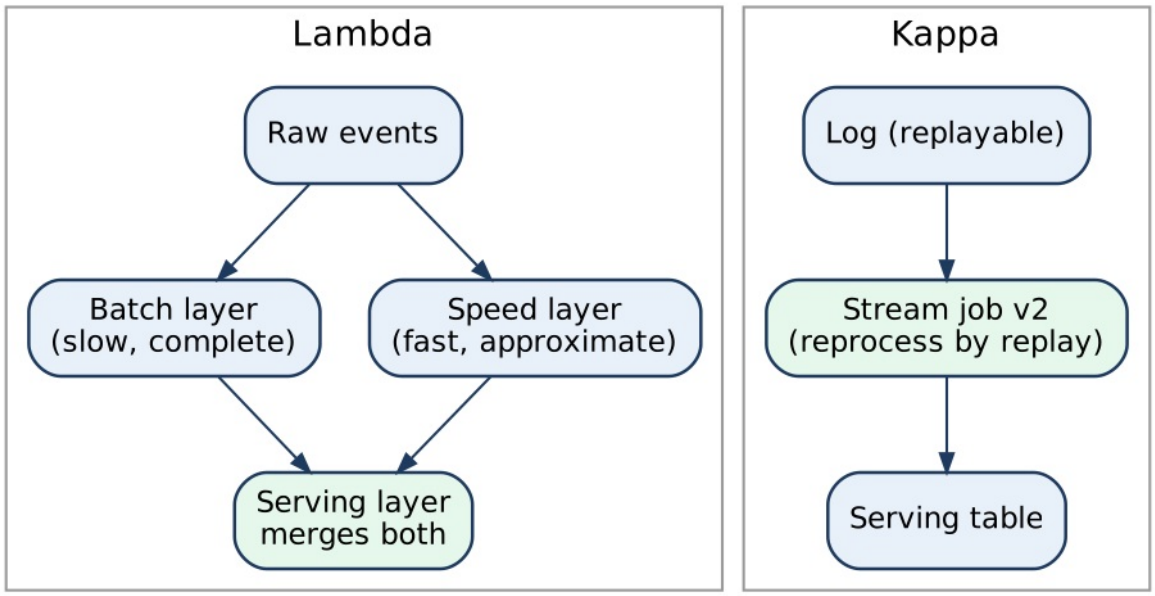
The idea in plain words

> **Plain words.** A weather station records the temperature every second. For last week's chart, you do not need every second; one reading per hour is enough. Keep detail for recent days, and summaries for older years.

The key ideas

- **Series:** one metric plus its labels (for example, CPU on host 17 in region eu). Limit labels to a bounded set, such as service and region. Never use a user ID as a label, because the number of series explodes.
- **Compression:** timestamps arrive at regular intervals, so store the change in the change. Float values are stored as the XOR of consecutive values. Together, this takes a couple of bytes per point.
- **Downsampling:** keep raw points for 7 days, one-minute summaries for 90 days, one-hour summaries for 2 years. Store min, max, sum, and count so averages stay correct.
- **Percentiles** cannot be averaged across hosts. Use histograms or sketches instead.

Figure 22: Lambda (two paths) vs Kappa (one stream path)



Lambda (two paths merged) vs Kappa (one replayable stream). Kappa is simpler when the log is kept long enough.

> **Common mistake.** Averaging averages. An average of hourly averages is not the true average when hours have different numbers of events.

Going deeper

Ingestion. Agents batch points and send them every 10 seconds. A write-ahead log makes the write durable before it is acknowledged; an in-memory buffer then forms time-ordered blocks.

Query shape. Most queries ask for a metric over a time range, grouped by a few labels. Storing blocks sorted by series and time makes these reads sequential.

Cardinality. The number of distinct series is the main cost driver. Each new label value creates a new series. Set limits per tenant and alert on growth.

Alerting on metrics. Evaluate rules over windows, not single points, to avoid flapping alerts.

Interview follow-up

- **Q: Why not store metrics in a relational database?** A: Row-per-point storage has poor compression and slow range scans at this write rate.
- **Q: How do you answer a p99 over a long range?** A: Use histogram buckets stored per interval and merge them; percentiles of percentiles are wrong.

Complete interview coverage

What the interviewer checks: that you can design write-heavy time-series storage and control its cost.

A complete answer you should be able to give: Agents batch points and send them to a write-ahead log, then blocks are stored sorted by series and time, compressed, and downsampled as they age. Keep label cardinality bounded, because the number of series drives cost. Store histograms or sketches for percentiles, since percentiles cannot be averaged.

Follow-up questions and answers:

- **Q: Why not a relational database?**
  A: Row-per-point storage compresses poorly and scans slowly at this write rate.
- **Q: How do you answer a p99 over a long range?**
  A: Merge stored histogram buckets; never average percentiles.

Mistakes that cost points: Using user IDs as labels; averaging averages.

Level: Intermediate · About 45 minutes

Lesson 26: Video streaming and the CDN

Goal

Explain how an uploaded video becomes a stream that plays smoothly on a slow phone, and why most of the work happens at the network edge.

The idea in plain words

Plain words. A film is cut into short pieces, each available in several quality levels. Your player picks the quality for the next piece based on how fast your connection is right now. That is adaptive bitrate (ABR).

From upload to playback

1. The upload goes straight to object storage in resumable parts.
2. Transcoding workers create several resolutions and codecs, in parallel chunks.
3. Each rendition is cut into segments of 2 to 6 seconds, with a manifest that lists them.
4. The CDN caches segments close to viewers. Popular videos are served almost entirely from the edge.
5. The player downloads a manifest, then fetches segments and switches quality as needed.

Key idea. Bandwidth is the main cost. The CDN hit ratio is the metric that matters most. Origin shielding stops a newly viral video from overwhelming storage.

Common mistake. Serving the original file. Devices and networks vary too much; you need several renditions.

Views count without slowing playback

Do not write one database row per view: at a billion views a day, that is about 11,500 writes per second. Clients batch view events; a stream job aggregates them per minute; the total is written periodically. Displayed counts can be rounded ("1.2M").

Going deeper

Encoding ladder. A typical ladder produces several resolutions (for example 240p to 4K) and bitrates per resolution. Smarter encoding per scene (content-aware) saves bandwidth on simple scenes.

Live streaming. Live video uses the same segment model with much shorter latency: segments of 1 to 2 seconds, and a smaller player buffer. Latency goes up as buffer goes down, so the design trades smoothness for delay.

DRM and access. Encrypted segments with keys delivered only to authorised players. Signed URLs or tokens expire quickly.

Failure handling. A failed transcode job retries from the last completed chunk, not from the start.

Interview follow-up

- **Q: Why store several renditions instead of transcoding on demand?** A: On-demand transcoding adds latency to the first play and repeats work per viewer; precomputing is cheap per view.
- **Q: What is a manifest?** A: A playlist (HLS or DASH) that lists the renditions and segment URLs for the player.

Complete interview coverage

What the interviewer checks: that you can explain adaptive streaming, the CDN role and the cost model.

A complete answer you should be able to give: Transcode each upload into several renditions, cut them into short segments listed in a manifest, and serve them from a CDN close to viewers. The player switches quality based on measured bandwidth. Bandwidth dominates cost, so the CDN hit ratio and origin shielding

matter most. View counts are aggregated, not written per view.

Follow-up questions and answers:

- **Q: Why precompute renditions?**
  A: On-demand transcoding delays the first play and repeats work for every viewer.
- **Q: What changes for live streaming?**
  A: Shorter segments and a smaller buffer, trading smoothness for lower delay.

Mistakes that cost points: Serving only the original file; writing one database row per view.

Level: Intermediate to advanced · About 45 minutes

Lesson 27: Job schedulers and workflows

Goal

Run jobs at a set time, retry them safely, and manage long multi-step workflows that survive crashes.

The idea in plain words

> **Plain words.** A hospital ward has a list of tasks with times. A nurse "claims" a task by writing their name on it for 10 minutes. If the nurse collapses, the claim expires and another nurse takes over. The task must be safe to do twice, in case the first nurse was just slow.

How claiming works

1. The scheduler finds jobs that are due and not claimed.
2. A worker claims a job with a lease (an owner and an expiry time). The claim is atomic.
3. The worker sends heartbeats to extend the lease while it works.
4. If the worker dies, the lease expires and another worker can claim the job.
5. Each claim gets a higher version number, which the job's side effects can check (fencing).

> **Key idea.** Expect at-least-once execution. Make job handlers idempotent, and use the version number to reject work from an old claim.

Durable workflows

A long workflow (for example, "onboard a customer over 7 days") cannot live in one worker's memory. Durable execution engines (such as Temporal) record every step in a history. After a crash, another worker replays the history to rebuild the state. The workflow code must be deterministic: no direct random numbers or current-time calls; use the engine's versions.

> **Common mistake.** Scheduling every job on one cron server. It is a single point of failure, with no retries and no visibility.

Going deeper

Cron at scale. Store scheduled jobs in a database partitioned by next-run time. A scanner loads the next minute's jobs into a queue. This avoids one machine holding millions of timers in memory.

Missed runs. If the scheduler was down, decide per job: run once now, run every missed slot, or skip. Name the policy per job type.

Workflow history. Durable engines store every activity result. Replay re-runs the workflow code but uses stored results for completed steps, so side effects are not repeated.

Timeouts and cancellation. Every activity has a timeout and a retry policy. Cancellation sets a flag the workflow checks at each step.

Interview follow-up

- **Q: How do you run a job exactly at 09:00 for a million users?** A: Spread the start across a window (jitter), queue per user, and rate-limit downstream calls.
- **Q: Why not a sleep in the workflow?** A: A sleep holds a worker. Durable timers persist and free the worker until due.

Complete interview coverage

What the interviewer checks: that you can run scheduled and long multi-step work reliably.

A complete answer you should be able to give: Store jobs in a database partitioned by next-run time, and have workers claim due jobs with leases that expire if the worker dies. Make every handler idempotent and use

a version to reject stale workers. For long workflows, use a durable execution engine that records each step, so a crash resumes instead of restarting.

Follow-up questions and answers:

- **Q: Why not a sleep in a workflow?**
  A: It holds a worker; durable timers persist and release the worker.
- **Q: What about a missed run?**
  A: Choose a policy per job: run once, run every missed slot, or skip.

Mistakes that cost points: One cron machine as the single point of failure; non-idempotent job handlers.

PART 5 · ADVANCED

Architecture and Operations

How large systems are organised, observed, secured, deployed, and kept alive across regions.

- Lesson 28: Architecture styles and when to split a system
- Lesson 29: CQRS and event sourcing
- Lesson 30: Observability and SLOs
- Lesson 31: Safe deployment
- Lesson 32: Security in design
- Lesson 33: Multi-region and disaster recovery

Level: Advanced · About 40 minutes

Lesson 28: Architecture styles and when to split a system

Goal

Choose between a monolith, services, and event-driven parts, and justify the choice with team and load facts.

The idea in plain words

> **Plain words.** A single kitchen can cook for a family; a restaurant chain needs separate kitchens. Splitting a kitchen too early means more couriers, more coordination, and more things that go wrong. Split when a real limit forces you to, not because it looks modern.

Comparing styles

Style	Good when	Costs
Modular monolith	Small team, one deploy, shared data	Discipline needed to keep modules apart
Services	Teams must ship independently; parts scale differently	Network failures, tracing, data ownership, versioning
Event-driven	Many consumers react to one fact; loose coupling matters	Harder to trace flows; eventual consistency

Steps to decide

1. Name the team sizes and how often each part must deploy alone.
2. Name the parts whose load differs by 10x or more (for example, search vs checkout).
3. Draw data ownership: each piece of data has one owner that writes it.
4. Only then split. Start with a modular monolith when unsure, and cut along the module boundaries later.

> **Key idea.** Services are split by data ownership and team boundaries, not by technical layers ("a database service", "a validation service").

> **Common mistake.** Starting with twenty microservices for a product with five engineers. Each service needs its own deploy, monitoring, and on-call.

Going deeper

Conway's law in practice. System boundaries tend to follow team boundaries. If one team owns checkout end to end, a checkout service makes sense; if three teams share one table, that is a sign the boundary is wrong.

Costs of distribution. Each network call adds latency, failure modes and a timeout to tune. A call that was a function call in a monolith becomes a retryable, observable, versioned API.

Modular monolith path. Enforce module boundaries with code ownership and rules that forbid importing another module's internals. Extract a module into a service only when a real limit forces it.

Interview follow-up

- **Q: How many services is too many?** A: When each team cannot run its services with its own on-call and deploy, you have more services than you can operate.
- **Q: Monolith or microservices for a new startup?** A: A modular monolith, unless a concrete scale or team constraint already exists.

Complete interview coverage

What the interviewer checks: that you can justify an architecture style by team, load and data ownership.

A complete answer you should be able to give: Start with a modular monolith unless a team or load constraint already requires separate services. Split along data ownership and team boundaries, so each service owns the data it writes. Accept the cost of network calls, versioning and observability that each split adds, and extract services only where a real limit forces it.

Follow-up questions and answers:

- **Q: How many services is too many?**
  A: When teams cannot run their own deploys and on-call for them.
- **Q: Monolith or microservices for a startup?**
  A: A modular monolith, since team size and load rarely justify more.

Mistakes that cost points: Splitting by technical layer; splitting before there is a reason.

Level: Advanced · About 50 minutes

Lesson 29: CQRS and event sourcing

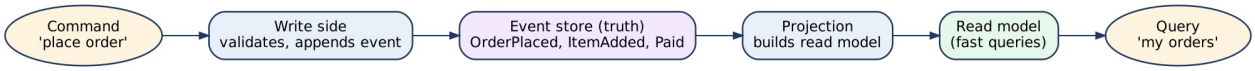
Goal

Separate writes from reads when they need different shapes, and store changes as a history when auditing or replay matters.

The idea in plain words

Plain words. A bank statement is a list of every transaction. Your balance is a summary computed from that list. Event sourcing stores the list (the truth). CQRS keeps a separate summary for quick reading, updated from the list.

Figure 16: CQRS + event sourcing



Commands write events to a log; projections build read models for queries.

Steps

1. A command ("withdraw 50") is validated against the current state.
2. If valid, an event ("withdrawn 50, at version 7") is appended to the log, guarded by the expected version.
3. Projections read the log and update read models (for example, a balance table or a search index).
4. Snapshots every 1,000 events keep rebuilding fast.

Key idea. Read models are disposable. If one is wrong, delete it and rebuild it from the log.

Common mistake. Using event sourcing for everything. Most CRUD data does not need a history, and the extra complexity is real. Use it where audit, replay, or time-travel queries are core.

Going deeper

Snapshots. Replaying a million events on every read is slow. Store a snapshot of the aggregate every N events; load the snapshot, then replay only newer events.

Event schema evolution. Events are stored forever, so changing their shape is hard. Add fields, never rename or remove. Use upcasters that translate old events when read.

Projections and rebuilds. A new read model is built by replaying the log from the start. Plan for the time this takes and the load it places on the store.

Eventual consistency. Read models lag the log. Show the user their own write optimistically, or read from the write side for that user.

Interview follow-up

- **Q: Is CQRS always event sourcing?** A: No. CQRS can use a normal database with a separate read table. Event sourcing is an extra choice.
- **Q: How do you delete personal data in an immutable log?** A: Store personal fields encrypted with a per-user key, and delete the key (crypto-shredding).

Complete interview coverage

What the interviewer checks: that you know when event sourcing and CQRS are worth their cost.

A complete answer you should be able to give: Use CQRS when reads and writes need different shapes or scale, and event sourcing when history, audit or replay is central. The event log is the source of truth; read models are rebuilt from it. Plan for schema evolution, snapshots for long histories, and eventual consistency

between writes and reads.

Follow-up questions and answers:

- **Q: Is CQRS the same as event sourcing?**
  A: No; CQRS can use an ordinary database with separate read tables.
- **Q: How do you delete personal data from a log?**
  A: Encrypt personal fields per user and delete the key.

Mistakes that cost points: Using event sourcing for simple CRUD data.

Level: Advanced · About 45 minutes

Lesson 30: Observability and SLOs

Goal

Measure what users feel, alert on it early, and decide when to spend reliability budget on new features.

The idea in plain words

> **Plain words.** A car dashboard shows speed, fuel, and warning lights, so you know the car is fine before it stops. Observability is that dashboard for software. An SLO is the promise you make: "99.9% of requests succeed within 300 ms".

The three signals

- **Metrics:** numbers over time (request rate, error rate, latency percentiles).
- **Logs:** detailed events, searchable by request ID.
- **Traces:** one request's path across services, with timings for each hop.

Error budget

A 99.9% monthly SLO allows about 43 minutes of failure per 30 days. That is the error budget. If it is spent, freeze risky releases. If plenty remains, ship faster.

Burn-rate alerts

Alert when the budget is burning fast, for example if 2% of it is gone in one hour. A single short spike does not wake anyone; a sustained burn does.

> **Key idea.** Use percentiles (p95, p99), not averages. An average hides the slow tail that real users hit.

> **Common mistake.** Alerting on CPU. Users do not feel CPU; they feel slow or failed requests. Alert on symptoms, and use CPU for diagnosis.

Going deeper

Golden signals. Latency, traffic, errors and saturation. Start every dashboard with these four for each user-facing service.

Distributed tracing. Pass a trace ID through every call. A slow request then shows which hop took the time, instead of a set of service graphs that all look normal.

Structured logs. Use key-value logs with request IDs. Free text logs are hard to query during an incident.

Sampling. Keep all errors and a sample of successful traces. Logging everything at high traffic costs more than the service.

Multi-window burn alerts. Alert when the budget burns fast over a short window and also over a longer one. This catches both sudden outages and slow leaks while ignoring one-off blips.

Interview follow-up

- **Q: Why is average latency misleading?** A: A few very slow requests barely move the average but ruin the experience for those users. Look at p95 and p99.
- **Q: What do you do when an alert fires at 3 a.m.?** A: Check the user impact on the dashboard, mitigate first (roll back, shed load), then investigate with traces and logs.

Complete interview coverage

What the interviewer checks: that you measure what users feel and alert on it.

A complete answer you should be able to give: Define SLOs for user-facing behaviour, such as 99.9

percent of requests succeed within 300 ms, and track the error budget. Alert on fast burn over short and long windows, not on CPU. Use metrics for trends, traces for the slow path of one request, and structured logs with request IDs for detail.

Follow-up questions and answers:

- **Q: Why percentiles?**
  A: Averages hide the slow tail that affects real users.
- **Q: What do you do at 3 a.m.?**
  A: Check user impact, mitigate first (roll back or shed load), then investigate.

Mistakes that cost points: Alerting on causes like CPU instead of user symptoms.

Level: Advanced · About 40 minutes

Lesson 31: Safe deployment

Goal

Ship changes to millions of users without outages, and roll back quickly when something goes wrong.

The idea in plain words

> **Plain words.** Before you sell a new recipe to the whole restaurant, you serve it to one table. If the table is happy, you serve more tables. If something goes wrong, only one table noticed.

Strategies

Strategy	How it works	Trade-off
Canary	Send 1%, then 10%, then 100% of traffic to the new version	Needs good metrics to judge the canary
Blue-green	Run two full copies; switch traffic at once	Double the capacity during the switch
Rolling	Replace instances a few at a time	Old and new versions run together

Database changes: expand and contract

1. **Expand:** add the new column or table; write to both old and new.
2. **Migrate:** backfill old rows in small batches.
3. **Switch:** read from the new place.
4. **Contract:** remove the old column, only after a full release cycle.

> **Key idea.** Every change must be compatible with the version before and after it. Old and new code run side by side during a rollout.

> **Common mistake.** Renaming a column in one deploy. The old code still reads the old name, and the rollout fails.

Going deeper

Automated canary analysis. Compare canary and baseline on error rate and latency over the same window. Promote only if the difference is within a threshold; roll back automatically otherwise.

Feature flags. Ship code dark, turn features on per user group. A flag is a runtime switch that is independent of the deploy, so a bad feature can be turned off in seconds.

Rollback limits. A rollback is safe only if the old version can read the data the new version wrote. This is why schema changes follow expand and contract.

Migrations at scale. Backfill in batches with a rate limit, monitor replication lag, and make the backfill resumable.

Interview follow-up

- **Q: Blue-green or canary for a stateful database?** A: Neither copies data cheaply; use expand and contract with dual writes, then switch reads.
- **Q: How do you deploy a change that needs two services updated together?** A: Make each change backward compatible first, deploy both, then remove the compatibility path.

Complete interview coverage

What the interviewer checks: that you can change production safely and roll back.

A complete answer you should be able to give: Ship with a canary that receives a small share of traffic and is compared with the baseline on error rate and latency, with automatic rollback. Use feature flags to turn

behaviour on and off without a deploy. Make schema changes with expand and contract, so old and new code run side by side.

Follow-up questions and answers:

- **Q: Blue-green or canary?**
  A: Canary for gradual evidence; blue-green for an instant switch with double capacity.
- **Q: Why expand and contract?**
  A: Renaming a column in one step breaks the old code that is still running.

Mistakes that cost points: A rollback that cannot read data written by the new version.

Level: Advanced · About 40 minutes

Lesson 32: Security in design

Goal

Design authentication, authorization, and data protection into the system from the start, and name the common attacks.

The idea in plain words

> **Plain words.** Authentication asks "who are you?" (a badge at the door). Authorization asks "may you enter this room?" (the badge must open that door). Many breaches happen because the badge works everywhere.

Core pieces

- **Login:** OAuth 2.0 and OpenID Connect hand users to an identity provider. Your service receives a short-lived token.
- **Tokens:** JWTs are signed, so services verify them without a call. Keep them short-lived (minutes) and revoke via refresh tokens.
- **Service to service:** mutual TLS, where both sides present certificates.
- **Secrets:** stored in a secrets manager, rotated, never in code.
- **Data:** encrypt in transit (TLS) and at rest; keep personal data minimal.

Attacks to design against

- **IDOR:** changing `/orders/123` to `/orders/124` and seeing someone else's order. Check ownership on every object access.
- **Rate abuse:** limit logins and expensive endpoints (Lesson 13).
- **Injection:** never build SQL by joining strings; use parameters.

> **Key idea.** Authorize every object access, not just the entry point. Most real breaches are "logged in but allowed to see too much".

Going deeper

Authorization models. Role-based access (RBAC) assigns permissions to roles. Attribute-based (ABAC) uses attributes such as department and location. Relationship-based (ReBAC) asks whether a user is related to an object, such as owner of a folder. Choose by how complex the rules are.

Token handling. Store refresh tokens securely, rotate them on each use, and detect reuse (a sign of theft). Access tokens are short-lived and checked by signature without a database call.

Secrets and keys. Use a managed key service, rotate keys on a schedule, and never log secrets. Encrypt data with keys the application cannot export.

Input and output. Validate input against a schema; encode output for its context (HTML, SQL, shell). Set security headers and CSRF protection on browser flows.

Interview follow-up

- **Q: Where do you enforce authorization?** A: At the service that owns the data, on every request, not only at the gateway. Gateways can be bypassed internally.
- **Q: How do you prevent credential stuffing?** A: Rate limit login per account and per IP, use breached-password checks, and add multi-factor authentication.

Complete interview coverage

What the interviewer checks: that you can design authentication, authorization and data protection.

A complete answer you should be able to give: Authenticate users with OAuth 2.0 and OpenID Connect, issue short-lived tokens, and rotate refresh tokens with reuse detection. Authorize every object access at the service that owns the data, which prevents IDOR. Protect service-to-service calls with mutual TLS, keep secrets in a managed store, and encrypt data in transit and at rest.

Follow-up questions and answers:

- **Q: Where do you check authorization?**
  A: On every request at the owning service, not only at the gateway.
- **Q: How do you prevent credential stuffing?**
  A: Rate limits per account and IP, breached-password checks and multi-factor authentication.

Mistakes that cost points: Checking only that the user is logged in, not that they own the object.

Level: Advanced · About 50 minutes

Lesson 33: Multi-region and disaster recovery

Goal

Keep the system running when a whole region fails, and state the recovery time and data loss you accept.

The idea in plain words

Plain words. A company keeps a copy of its records in a second city. If the first office burns, the second one opens. The question is how long the move takes (RTO) and how many recent records might be lost (RPO).

Patterns

Pattern	RTO	RPO	Cost
Backup and restore	Hours	Hours	Low
Warm standby	Minutes	Seconds to minutes	Medium
Active-active	Near zero	Near zero	High, and conflicts to solve

Steps

1. Write down RTO and RPO per service. Not every service needs the same.
2. Choose the pattern that meets them at the lowest cost.
3. Replicate data across regions; accept asynchronous lag if RPO allows it.
4. Route users with DNS or a global load balancer; test failover regularly.
5. Keep state for each user in one home region to avoid conflicting writes.

Key idea. A disaster plan that has never been rehearsed is a guess. Practice failover on a schedule.

Common mistake. Active-active writes to the same record in two regions without a conflict rule. Two regions can accept contradictory updates.

Going deeper

Choosing RTO and RPO. Tie each to money: how much revenue an hour of downtime costs, and how much data loss is tolerable for each dataset. A payment ledger usually has near-zero RPO; an analytics table may tolerate hours.

Data residency. Some data must stay in a country. Design the home region per user or tenant so the data never leaves it, and replicate only permitted data.

Failover drills. Run planned failovers on a schedule, measure the real RTO, and record the runbook steps that were slow or unclear.

Dependencies. A region is only as available as its shared dependencies: DNS, identity, the deployment pipeline. List them in the design.

Interview follow-up

- **Q: What if the primary region is only partly down?** A: Partial failures are harder than full ones. Use health signals per service and shift traffic gradually, with clear rules for when to fail over.
- **Q: Why not always active-active?** A: It doubles cost and adds conflict handling. Use it where the business value justifies it.

Complete interview coverage

What the interviewer checks: that you can plan for region failure with stated recovery targets.

A complete answer you should be able to give: State RTO and RPO per dataset and tie them to the business cost of downtime. Choose backup, warm standby or active-active to meet them at the lowest cost. Keep each user's writes in a home region to avoid conflicts, route traffic with health checks, and run failover drills to measure real recovery time.

Follow-up questions and answers:

- **Q: Why not always active-active?**
  A: It doubles cost and adds conflict handling; use it only where the value justifies it.
- **Q: What about partial failure?**
  A: Shift traffic gradually using per-service health signals and clear failover rules.

Mistakes that cost points: Promising zero data loss and zero downtime without the cost of replication.

PART 6 · LOW-LEVEL DESIGN

Object-Oriented Design

Classes, patterns, and concurrency, applied to classic interview problems. Every coded example in this part was run and its output checked.

- Lesson 34: Design principles (SOLID) and a method
- Lesson 35: Patterns you will actually use
- Lesson 36: Concurrency in LLD
- Lesson 37: Parking lot
- Lesson 38: LRU cache
- Lesson 39: Vending machine and elevator (state)
- Lesson 40: Splitwise (debt simplification)
- Lesson 41: Movie ticket booking

Level: Beginner to intermediate · About 40 minutes

Lesson 34: Design principles (SOLID) and a method

Goal

Turn a vague problem into classes using a fixed method, and check the result against five principles.

The method, step by step

1. **Clarify (3 minutes):** ask about actors, actions, scale of one machine, concurrency, and what is out of scope. Write the answers down.
2. **List nouns and verbs:** nouns become classes (Vehicle, Spot, Ticket). Verbs become methods (park, leave). Adjectives become fields or enums (size, status).
3. **Relationships:** "has a" is composition (a Lot has Spots). "is a" is inheritance (a Truck is a Vehicle), and you should use it sparingly.
4. **Name the varying behaviour:** pricing, eviction, seat selection. These become interfaces or strategy objects.
5. **Walk one scenario end to end** against your classes. Most design bugs show up here.
6. **Handle concurrency and failure** last, but say you will.

SOLID in plain words

Letter	Principle	Plain words	Parking lot example
S	Single responsibility	One reason to change	Pricing is not inside Vehicle
O	Open/closed	Add behaviour by adding a class	Add a weekend price class; no edit to the lot
L	Liskov substitution	A subtype can replace its parent	Any Vehicle subtype can be parked and charged
I	Interface segregation	Small, focused interfaces	Gate reader does not need payment methods
D	Dependency inversion	Depend on interfaces, not concrete classes	Lot takes any pricing function, not a fixed rate table

Worked example: the question "should this be inheritance?"

Suppose you start with a class hierarchy: Car, Truck, Bike, each with its own fee method. Now the business adds "electric car at a discount". You must edit the Car class, and every new rule adds a branch or a subclass. Instead, keep the vehicle a simple data object and pass the pricing rule in. Adding a rule then adds one function and no edits to existing code. This is the open/closed principle in practice.

Interview follow-ups

- **"Why not inheritance for vehicles?"** Vehicles differ by size, not by behaviour. A size field plus a pricing strategy is simpler.
- **"How do you test this?"** Inject the pricing function and the clock. Then a test controls both.

> **Key idea.** Prefer composition over inheritance. Keep rules you expect to change behind an interface.

> **Common mistake.** Typing code before clarifying. Ten minutes of questions saves thirty minutes of rewrites.

Complete interview coverage

What the interviewer checks: that you can turn a vague problem into a class design with a repeatable method.

A complete answer you should be able to give: Clarify actors, actions and rules, then list nouns as classes and verbs as methods. Identify the behaviour that varies and put it behind an interface. Walk one scenario end to end, then check the design against SOLID. Mention concurrency and failure handling last but explicitly.

Follow-up questions and answers:

- **Q: Why not inheritance for vehicles?**
  A: Vehicles differ by size and rules, which composition expresses without a fragile hierarchy.
- **Q: How do you test the design?**
  A: Inject the pricing and clock dependencies so tests control them.

Mistakes that cost points: Coding before clarifying; deep inheritance trees.

Level: Intermediate · About 45 minutes

Lesson 35: Patterns you will actually use

Goal

Recognise the few patterns that show up in most interview problems, and know the signal that calls for each.

The idea in plain words

> **Plain words.** A pattern is a named shape you have seen before. Strategy is "swap the rule without touching the user". State is "the same button does different things in different modes". Name the pattern only after you have the shape in mind.

The patterns, with their signals

Pattern	Signal in the problem	Example
Strategy	"Rules vary" or "algorithm can change"	Pricing, eviction policy, seat ranking
State	"Behaviour depends on the current mode"	Vending machine, elevator, order status
Observer	"When X happens, notify several parties"	Booking confirmation to email, SMS, app
Factory	"Create the right subtype from a input"	Vehicle types from a request
Command	"Undo", "queue", or "log actions"	Text editor undo, job queue
Chain of responsibility	"Each handler may pass the request on"	Support escalation levels

Strategy vs State, the most common confusion

Both swap an object's behaviour. The difference is who decides the swap. With **Strategy**, the caller picks the algorithm (the customer chooses a discount rule). With **State**, the object changes its own behaviour as events happen (a vending machine moves from Idle to HasMoney after a coin). If the object transitions itself, use State.

Interview follow-ups

- **"Is this over-engineered?"** If there are only two cases, use a plain if statement and say so. Patterns earn their place when a third case is likely.
- **"How do you add a new case?"** Show that the pattern adds one class and touches nothing else.

> **Key idea.** In an interview, explain why a pattern fits: "I used Strategy so pricing can change without editing booking." A name alone earns little credit.

> **Common mistake.** Forcing a pattern where a simple condition is clearer.

Complete interview coverage

What the interviewer checks: that you recognise the common patterns and can say why each fits.

A complete answer you should be able to give: Strategy when interchangeable rules must vary, State when behaviour depends on a mode the object moves through, Observer when many parties react to one event, Factory when the type is chosen from input, Command when actions must be queued or undone. Name the pattern only after the shape fits, and say what it lets you change without editing.

Follow-up questions and answers:

- **Q: Strategy or State?**
  A: Strategy: the caller picks the algorithm. State: the object changes its own behaviour.
- **Q: When is a pattern overkill?**
  A: When there are only two cases and no sign of a third.

Mistakes that cost points: Naming patterns without a reason.

Level: Intermediate to advanced · About 45 minutes

Lesson 36: Concurrency in LLD

Goal

Make shared state safe when many threads act at once, while keeping the locked part as small as possible.

The idea in plain words

> **Plain words.** Two people reach for the last seat in a cinema. Each checks "is it free?" and sees yes. Both sit down. The check and the sit must be one indivisible step, or someone else slips in between. This is a check-then-act race.

Tools and when to use them

- **Lock** around exactly the check-and-act pair. Keep the critical section short: no network calls or printing inside the lock.
- **Conditional update** (compare-and-set): "set to booked only if still free". This is the database version of the same idea.
- **Single-flight:** when many threads ask for the same missing value, compute it once and share the result.
- **Immutable objects**: nothing to protect when nothing changes.

Worked example: why the lock must cover both steps

If you check availability under a lock but mark the seat outside it, a second thread can pass the check between your two steps. The rule: the lock must cover the read and the write together.

Interview follow-ups

- **"Why not lock the whole system?"** It serialises every request and removes the point of concurrency. Lock per resource where you can.
- **"What about deadlock?"** Acquire locks in a fixed order, and keep each critical section to one lock where possible.

> **Key idea.** Name the shared state and the check-then-act pair. Lock exactly that.

> **Common mistake.** Locking around a slow external call, which turns a fast system into a queue.

Complete interview coverage

What the interviewer checks: that you can make shared state safe without serialising everything.

A complete answer you should be able to give: Identify the shared state and the check-then-act pair, then protect exactly that pair with a lock or a conditional update. Keep the critical section short, avoid calls to slow services inside it, and acquire multiple locks in a fixed order to prevent deadlock. Use single-flight when many threads want the same computed value.

Follow-up questions and answers:

- **Q: Why not lock the whole object?**
  A: It serialises unrelated requests and removes the benefit of concurrency.
- **Q: What is single-flight?**
  A: One caller computes a missing value while others wait for and share the result.

Mistakes that cost points: A lock that covers the read but not the write.

Level: Beginner to intermediate · About 60 minutes

Lesson 37: Parking lot

Goal

Design a parking lot with several vehicle sizes, pricing, and safe concurrent entry, and run it.

Step 1: Clarify

- Vehicle types: bike, car, truck. Can a truck use a car spot? Assume a bigger spot can take a smaller vehicle.
- Pricing: per hour, rounded up. Can pricing change by time of day? (We will allow it through a strategy.)
- Entrances and exits: several gates, one ticket per entry, one exit payment.
- When full: reject the entry with a clear reason.

Step 2: Classes and relationships

Class	Responsibility
VehicleType	The kinds of vehicle and which spot sizes fit them
Spot	One space with a size and free flag
ParkingLot	Owns spots, issues tickets, enforces the lock
Pricing (function or strategy)	Turns vehicle type and duration into a fee

Step 3: Allocation and concurrency

Keep free spots in a list per size. Assigning a spot removes it from the list, which is a check-then-act step. One lock around park and leave makes it safe (Lesson 36). Pick the smallest fitting size first, so trucks are not blocked by bikes parking in big spaces.

Step 4: The code (verified)

```
class VehicleType(Enum):
    BIKE = 1; CAR = 2; TRUCK = 3

class Spot:
    def __init__(self, sid, size):
        self.id, self.size, self.free = sid, size, True

class ParkingLot:
    """Allocation is guarded by one lock: check-then-act must be atomic."""
    FIT = {VehicleType.BIKE: (1, 2, 3), VehicleType.CAR: (2, 3), VehicleType.TRUCK: (3,)}
    def __init__(self, spots, pricing):
        self._lock = threading.Lock()
        self._free = {s: [] for s in (1, 2, 3)}
        self._tickets = {}
        self._pricing = pricing          # Strategy: callable(vtype, minutes) -> cents
        for sp in spots:
            self._free[sp.size].append(sp)
    def park(self, vtype, ticket_id, now):
        with self._lock:
            for size in self.FIT[vtype]:           # smallest fitting spot first
                if self._free[size]:
                    spot = self._free[size].pop()
                    self._tickets[ticket_id] = (spot, vtype, now)
                    return spot.id
            raise RuntimeError("no spot available")
    def leave(self, ticket_id, now):
        with self._lock:
            spot, vtype, start = self._tickets.pop(ticket_id)
            self._free[spot.size].append(spot)
            minutes = max(1, -(-(now - start) // 60))     # round up to whole minutes
            return self._pricing(vtype, minutes)

def hourly(vtype, minutes):
    rate = {VehicleType.BIKE: 20, VehicleType.CAR: 50, VehicleType.TRUCK: 120}[vtype]  # cents/hour
    return rate * -(-minutes // 60)

lot = ParkingLot([Spot(1,1), Spot(2,2), Spot(3,3), Spot(4,2)], hourly)
print("car ->", lot.park(VehicleType.CAR, "T1", 0))
print("truck ->", lot.park(VehicleType.TRUCK, "T2", 0))
print("fee for T1 after 150 min (cents):", lot.leave("T1", 150*60))
```

```
try:
    lot.park(VehicleType.TRUCK, "T3", 0)
except RuntimeError as e:
    print("truck again ->", e)
```

Step 5: Output from the run

```
car -> 4
truck -> 3
fee for T1 after 150 min (cents): 150
truck again -> no spot available
```

Step 6: Walk one scenario by hand

A car arrives at time 0 and takes spot 4 (a size 2 spot). It leaves after 150 minutes. Duration rounds up to 150 minutes, which is 3 billed hours at 50 cents each, so the fee is 150 cents. Spot 4 returns to the free list. A truck arriving next finds size 3 spot 3 free and takes it.

Interview follow-ups

- **"Add EV charging spots."** Add a flag on Spot and a filter on allocation. The pricing does not change.
- **"Add weekend pricing."** Pass a different pricing function. Nothing else changes (Lesson 34).
- **"What if the lot is distributed across floors?"** Track free spots per floor and per size, and choose a floor first.
- **"How do you avoid two cars getting the same spot?"** The lock. Explain why check-then-act needs both steps inside it.

> **Key idea.** Keep allocation separate from pricing. Changing the price rule must not touch spot assignment.

> **Common mistake.** Putting fee logic inside Vehicle. A vehicle should not know how the lot charges.

Complete interview coverage

What the interviewer checks: that you can design a parking lot with allocation, pricing and concurrency separated.

A complete answer you should be able to give: Model vehicles, spots by size, floors and tickets. Allocate the smallest fitting free spot under a lock so two cars never get the same spot. Put pricing behind a strategy so rules change without touching allocation. Explain how a full lot is reported and how EV spots or new pricing would be added.

Follow-up questions and answers:

- **Q: Where does pricing live?**
  A: In a strategy outside the vehicle; vehicles should not know how they are charged.
- **Q: How do you scale to many gates?**
  A: Shared spot state behind one atomic assignment, or per-floor partitions with a floor chosen first.

Mistakes that cost points: Fee logic inside the vehicle class.

Level: Intermediate · About 50 minutes

Lesson 38: LRU cache

Goal

Build a cache that evicts the least recently used item, with get and put in constant time, and explain the structure behind it.

The idea in plain words

> **Plain words.** A desk with room for five books. When you pick a book up, you put it on top. When a sixth book arrives, you remove the one that has sat at the bottom the longest.

Why two structures

- A **hash map** gives O(1) lookup of a key.
- A **doubly linked list** gives O(1) move-to-front and O(1) removal, because each node knows its neighbours.
- Together: the map finds the node, the list orders it. Neither alone is enough: a map cannot tell recency, and a list cannot find a key quickly.

Step by step

1. **get(k):** if missing, return nothing. If present, move its node to the front and return the value.
2. **put(k, v):** if present, update and move to front. Otherwise insert at the front; if over capacity, remove the tail node and delete its key from the map.

Code (verified)

Python's OrderedDict is a hash map with a linked list inside, so it does both jobs. In an interview, say so, then describe the two-structure version on the whiteboard.

```
class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.d = OrderedDict()          # key -> value; order = recency
        self.lock = threading.Lock()
    def get(self, key):
        with self.lock:
            if key not in self.d:
                return None
            self.d.move_to_end(key)     # most recent at the end
            return self.d[key]
    def put(self, key, value):
        with self.lock:
            if key in self.d:
                self.d.move_to_end(key)
            self.d[key] = value
            if len(self.d) > self.cap:
                self.d.popitem(last=False)   # evict least recent

c = LRUCache(2)
c.put("a", 1); c.put("b", 2)
c.get("a")                 # a is now most recent
c.put("c", 3)              # evicts b
print("LRU get b ->", c.get("b"), "| get a ->", c.get("a"), "| get c ->", c.get("c"))
```

Output from the run

```
LRU get b -> None | get a -> 1 | get c -> 3
```

Walk one scenario by hand

Capacity 2. put a, put b: order is b (newest), a. get a: order becomes a, b. put c: capacity exceeded, so the least recent, b, is evicted. get b misses. That is exactly the printed result.

Interview follow-ups

- **"Make it thread-safe."** One lock around get and put, since both reorder the list. Note that get writes too, so a read-write lock does not help here.
- **"Add a TTL."** Store the expiry with each value and check it on get; a background sweep removes old entries.
- **"Why not a plain dictionary with timestamps?"** Finding the least recent entry would need a scan of all entries, which is O(n) per eviction.

> **Key idea.** The map finds, the list orders. Each structure does one job, and both are O(1).

> **Common mistake.** Forgetting to delete the evicted key from the map, so the cache reports a value it no longer holds.

Complete interview coverage

What the interviewer checks: that you can build an O(1) cache and explain why it has two structures.

A complete answer you should be able to give: Use a hash map from key to node for constant-time lookup, and a doubly linked list ordered by recency for constant-time move and removal. get moves the node to the front; put inserts at the front and evicts the tail when full, removing its key from the map. Make both operations safe with one lock.

Follow-up questions and answers:

- **Q: Why doubly linked?**
  A: Removing a node in the middle needs its previous neighbour, which a singly linked list cannot reach cheaply.
- **Q: How do you add expiry?**
  A: Store an expiry with each value, check it on get, and sweep in the background.

Mistakes that cost points: Forgetting to delete the evicted key from the map.

Level: Intermediate · About 55 minutes

Lesson 39: Vending machine and elevator (state)

Goal

Model a machine whose buttons mean different things in different states, using the State pattern, and reason about the elevator's choice of car.

The idea in plain words

> **Plain words.** Press a product button before inserting money and nothing happens. Insert money and the same button now dispenses. The button did not change; the state did.

Step by step: vending machine

1. States: Idle, HasMoney. (Add Dispensing and OutOfStock in a fuller design.)
2. Every state implements the same actions: insert, select. Actions a state does not allow raise an error.
3. Transitions happen inside the states: Idle moves to HasMoney after a coin; HasMoney moves to Idle after a sale.
4. Stock and credit live in the machine. States only decide what is allowed.

Code (verified)

```
class State:
    def insert(self, m, coin): raise RuntimeError("not allowed now")
    def select(self, m, code): raise RuntimeError("not allowed now")
class Idle(State):
    def insert(self, m, coin):
        m.credit += coin; m.state = HasMoney()
class HasMoney(State):
    def insert(self, m, coin): m.credit += coin
    def select(self, m, code):
        price = m.stock[code][1]
        if m.stock[code][0] == 0: raise RuntimeError("sold out")
        if m.credit < price: raise RuntimeError(f"need {price - m.credit} more")
        m.stock[code][0] -= 1; m.credit -= price
        m.state = Idle()
        return f"dispensed {code}, change {m.credit}"
class Machine:
    def __init__(self, stock):
        self.stock, self.credit, self.state = stock, 0, Idle()
    def insert(self, coin): self.state.insert(self, coin)
    def select(self, code): return self.state.select(self, code)

vm = Machine({"A1": [1, 150]})
print("select before coin ->", end=" ")
try: vm.select("A1")
except RuntimeError as e: print(e)
vm.insert(100); print("insert 100 ->", vm.state.__class__.__name__)
try: vm.select("A1")
except RuntimeError as e: print("select ->", e)
vm.insert(100)
print(vm.select("A1"))
```

Output from the run

```
select before coin -> not allowed now
insert 100 -> HasMoney
select -> need 50 more
dispensed A1, change 50
```

Elevator: the decision

For a new hall call, score each car. A car that is idle costs its distance to the floor. A car moving toward the floor in the same direction, and not yet past it, costs the same distance. A car moving the other way costs the distance plus a penalty, because it must finish its run first. Assign the request to the lowest score. Keep stops in sorted sets so the car serves them in order.

Interview follow-ups

- **"Add an OutOfStock state."** Add one class. The transitions into it are the only edits.
- **"Why not one if-else on a state string?"** Each new state touches every method. The State pattern confines the change to one class.
- **"How do elevators avoid starvation?"** Serve requests in the chosen direction first, then reverse (the SCAN algorithm). A request that waits too long can get a priority boost.

> **Key idea.** Each state decides what is allowed; the context holds the data.

> **Common mistake.** A long if-else on a state string spread across methods. It is the same bug as an unclear state machine in production.

Complete interview coverage

What the interviewer checks: that you can model behaviour that depends on state and reason about an elevator's assignment.

A complete answer you should be able to give: Give each state its own class that implements the allowed actions, and let transitions happen inside the states. The machine holds stock and credit, so state classes stay small. For an elevator, score each car by distance, direction and whether it is moving toward the request, and assign the lowest score; serve stops in direction order to avoid starvation.

Follow-up questions and answers:

- **Q: Where should stock live?**
  A: In the machine, since it is shared data rather than a property of one state.
- **Q: How do you add a new state?**
  A: Add one class; only the transitions into and out of it change.

Mistakes that cost points: A long if-else on a state string spread across methods.

Level: Intermediate · About 55 minutes

Lesson 40: Splitwise (debt simplification)

Goal

Record shared expenses and reduce many small debts to the fewest payments needed to settle everything.

The idea in plain words

> **Plain words.** Ann is owed 20 and Ben owes Cara 20 and Ann owes Ben 20 in the same week. You do not need three payments. Netting shows that Cara can pay Ann directly, and Ben is out of it.

Step by step

1. For each expense, add to the payer's balance and subtract each person's share. Work in integer cents.
2. After all expenses, each person has one net number: positive means they are owed money, negative means they owe.
3. Put creditors and debtors in two max-heaps, keyed by amount.
4. Repeatedly pair the largest debtor with the largest creditor. Pay the smaller amount. Put back whoever still has a balance.
5. Stop when one side is empty. The totals always match, because the balances sum to zero.

Worked example

Ann pays 30.00 for dinner shared by three (Ann, Ben, Cara), so each owes 10.00. Ben pays 6.00 for a drink shared by Ben and Cara, so each owes 3.00. Net balances: Ann paid 30.00 and owes 10.00, so she is owed 20.00. Ben paid 6.00 and owes 13.00, so he owes 7.00. Cara owes 13.00 and paid nothing, so she owes 13.00. Check: 20.00 minus 7.00 minus 13.00 is zero. Cara pays Ann 13.00 and Ben pays Ann 7.00: two payments, not four.

Code (verified)

```
def simplify(balances):
    """balances: name -> net cents (positive = is owed). Returns list of (payer, payee, cents)."""
    creditors = [[-v, k] for k, v in balances.items() if v > 0]  # heap of (-amount, name)
    debtors = [[v, k] for k, v in balances.items() if v < 0]
    heapq.heapify(creditors)
    heapq.heapify(debtors)
    payments = []
    while creditors and debtors:
        c_amt, c_name = heapq.heappop(creditors)
        d_amt, d_name = heapq.heappop(debtors)
        c_amt, d_amt = -c_amt, -d_amt
        pay = min(c_amt, d_amt)
        payments.append((d_name, c_name, pay))
        if c_amt - pay > 0: heapq.heappush(creditors, [-(c_amt - pay), c_name])
        if d_amt - pay > 0: heapq.heappush(debtors, [-(d_amt - pay), d_name])
    return payments

# Ann paid 3000 for dinner shared by 3 -> Ann +2000, Ben -1000, Cara -1000 ; Ben paid 600 for 2 (Ben, Cara) -> Ben +300,
Cara -300
bal = {"Ann": 2000, "Ben": -1000 + 300, "Cara": -1000 - 300}
print("balances:", bal)
for d, c, amt in simplify(bal):
    print(f"{d} pays {c} {amt/100:.2f}")
```

Output from the run

```
balances: {'Ann': 2000, 'Ben': -700, 'Cara': -1300}
Cara pays Ann 13.00
Ben pays Ann 7.00
```

Interview follow-ups

- **"How many payments at most?"** n minus 1 for n people with nonzero balances. Each payment settles at least one person fully.

- **"Why are the balances netted first?"** Netting removes intermediate debts. Simplifying pairs before netting can produce more payments.
- **"Why integer cents?"** Floating-point rounding can leave a balance of 0.01 that never clears (Lesson 23).
- **"Does this always give the minimum number of payments?"** Finding the true minimum is NP-hard in general, so the greedy approach is the practical answer. Say so, and mention it is optimal in common cases.

> **Key idea.** Work with net balances, not individual IOUs.

> **Common mistake.** Using floating-point money.

Complete interview coverage

What the interviewer checks: that you can reduce many debts to few payments and keep money exact.

A complete answer you should be able to give: Store each expense in integer cents, compute each person's net balance, and split the people into creditors and debtors. Repeatedly match the largest debtor with the largest creditor and pay the smaller amount. The result uses at most n minus 1 payments. Note that the true minimum is hard in general, so the greedy method is the practical answer.

Follow-up questions and answers:

- **Q: Why net first?**
  A: Netting removes intermediate debts that a pairwise approach would keep.
- **Q: Why not floats?**
  A: Rounding leaves residual cents that never clear.

Mistakes that cost points: Floating-point money; simplifying pairs before netting.

Level: Intermediate to advanced · About 60 minutes

Lesson 41: Movie ticket booking

Goal

Let users pick seats for a show, hold them briefly, and confirm payment without double booking or stuck seats.

Step 1: Clarify

- One show, many seats. Can a user pick seats across rows? Assume yes.
- Hold duration: 5 minutes. A held seat is free again after expiry.
- Payment fails or is late: the hold must not be confirmed after expiry.

Step 2: The seat state machine

Each seat is free, held (by one user, until a time), or booked. Only these transitions exist: free to held, held to booked, held to free (on expiry or cancel), and held to free when a hold expires. Booked is final.

Step 3: All-or-nothing holds

A user selecting A1, A2 and A3 must get all three or none. The hold checks every seat and then writes every seat, under one lock. A partial hold would leave the user with an unusable selection.

Step 4: Confirm only if the hold is still yours

Confirmation checks that the seat is still held by this user and has not expired. If the payment finishes late, the confirm fails, and the user must select again. This stops a slow payment from taking seats that someone else has since been offered.

Code (verified)

```
import time as _t
class Seats:
    def __init__(self):
        self.status = {}              # seat -> (state, holder, expires_at)
        self.lock = threading.Lock()
    def hold(self, seats, user, now, ttl=300):
        with self.lock:               # all-or-nothing for the request
            for s in seats:
                st, _, exp = self.status.get(s, ("free", None, 0))
                if st == "booked" or (st == "held" and exp > now):
                    raise RuntimeError(f"{s} unavailable")
            for s in seats:
                self.status[s] = ("held", user, now + ttl)
    def confirm(self, seats, user, now):
        with self.lock:
            for s in seats:
                st, holder, exp = self.status.get(s, ("free", None, 0))
                if not (st == "held" and holder == user and exp > now):
                    raise RuntimeError(f"hold on {s} expired or lost")
            for s in seats:
                self.status[s] = ("booked", user, None)

seats = Seats()
seats.hold(["A1", "A2"], "u1", now=0)
try: seats.hold(["A2", "A3"], "u2", now=10)
except RuntimeError as e: print("u2 hold ->", e)
try: seats.confirm(["A1", "A2"], "u1", now=400)
except RuntimeError as e: print("late confirm ->", e)
seats.hold(["A1", "A2"], "u1", now=400)
seats.confirm(["A1", "A2"], "u1", now=401)
print("A1 status ->", seats.status["A1"][0])
```

Output from the run

```
u2 hold -> A2 unavailable
late confirm -> hold on A1 expired or lost
A1 status -> booked
```

Interview follow-ups

- **"Where should holds live in production?"** Not in process memory: a restart loses them. Use a database with conditional updates, or a store with TTLs (for example, Redis with expiry). Holds must survive a restart or the seats are lost.
- **"How do you scale to a blockbuster opening?"** Partition by show, so each show's seats live on one shard. Expect hot spots and put a waiting room in front.
- **"Who releases expired holds?"** A background job plus a check on every read. The check makes correctness independent of the job's timing.

> **Key idea.** A hold is a lease with an expiry. It protects users from each other without locking seats forever.

> **Common mistake.** Holding seats only in application memory.

Complete interview coverage

What the interviewer checks: that you can prevent double booking with holds, expiry and all-or-nothing selection.

A complete answer you should be able to give: Model each seat as free, held with an expiry, or booked. A selection is held all or nothing under one lock or conditional update. Confirmation succeeds only if the hold is still the user's and unexpired, so late payments cannot take seats. Store holds in a durable store so a restart does not lose them, and release expired holds in a background job.

Follow-up questions and answers:

- **Q: How do you scale a blockbuster opening?**
  A: Partition by show and add a waiting room to smooth the spike.
- **Q: Who releases expired holds?**
  A: A background job plus a check on every read, so correctness does not depend on job timing.

Mistakes that cost points: Holding seats only in application memory.

PART 7 · MASTERY

Running the Interview

The 45-minute method, the mistakes to avoid, your own stories, and full mock interviews.

- Lesson 42: The 45-minute method, step by step
- Lesson 43: Recovering from mistakes and pressure
- Lesson 44: Your resume and behavioural stories
- Lesson 45: Mock interviews for each level
- Lesson 46: Final self-assessment

Level: All levels · About 40 minutes

Lesson 42: The 45-minute method, step by step

Goal

Run any system design interview with a clock, so you never run out of time in the middle of the design.

Minutes	Step	What you produce
0 to 5	Requirements	Two to three functional and two to three non-functional requirements; what is out of scope
5 to 8	Estimates	QPS (average and peak), storage per year, bandwidth
8 to 12	Core API and data model	Endpoints and the main tables or documents
12 to 25	High-level design	Diagram: clients, gateway, services, storage, queues, caches
25 to 40	Deep dives	Two or three hard parts, chosen with the interviewer
40 to 45	Wrap-up	Bottlenecks, failure handling, monitoring, next steps

Tips that matter

- Say the time out loud at each step: "I have 5 minutes for estimates."
- Ask one question, then wait. The interviewer's answer decides the design.
- Draw as you talk. Label arrows with the protocol and the data.
- Always end the high-level design with the question "where does this fail?"

> **Key idea.** The deep dive is where you score. Pick the part with the most trade-offs (usually consistency, scale, or failure) and go deep.

> **Common mistake.** Spending 25 minutes on requirements and estimates, then running out of time for the design.

Going deeper

Time check phrases. "We're at 12 minutes; I'll sketch the design now and go deep on the write path at 25." Interviewers relax when they can see you own the clock.

Choosing deep dives. Pick parts where the requirements create tension: consistency for payments, latency for search, fan-out for feeds. Ask the interviewer which part they want to see.

Common shape of a 45-minute answer. Requirements and estimates first (about 8 minutes), a clear diagram by minute 15, and two deep dives that each name the trade-off and the failure case.

Interview follow-up

- **Q: What if the interviewer keeps asking about details you have not thought of?** A: Say so, reason aloud to a sensible answer, and label it as your current best guess.
- **Q: Should you name technologies?** A: Yes, but justify each by the property it provides, not by familiarity.

Complete interview coverage

What the interviewer checks: that you can run the whole 45 minutes with a clock and a clear structure.

A complete answer you should be able to give: Spend about 5 minutes on requirements, 3 on estimates, 4 on the API and data model, 13 on the high-level design, 13 on two deep dives, and 5 on wrap-up with bottlenecks and failure handling. Say the time aloud at each stage, and ask the interviewer which deep dive they want.

Follow-up questions and answers:

- **Q: What if you run out of time?**

A: Shorten the estimates and say which part you would expand next.

- **Q: What is the most scored part?**
  A: The deep dives, where trade-offs and failure handling show depth.

Mistakes that cost points: Over-long requirements; no time for the design.

Level: All levels · About 30 minutes

Lesson 43: Recovering from mistakes and pressure

Goal

Handle the moments when you are stuck, wrong, or interrupted, without losing the interviewer.

Common situations and responses

Situation	Say or do
You don't know a technology	"I haven't used it, but the problem needs X. Here is how I would reason about X." Then reason from first principles.
You made a mistake	"I see a problem with what I drew: the cache can serve stale data. Here is the fix." Own it briefly and move on.
You are stuck	List two options with one trade-off each, then pick one and say why.
The interviewer pushes back	"That's a good point. What would you prefer?" Treat it as a request for a trade-off, not a verdict.

Plain words. Interviewers rarely care that you know every answer. They care whether you can reason in public, notice your own errors, and make a decision.

Key idea. Narrate your reasoning. A wrong idea that is clearly reasoned scores better than a silent guess.

Common mistake. Going silent when stuck. Silence looks like you have stopped working.

Going deeper

Rehearse failure. Practise saying "I was wrong about X, because Y; the fix is Z" in one breath. Strong candidates treat correction as normal engineering.

Handling a hint. If the interviewer says "what about the database failing?", take it as a cue for the next deep dive. Answer, then connect it back to the diagram.

Keeping calm. Pause for three seconds before answering a hard question. A short silence reads as thinking; a rushed answer reads as guessing.

Interview follow-up

- **Q: What if you disagree with the interviewer?** A: Explain your reasoning with one concrete scenario, then accept their direction if they still prefer theirs and move on.

Complete interview coverage

What the interviewer checks: that you handle uncertainty, correction and pressure professionally.

A complete answer you should be able to give: When you do not know something, say so, reason from principles to a best answer, and label it as a current view. When you make an error, name it, explain the fix, and continue. Pause before hard answers, and treat challenges as requests for trade-offs rather than verdicts.

Follow-up questions and answers:

- **Q: What if the interviewer disagrees?**
  A: Give one concrete scenario supporting your view, then adopt their direction if they still prefer it.
- **Q: What if you freeze?**
  A: List two options with one trade-off each and choose one aloud.

Mistakes that cost points: Going silent when stuck; defending a flawed design after it is shown to be wrong.

Level: All levels · About 45 minutes

Lesson 44: Your resume and behavioural stories

Goal

Turn your real project work into short stories you can tell with numbers, and connect each one to a design concept.

Use the STAR format

Situation, Task, Action, Result. Keep each part to one or two sentences. Spend most of the time on Action: the decisions you made and why.

Your stories, from your resume

Project	Design concept it shows	Number to state
Kafka to Druid pipeline	Streaming ingestion, time-series storage	1M+ events per day, sub-second queries
Temporal XML processing	Durable workflows, retries	1M+ files per day
Kafka to Iceberg	Compaction, deduplication, rollup	Load down 40%; up to 10x reduction
SQL query generation service	Caching, query design	Report time down 95%
Distributed cache (Node.js)	Partitioning with consistent hashing, replication	200 virtual nodes; zero invalidation
CamelThar (chat)	Real-time delivery, caching	50 ms chat; sub-10 ms cache hits

Check before any interview. Earlier, the LaTeX resume and the PDF showed different numbers. This course uses the PDF. Confirm each number, because an interviewer may ask how you measured it.

Key idea. Be ready to go one level deeper on any claim. "Why 200 virtual nodes?" is a likely question; answer with the trade-off of balance versus memory.

Going deeper

Behavioural questions to prepare. Tell me about a time you disagreed with a teammate. Tell me about a production incident you handled. Tell me about a project that failed. Tell me about a time you had to learn something quickly. For each, use one story from your own work.

Making numbers credible. Say how you measured it: the dashboard, the test, the sample size. A number without a method invites a follow-up you may not be ready for.

Interview follow-up

- **Q: Tell me about a system you built end to end.** A: Use the STAR structure, then go deep on one technical decision, such as why you chose the queue or the partition key.
- **Q: What would you do differently?** A: Name one real change, the reason, and the result you observed later.

Complete interview coverage

What the interviewer checks: that your stories are real, specific and backed by numbers you can explain.

A complete answer you should be able to give: Prepare four to six stories from your work, each in STAR form with most time on the action and the reasons behind it. Attach a number with its method, and be ready to go one level deeper on any technical claim. Link each story to the design concept it demonstrates.

Follow-up questions and answers:

- **Q: Tell me about a failure.**
  A: Name the failure, your part, what you changed, and what you learned.
- **Q: How did you measure it?**
  A: Name the metric, baseline, method and result.

Mistakes that cost points: Stories without numbers or without your own role.

Level: Beginner, intermediate, advanced · About 60 minutes

Lesson 45: Mock interviews for each level

Goal

Practise full interviews at your level, and score yourself honestly.

Beginner mock: URL shortener

1. Requirements: shorten a URL, redirect, 100M new links a year, redirects fast.
2. Estimate: writes about 3 per second; reads about 300 per second at peak; storage a few hundred GB over five years.
3. API: POST /links, GET /{code}. Table: code, long URL, owner, created time.
4. High level: stateless service, key-value store, cache for hot codes (Lesson 6).
5. Deep dive: how codes are generated without collisions (Lesson 7, Snowflake or base62 counter).

Intermediate mock: chat system

1. Requirements: one-to-one and group chats, online status, message history, delivery guarantees.
2. Estimate: a few million concurrent connections; write rate of messages per second.
3. High level: gateway with WebSocket connections, message service, storage partitioned by conversation, queue for fan-out.
4. Deep dive: ordering inside a conversation, and at-least-once delivery with idempotent clients (Lesson 16).

Advanced mock: payments platform

1. Requirements: charge once, refunds, reconciliation with the bank, 99.99% availability.
2. High level: API, payment state machine (Lesson 23), ledger, processor adapters, outbox to events.
3. Deep dive: a processor timeout, and how reconciliation finds the missing money.
4. Wrap-up: multi-region plan with RTO and RPO stated (Lesson 33).

> **Key idea.** Record each mock. Time yourself, then compare with the answer in the appendix.

Going deeper

How to review a mock. Score yourself on five things: requirements asked, estimate done, design drawn, deep dive quality, and failure handling. Write one improvement for the next mock.

Expected depth by level. Beginner: a correct design with clear components. Intermediate: justified choices and one deep dive. Advanced: multiple deep dives, failure and cost reasoning, and an operational plan.

Interview follow-up

- **Q: How many mocks before an interview?** A: At least five on different problems, with feedback after each. Repeating the same problem builds memory, not skill.

Complete interview coverage

What the interviewer checks: that you can run a full interview and judge your own performance honestly.

A complete answer you should be able to give: Practise a full 45-minute problem at your level, record the session, and score requirements, estimates, design, deep dive and failure handling. Note one improvement for the next session. Repeat on different problems so the method, not one answer, becomes automatic.

Follow-up questions and answers:

- **Q: How many mocks are enough?**
  A: At least five across different problems, with feedback after each.
- **Q: How do I know I am ready?**
  A: You can complete the framework in time, defend trade-offs, and handle a failure question calmly.

Mistakes that cost points: Repeating one problem until it is memorised.

Level: All levels · About 30 minutes

Lesson 46: Final self-assessment

Goal

Check your readiness honestly, find the gaps, and know which lesson to revisit.

Skill	Ready when you can	Revisit
Estimation	Compute QPS and storage in under 3 minutes	Lesson 4
Storage and indexes	Explain B-tree vs LSM and a composite index	Lessons 5, 8
Caching	Explain cache-aside, stampede, and invalidation	Lesson 6
Consistency	Explain quorum and read-your-writes	Lessons 8, 10
Messaging	Explain Kafka partitions, at-least-once, and idempotency	Lessons 16, 17
Resilience	Explain retries with jitter and circuit breakers	Lesson 15
Design at scale	Walk through feeds, ride matching, and payments	Lessons 20, 22, 23
LLD	Build a parking lot and LRU cache from a blank page	Lessons 37, 38

> **Key idea.** If you score "not yet" on any row, revisit that lesson, then do one mock on the same topic.

Going deeper

Use the question bank in the final section as your last check: answer each question aloud in under two minutes, and mark the ones you stumble on. Your stumbling list is the revision plan for the final week.

Interview follow-up

- **Q: What if you are weak in one area on the day?** A: Say so calmly, state what you would check, and reason from the principles in this course.

Complete interview coverage

What the interviewer checks: that you know your strengths and gaps before the interview.

A complete answer you should be able to give: Rate yourself on estimation, storage, caching, consistency, messaging, resilience, design at scale and low-level design. For each weak row, revisit the lesson, do one mock on that topic, and answer the question bank aloud. Your remaining weak rows form the plan for the final week.

Follow-up questions and answers:

- **Q: What if I am weak in one area on the day?**
  A: Say so calmly, state what you would check, and reason from the principles.
- **Q: When should I stop studying?**
  A: When your weakest rows are at least acceptable and you can explain every resume claim.

Mistakes that cost points: Re-reading everything instead of targeting the weak rows.

APPENDIX

Cheat Sheets and Glossary

Use this section during revision.

- Numbers and formulas cheat sheet
- Glossary

Reference

Appendix B: Numbers and formulas cheat sheet

Item	Value or formula
Seconds per day	86,400 (round to 10^5 for quick math)
Average QPS	daily actions / 86,400
Peak QPS	2 to 3 times average
Availability 99.9%	about 43 minutes per 30 days; 8.8 hours per year
Availability 99.99%	about 4.3 minutes per 30 days; 52 minutes per year
Cache average latency	h × t_cache + (1 − h) × t_db
Quorum overlap	R + W > N
Retry amplification	3 layers × 3 retries = 27 times the load
Snowflake ID	41-bit time, 10-bit machine, 12-bit sequence
Token bucket refill	tokens added per second; allows a burst up to capacity
Round trip across an ocean	about 100 to 150 ms
Read from memory vs disk	memory is roughly 100,000 times faster than a disk seek

Tip. Memorise the rows you use in estimates. The rest you can look up in the latency ladder (Lesson 4).

Appendix C: Glossary

Term	Meaning
Cache-aside	App reads the cache, falls back to the database, and fills the cache.
CDN	Servers near users that cache content.
Circuit breaker	Stops calls to a failing dependency for a while.
Consistent hashing	Maps keys and servers on a ring so few keys move when servers change.
CQRS	Separate models for writes and reads.
Error budget	The failure allowed by an SLO over a period.
Fencing token	An increasing number that rejects writes from stale holders.
Idempotency	Doing an operation twice has the same effect as once.
Idempotency key	Client-chosen ID that lets the server recognise a repeat.
Outbox	Event saved in the same transaction as the change, then published.
Partition	A slice of data or a log, handled by one node.
Quorum	Enough replicas answering to guarantee overlap.
RPO / RTO	How much data you may lose / how long recovery may take.
Saga	A sequence of steps, each with a compensation to undo it.
SLO	A target for a service level, such as 99.9% of requests under 300 ms.
Watermark	The point in event time before which no late events are expected.

FINAL SECTION

Interview Question Bank

The questions interviewers most often ask, grouped by topic, each with a model answer. Answer aloud first, then compare.

- A. Framework and approach
- B. Estimation and numbers
- C. Storage, caching and sharding
- D. Consistency, transactions and replication
- E. Messaging, streaming and resilience
- F. Case-study questions
- G. Low-level design
- H. Behavioural and resume

Reference · Practise aloud, under two minutes each

A. Framework and approach

1. **How do you start a system design interview?**
   Clarify requirements: functional features, scale (users and requests per second), latency and availability targets, and what is out of scope. State assumptions out loud and confirm them.
2. **What do you do when the interviewer gives no numbers?**
   Assume a realistic scale, label the assumption, and show how the design changes if it is off by 10 times.
3. **How do you decide what to deep dive?**
   Pick the part with the most tension between requirements (consistency, latency, cost, failure). Ask the interviewer which part they want to see.
4. **How do you handle a question you cannot answer?**
   Say what you do know, reason from first principles to a candidate answer, and label it as your best current view.
5. **What is the difference between functional and non-functional requirements?**
   Functional says what the system does (post a message, search). Non-functional says how well (latency under 200 ms, 99.9 percent availability, durability).

Reference

B. Estimation and numbers

6. **Estimate queries per second for 500 million daily active users, each doing 20 reads a day.**
   Reads per day: 10 billion. Divide by 86,400: about 115,000 per second on average. Peak at 2 to 3 times: about 230,000 to 350,000 per second.
7. **How much storage for 1 billion posts of 1 KB, kept 5 years, with 3 replicas?**
   1 billion times 1 KB is 1 TB per copy. Five years of growth at a steady rate: about 5 TB. With 3 replicas: about 15 TB. Say the growth assumption.
8. **Why compute peak QPS?**
   Capacity is sized for peak traffic, not average. A system that works at average and fails at peak is an outage.
9. **How do you estimate cache memory?**
   Take the hot working set (often about 20 percent of items cause most reads) and multiply by the average item size, plus overhead.
10. **What is the rough latency of a cross-region round trip?**
    About 100 to 150 ms. A same-data-centre round trip is about 0.5 ms.

C. Storage, caching and sharding

11. **B-tree or LSM tree: which is better?**
    B-trees give fast reads and balanced writes, and suit transactional databases. LSM trees give fast writes by appending and merging, and suit write-heavy workloads. Choose by read and write mix.
12. **When would you not add an index?**
    When the table is small, the column has very few distinct values, or writes are much more frequent than reads for that column. Each index slows writes.
13. **SQL or NoSQL for a new service?**
    SQL when you need joins, multi-row transactions, and flexible queries. NoSQL when access is by a known key at very large scale and the access pattern is fixed.
14. **Explain cache-aside.**
    The app reads the cache first. On a miss it reads the database and fills the cache. On a write it updates the database and deletes the cache key.
15. **Why delete the cache key on write rather than update it?**
    Concurrent writers can leave the older value in the cache if updates race. Deleting forces the next read to load the current value.
16. **What is a cache stampede and how do you stop it?**
    Many requests miss at once on a popular expiring key and overload the database. Stop it with single-flight (one request recomputes), early refresh, and TTL jitter.
17. **Range or hash sharding?**
    Range keeps related keys together and supports range queries, but creates hot ranges. Hash spreads load evenly but loses range queries. Consistent hashing reduces key movement when nodes change.
18. **Why does hash mod N fail when adding a server?**
    Changing N remaps almost every key, causing a cache-wide miss storm and data movement. Consistent hashing moves only about 1 of N keys.
19. **How do you handle a hot key?**
    Split it into sub-keys and aggregate on read, cache it at the edge, or give it a dedicated shard.
20. **What makes a good shard key?**
    High cardinality, even traffic, and present in most queries so each query hits one shard.

D. Consistency, transactions and replication

21. **Explain CAP in one sentence.**
    During a network partition, a system must choose between returning consistent answers and staying available.
22. **What is PACELC?**
    If there is a partition, choose availability or consistency; else, choose latency or consistency. It covers the everyday trade-off too.
23. **What is read-your-writes consistency?**
    After you write, your later reads reflect that write, even if other users may still see older data.
24. **Synchronous or asynchronous replication?**
    Synchronous: no acknowledged write is lost, but writes are slower and a slow replica slows everything. Asynchronous: fast, but recent writes can be lost on failover.
25. **What is split brain and how do you prevent it?**
    Two nodes both believe they are the leader and accept conflicting writes. Prevent it with consensus (a majority to elect a leader) and fencing tokens.
26. **Explain quorum with an example.**
    With N=3 replicas, a write to 2 and a read from 2 always overlap, so a read sees the latest write (R+W greater than N).
27. **What is a lost update?**
    Two transactions read the same value, compute new values, and write; one write overwrites the other. Fix with an atomic update, a row lock, or a version check.
28. **What is write skew?**
    Two transactions check a rule on overlapping data, then write disjoint rows so the rule breaks. Snapshot isolation allows it; serializable prevents it.
29. **Snapshot isolation or serializable: when?**
    Snapshot for most read-heavy work where anomalies are rare. Serializable for rules that span rows, such as booking the last seat or on-call coverage.
30. **How does Raft choose a leader?**
    A follower that times out becomes a candidate, increases the term, and asks for votes. A majority makes it leader. Random timeouts reduce split votes.

E. Messaging, streaming and resilience

31. **Queue or log: what is the difference?**
    A queue removes a message once it is acknowledged. A log keeps messages for a retention period, and consumers track their own offsets so several groups can read and replay.
32. **How do Kafka partitions give ordering?**
    Messages with the same key go to one partition, which is ordered. There is no ordering across partitions.
33. **What is at-least-once delivery and why does it matter?**
    A message may be delivered more than once after a crash before acknowledgement. Handlers must be idempotent so repeats are harmless.
34. **How do you achieve effectively-once processing?**
    At-least-once delivery plus idempotent handling, using an idempotency key stored with the result in one transaction.
35. **What is the outbox pattern?**
    Write the event to an outbox table in the same transaction as the business change. A relay publishes it, so the event is never lost or sent without the change.
36. **Orchestration or choreography for a saga?**
    Orchestration uses a central coordinator and is easier to follow. Choreography uses events between services, with no central point but harder to trace.
37. **What is a compensation?**
    A business action that undoes a completed step, such as refunding a payment. It is not a database rollback.
38. **How do watermarks work in stream processing?**
    A watermark estimates that no earlier event will arrive. When it passes a window's end, the window emits. Later events are late data.
39. **Tumbling, sliding or session window?**
    Tumbling for fixed, non-overlapping periods; sliding for rolling periods; session for activity separated by inactivity gaps.
40. **Why is a retry without jitter dangerous?**
    Many clients retry at the same moments, creating synchronized spikes that keep the failing service down. Full jitter spreads the retries.
41. **What does a circuit breaker do?**
    After repeated failures it opens and fails calls fast for a cool-down period, then half-opens to probe recovery. This protects the caller and the dependency.
42. **What is a retry budget?**
    A cap on the fraction of traffic that can be retries, so retries cannot multiply load during an outage.
43. **How do you rate limit across ten servers?**
    Keep counters in a shared store and make check-and-decrement one atomic operation, so limits hold globally.
44. **Token bucket or fixed window?**
    Token bucket allows controlled bursts and smooths traffic. Fixed window is simpler but allows double bursts at the boundary.

F. Case-study questions

43. **Design a URL shortener. How do you generate codes?**
    Use a Snowflake-style unique ID encoded in base 62, or a counter range per server. Store code to long URL with a unique key, cache hot codes, and choose 301 or 302 with a stated reason.
44. **Design a news feed. Push or pull?**
    Hybrid: push posts to followers for normal accounts, and pull celebrity posts at read time, then merge with the precomputed timeline.
45. **Design typeahead.**
    Offline job computes top suggestions per prefix from query logs; serve from memory sharded by prefix; swap versions; merge a small trending counter at read time.
46. **Design a chat system. How do you order messages?**
    Order within each conversation using a per-conversation sequence. Use client message IDs for idempotent storage, WebSocket gateways, and a directory mapping users to gateways.
47. **Design ride matching.**
    Index drivers by geographic cell, search the rider's cell and neighbours, rank by road ETA, offer to one driver with a timeout, and lock the match with a conditional write.
48. **Design a payment system. How do you avoid double charging?**
    Idempotency keys on every request, a state machine with version checks, the processor's own idempotency with our payment ID, and treating timeouts as unknown until resolved.
49. **How do you reconcile payments?**
    Compare internal records with the bank's settlement file by payment ID, amount and status. Investigate mismatches; never silently correct them.
50. **Design file storage with sync.**
    Split files into content-addressed chunks, upload only missing chunks, and commit a manifest with the version it started from. A conflict creates a copy instead of overwriting.
51. **Design a metrics system. How do you control cost?**
    Limit label cardinality, compress time series, downsample old data, and keep min, max, sum and count so averages stay correct.
52. **Design video streaming.**
    Transcode to several renditions, split into segments listed in a manifest, serve from a CDN, and let the player switch quality by bandwidth.
53. **Design a job scheduler.**
    Store jobs partitioned by next-run time; workers claim jobs with leases; heartbeats extend leases; jobs are idempotent; a fencing version rejects stale workers.
54. **Design a rate limiter for a public API.**
    Per authenticated user with an IP fallback, token bucket in a shared store, atomic script, 429 with Retry-After, and a stated fail-open or fail-closed choice.
55. **How would you design a distributed cache?**
    Consistent hashing with virtual nodes for partitioning, replicas for availability, TTL with jitter, and delete-on-write for invalidation; explain the stale window.
56. **How would you make a system multi-region?**
    Home region per user or tenant, asynchronous replication with stated lag, global routing with health checks, and failover drills that measure RTO and RPO.

G. Low-level design

56. **How do you start an LLD question?**
    Clarify actors, actions, rules and scope. List nouns as classes and verbs as methods. Walk one scenario before writing code.
57. **Explain SOLID briefly.**
    Single responsibility, open/closed, Liskov substitution, interface segregation, dependency inversion. Each limits how far a change spreads.
58. **Composition or inheritance?**
    Prefer composition. Inheritance couples classes tightly; composition swaps behaviour through objects.
59. **Strategy or State?**
    Strategy: the caller chooses the algorithm. State: the object changes its own behaviour as its mode changes.
60. **How do you make a parking lot safe for concurrent entries?**
    Guard spot assignment with a lock (or conditional update) around check-and-assign, keeping the critical section short.
61. **Implement an LRU cache. Why two structures?**
    A hash map finds the key in O(1); a doubly linked list orders by recency with O(1) move and removal. Together they give O(1) get and put.
62. **How do you make the LRU cache thread-safe?**
    One lock around get and put, since get also changes order, so a read-write lock does not help.
63. **How does the vending machine avoid invalid actions?**
    Each state implements the allowed actions; invalid ones raise an error. Stock and credit live in the machine, not the state.
64. **How does an elevator choose a car?**
    Score each car by distance, direction and whether it is moving toward the request, choose the lowest score, and serve stops in direction order.
65. **How do you simplify group debts?**
    Compute net balances, then match the largest debtor with the largest creditor repeatedly. Use integer cents. At most n minus 1 payments.
66. **How do you prevent double booking of a seat?**
    Hold seats with a conditional update and an expiry, all or nothing per request; confirm only if the hold is still yours and unexpired.
67. **Where should seat holds be stored?**
    In a durable store with conditional updates or TTLs, not process memory, so a restart does not lose them.
68. **What is check-then-act and how do you fix it?**
    Two threads check a condition, both pass, and both act. Fix by making check and act one atomic step under a lock or a conditional write.

H. Behavioural and resume

68. **Walk me through your most complex project.**
    Use STAR. Spend most time on one technical decision and why you chose it over the alternative.
69. **Tell me about a production incident.**
    Describe the impact, how you mitigated first, how you found the root cause, and the change you made so it does not recur.
70. **How did you measure the improvement?**
    Name the metric, the baseline, the method and the result, for example report time reduced by 95 percent on the same report set.
71. **Tell me about a disagreement with a teammate.**
    Describe the technical point, how you listened, the data you used, and how you agreed or committed.
72. **What would you do differently in that project?**
    Name one real change, the reason, and what you observed after.
73. **Why should we hire you for this role?**
    Connect two or three specific achievements to what the role needs, with numbers you can explain.